

摘 要

本文档用于详细阐述编译器开发的功能拓展部分以及性能优化部分实现，将分三个大章进行介绍。首先梳理编译器对于程序的整个分析流程，接着介绍功能拓展部分与性能优化部分实现的实现思路与代码细节等内容。最后展示所有的拓展功能测试用例通过情况以及性能优化测试用例的跑分。

目 录

第一章 程序分析、抽象、拆解、重组、检查、翻译与运行.....	1
1.1 项目结构为什么会是如此，他们都是做什么的？	1
第二章 功能拓展部分的实现	3
2.1 总体实现方法论.....	3
2.2 switch case.....	4
2.2.1 实现思路.....	4
2.2.2 代码细节.....	5
2.3 for	11
2.3.1 实现思路.....	11
2.3.2 代码细节.....	11
2.4 while、do_while.....	14
2.4.1 实现思路.....	14
2.4.2 代码细节.....	14
2.5 char	17
2.5.1 实现思路.....	17
2.5.2 代码细节.....	18
2.6 pointer.....	27
2.6.1 实现思路.....	27
2.6.2 代码细节.....	27
2.7 array	38
2.7.1 实现思路.....	38
2.7.2 代码细节.....	39
2.8 struct.....	55
2.8.1 实现思路.....	55
2.8.2 代码细节.....	56
2.9 function	72
2.9.1 实现思路.....	72
2.9.2 代码细节.....	73
第三章 性能优化部分的实现.....	87
3.1 程序控制流程图	87

3.1.1 实现思路.....	87
3.1.2 代码细节.....	87
3.2 数据流分析	96
3.2.1 Reaching Definitions	96
3.2.2 Live Variable Analysis	97
3.2.3 Available Expressions Analysis	98
3.3 ConstantFolding.....	100
3.3.1 实现思路.....	100
3.3.2 代码细节.....	100
3.4 StrengthReduction.....	103
3.4.1 实现思路.....	103
3.4.2 代码细节.....	103
3.5 CopyPropagation.....	105
3.5.1 实现思路.....	105
3.5.2 代码细节.....	106
3.6 CSE	114
3.6.1 实现思路.....	114
3.6.2 代码细节.....	114
3.7 LICM.....	123
3.7.1 实现思路.....	123
3.7.2 代码细节.....	123
3.8 DCE.....	132
3.8.1 实现思路.....	132
3.8.2 代码细节.....	132
3.9 LRU.....	141
3.9.1 实现思路.....	141
3.9.2 代码细节.....	141
3.10Clean	144
3.10.1实现思路.....	144
3.10.2代码细节.....	145
第四章 测试.....	147
4.1 拓展功能测试用例通过情况.....	147
4.1.1 自动测试.....	147

4.1.2 手动测试.....	151
4.2 性能优化测试用例跑分	157
4.2.1 自动测试.....	157
4.2.2 手动测试.....	160
第五章 总结.....	162

第一章 程序分析、抽象、拆解、重组、检查、翻译与运行

1.1 项目结构为什么会是如此，他们都是做什么的？

在进行某一项工作之前，了解该工作的内容由什么组成，工作项目是怎样流转的，各个部分为何存在以及做什么用是必要的，所以下文将厘清编译器中所牵扯到的所有文件的作用与联系，并在此过程中说明项目编译器的工作流程。

- **mini.l**: 对于一个陌生的程序，首先要做的是将连续的文本段拆解抽象成一个个约定好的被使用的字符，语言由字符组成，编写者拼凑字符，分析者也从字符单位上去理解文本所写的内容。`.l`文件就是使用所立下的规则来将程序文本转化为一个一个 `token` 并传到下层。
- **mini.y**: 现在程序文本被拆解成为一个个 `token`，这些 `token` 都是精确的描述或者表示，对于一个语言，通常采用某一些抽象的语法规则来约束沟通方式，在保证使用双方能够精确描述沟通内容与想法的同时，在混沌的沟通环境中建立简单的高扩展高延伸性的被双方认可共同使用的共识。`.y`文件中就包含所设定的语法规则，他负责将传来的 `token` 拼接重组并尝试规约至语法规则之中，一旦被规约至语法规则之中，就可以确定语句的类型，从而将这一句翻译为更加简单低级的中间表示形式。
- **tac.c**: `tac.h` 就不再提，只是用于规定结构体与各文件之间函数互用。`tac`文件则是上一流程中规约到某一语法规则时会使用的文件，将符合语法规则的语句中的信息传入对应的函数，进行信息的记录和处理，最终转化成便于优化的中间表示（这里是三地址码）。
- **obj.c**: `obj.h` 同理不再提，`obj.c` 文件负责将转换好的中间表示（已经优化过的或者没有优化过的）翻译成汇编语言，在 `tac.c` 将程序的三地址码转换出来后，`obj.c` 会使用头三地址码 `tac_first` 作为入口，遍历三地址码完成转换，所以比起 `tac` 与 `obj` 的联系，`tac` 与 `.y` 的耦合性更高，对于每一句高级语言语句的处理，`tac` 与 `.y` 总是协作完成。
- **asm.l,asm.y**: 这两个文件与 `mini.l` 与 `mini.y` 的关系类似，只不过是针对汇编语言的词法与语法分析，最终生成目标文件。
- **machine.c**: 虚拟机文件，使用 `inst.h` 中枚举好的指令集，将通过 `asm` 转换汇编语言得到的指令集一条条执行。
- **opt.c**: 优化文件，同样附带 `opt.h`，负责对 `tac` 文件生成的三地址码进行优化，所有的优化手段集成到 `opt.c` 中的主优化函数，并在 `main.c` 中调用实现工作

流中对三地址码的优化进而得到高效的汇编代码，最后得到优化后的目标文件。

第二章 功能拓展部分的实现

本章节将介绍编译器功能拓展部分中的各个功能的实现思路与代码细节, 将依照表格2-1逐个展开说明。

2.1 总体实现方法论

在对每个功能展开说明其详细的实现思路与代码细节之前, 这里对所有功能的统一实现思路进行说明, 所用功能中每个个体的独有实现思路只是对下表实现思路的精细化与简略化实践。

- **功能定义与行为描述:** 首先, 对于一个需要被实现的功能, 需要明确它的定义, 即“它是什么”、由什么所构成, 以及它的行为描述, 即“它能做什么”、能够与谁联动、被谁调用、如何使用、错误情况有哪些。
- **词法与语法:** 了解要实现的功能是什么以及能做什么后, 就可以投入到项目之前已规定过的框架中来。这一步需要找出此功能牵扯到的所有关键字与符号, 写入到词法文件中建立规则使得编译器能够进行识别。接着对于该功能的每一种相关构造、利用形式写产生式, 写入语法文件中建立规则使得编译器能够对接收到的 token 进行规约。
- **数据结构:** 对于规约到某一种非终结符的 token 集合, 需要记录牵扯到的变量、常量、函数、类型定义的类型、作用域等数据, 此时往往需要设计数据结构存储对应信息, 使得语义层能够使用它们生成对应的中间表示语句。
- **语义分析:** 拥有数据结构后, 就可以在三地址码生成文件中设计对应的产生式处理函数, 在语法层将需要的数据传入函数处理, 通过函数构造中间表示。对于设计函数而言, 构造到三地址码的通用规则如下:
 - 表达式: 先求值再把结果放在临时变量里;
 - 赋值: 右值求出结果 → COPY 或 STORE;
 - 控制流: 统一用“标签 + 条件跳转 + 无条件跳转”拼装控制图。
 - 指针/数组/结构体: 统一先算地址, 再读写 (LOAD/STORE)。
- **汇编代码生成:** 现在, 对于要实现的功能, 在上一步中已产生某一产生式的中间表示, 原有的三地址码转换为汇编代码的支持是不够的, 需要在汇编代码生成文件中补充你之前为了生成三地址码而添加的新三地址码的类型转换的支持函数, 必要时还需要补足原有转换函数的逻辑。对于设计函数而言, 使用通用的指令类型可以满足所有功能的需求。

到最后一步，需要做的事情就全部结束了，接下来汇编代码会被老师提供的 asm 文件转换为可由 machine 虚拟机执行的指令集，最终运行出结果。虽然但是汇编代码能否被正确转换仍需要进行测试与纠错。

下面的功能描述基本不提及词法文件，除非在添加 token 之外还做了改动，不然就不再赘述。

break, continue 功能的实现主要体现在处理 switch case 主体块、for 循环循环体以及 while、do_while 循环体时对 TAC_BREAK 与 TAC_CONTINUE 的重写，故不再单独成章进行说明。

1) switch case	2) for
3) while、do_while	4) char
5) pointer	6) array
7) struct	8) function

表 2-1 功能拓展部分功能列表

2.2 switch case

2.2.1 实现思路

首先, 要规定好功能的明确行为语义，为：对 int/char 常量进行精确匹配，支持 default 与 break，并保留 C 语言的直落（这里做重要部分的描述，实际完整功能与 C 语言一致）。

随后在词法层加入 SWITCH/CASE/DEFAULT/BREAK token，在语法文件 mini.y 中以构造式 switch_statement: SWITCH '(' expression ')' '' setup case_list '' 描述整体结构。

其中 setup 动作为整个 switch 提前生成统一的结束标签（parser_switch_end, 供 break 跳出），case_list/case_item 则把每个 case/default 规约为一个中间表示结点 TAC_CASE: a 为匹配常量（整数或字符，default 用 NULL 表示），b 为该分支的唯一标签，etc 指向该分支的语句主体（无主体则为 NULL）。

语义生成方面，编写 do_switch 语义处理函数，先对 switch(expr) 求值并把结果放入临时 tmp，然后从后往前遍历所有 TAC_CASE 同时构造两条链，分别为分派链和主体块链。分派链对每个带常量的 case 生成比较与条件跳过序列：计算 tmp == 常量，用 IFZ skip 跳过不相等的情形，相等则 GOTO 目标标签；目标标签按是否有主体确定：有主体则跳本 case 标签，无主体则直落到后面最近的有主体

标签，若后续也没有主体则退化到有主体的 `default`，否则再退化到 `switch` 的 `end`。主体块仅为有主体的 `case/default` 生成 LABEL 分支标签 + 主体代码，并把主体中的 `TAC_BREAK` 重写为 `GOTO end_label` 来实现 `break`；刻意不在块尾自动补 `goto end`，以保留直落语义。所有分派都未命中时，统一 `GOTO default_label`（存在时）否则 `GOTO end_label`。

最终形成的 TAC 顺序是“计算开关表达式 → 分派比较链 → 未命中回退跳 → 各 `case/default` 的 LABEL+ 主体 → 统一 `end LABEL`”。

在转换三地址码时不需要添加新的转换函数，表示用的 `TAC_CASE` 只存在于中间态，在语义处理时会分解为通用控制流指令（如 `TAC_EQ/TAC_IFZ/TAC_GOTO/TAC_LABEL` 等）与已有算术/访存指令，最后由汇编器与虚拟机执行。

2.2.2 代码细节

2.2.2.1 mini.y

首先是语法上 `switch` 的整体结构，以及结束标签 `setup`，`setup` 为整个 `switch` 预设了统一的结束标签，从而使得 `break` 能够跳出。

```

1 switch_statement : SWITCH '(' expression ')' '{' setup case_list
2                 '}'
3 {
4     TAC *res = do_switch($3, $7);
5     parser_switch_end = NULL;
6     $$ = res;
7 }
8 ;
9 setup :
10 {
11     SYM *end_label = mk_label(mk_lstr(next_label++));
12     parser_switch_end = end_label;
13     $$ = NULL;
14 }
15 ;

```

Listing 第二章.1 mini.y: `switch_statement` 与 `setup`

`setup` 在进入 `switch` 时创建一个统一的 `end_label`（保存在 `parser_switch_end`），用于把 `break` 重写为跳出 `switch` 的目标，确保任何分支都能正确退出。

接着是 `case` 列表，各个分支会被规约为 `TAC_CASE`，每个 `case/default` 生成一

个 TAC_CASE，包含匹配常量、分支标签以及主体代码。

```

1 case_list : /* empty */ { $$ = NULL; }
2 | case_list case_item { $$ = join_tac($1, $2); }
3 ;
4
5 case_item : CASE INTEGER ':' statement_list
6 {
7     SYM *lbl = mk_label(mk_lstr(next_label++));
8     TAC *body = $4;
9     TAC *case_tac = mk_tac(TAC_CASE, mk_const(atoi($2)), lbl,
10        NULL);
11     case_tac->etc = (void *)body;
12     $$ = case_tac;
13 }
14 | CASE INTEGER ':'
15 {
16     SYM *lbl = mk_label(mk_lstr(next_label++));
17     TAC *case_tac = mk_tac(TAC_CASE, mk_const(atoi($2)), lbl,
18        NULL);
19     case_tac->etc = NULL;
20     $$ = case_tac;
21 }
22 | CASE CHARACTER ':' statement_list
23 {
24     SYM *lbl = mk_label(mk_lstr(next_label++));
25     TAC *body = $4;
26     TAC *case_tac = mk_tac(TAC_CASE, mk_char((int)$2), lbl, NULL);
27     case_tac->etc = (void *)body;
28     $$ = case_tac;
29 }
30 | CASE CHARACTER ':'
31 {
32     SYM *lbl = mk_label(mk_lstr(next_label++));
33     TAC *case_tac = mk_tac(TAC_CASE, mk_char((int)$2), lbl, NULL);
34     case_tac->etc = NULL;
35     $$ = case_tac;
36 }
37 | DEFAULT ':' statement_list

```

```

36 {
37     SYM *lbl = mk_label(mk_lstr(next_label++));
38     TAC *body = $3;
39     TAC *case_tac = mk_tac(TAC_CASE, NULL, lbl, NULL);
40     case_tac->etc = (void *)body;
41     $$ = case_tac;
42 }
43 | DEFAULT ':'
44 {
45     SYM *lbl = mk_label(mk_lstr(next_label++));
46     TAC *case_tac = mk_tac(TAC_CASE, NULL, lbl, NULL);
47     case_tac->etc = NULL;
48     $$ = case_tac;
49 }
50 ;

```

Listing 第二章.2 mini.y: case_list 与 case_item 构造 TAC_CASE

以上会分别覆盖 int 与 char 常量的 case（有主体与无主体两种），以及 default（有主体与无主体）。对每个分支，a 存匹配常量（NULL 表示 default），b 是分支标签，etc 是主体 TAC 链或 NULL。

2.2.2.2 tac.h

在 tac.h 文件中，定义了 TAC_CASE 类型以及 parser_switch_end 的全局引用，以供语义层和后续重写使用。

```

1 #define TAC_CASE 25
2 #define TAC_BREAK 26
3 extern SYM parser_switch_end; /* 统一 switch 结束标签，用于 break
   重写 */

```

Listing 第二章.3 tac.h: TAC_CASE 定义与 parser_switch_end

TAC_CASE 只存在于中间态，不会直接变成机器指令；parser_switch_end 在语义处理中被读取，用于把 break 统一改写为 GOTO end_label。

2.2.2.3 tac.c

核心在 tac.c 的 do_switch 函数中，该函数负责求值、生成分派链、拼接主体块、处理 default 与 break、最终形成一条完整的 TAC 序列。

```

1 static void rewrite_break_only(TAC *code, SYM *l_end)
2 {
3     for (TAC *p = code; p; p = p->prev)
4     {
5         if (p->op == TAC_BREAK)
6         {
7             p->op = TAC_GOTO;
8             p->a = l_end;
9             p->b = p->c = NULL;
10        }
11    }
12 }

```

Listing 第二章.4 tac.c: 重写 break 为跳到统一 end 标签

主体代码里的 TAC_BREAK 被局部重写为 GOTO end_label，保证任何分支中的 break 都能直接离开整个 switch。

```

1 TAC *do_switch(EXP *switch_exp, TAC *case_list)
2 {
3     // tmp = switch_exp
4     SYM *tmp = mk_tmp();
5     TAC *code = mk_tac(TAC_VAR, tmp, NULL, NULL);
6     code = join_tac(code, switch_exp->tac);
7     code = join_tac(code, mk_tac(TAC_COPY, tmp, switch_exp->ret,
8     NULL));
9
10    // end label
11    SYM *end_label = parser_switch_end ? parser_switch_end :
12        mk_label(mk_lstr(next_label++));
13    TAC *lab_end = mk_tac(TAC_LABEL, end_label, NULL, NULL);
14
15
16    TAC *dispatch = NULL; // 判等+跳转链
17    TAC *bodies = NULL; // 仅包含“有主体”的 case/default 块
18
19    SYM *default_label = NULL;
20    TAC *default_body = NULL;
21    int default_has_body = 0;
22
23    // 直落目标：记录最近的“有主体”的标签（从后往前扫描）

```

```

21     SYM *next_body_label = NULL;
22
23     for (TAC *it = case_list; it; it = it->prev)
24     {
25         if (it->op != TAC_CASE)
26             continue;

```

Listing 第二章.5 tac.c: do_switch 求值与统一 end 标签

先把 switch(expr) 的值计算到临时 tmp，再准备统一的 end_label；随后从后往前遍历所有 TAC_CASE，以便直落时定位后面最近的有主体标签。

```

1     SYM *k = it->a;           // 常量，NULL 表示 default
2     SYM *lbl = it->b;         // 该 case 标签
3     TAC *bod = (TAC *)it->etc; // 该 case 的语句链（无主体则为
                                NULL）
4
5     int has_body = (bod != NULL);
6
7     if (k == NULL)
8     {
9         // default
10        default_label = lbl;
11        default_body = bod;
12        default_has_body = has_body;
13
14        if (has_body)
15        {
16            rewrite_break_only(default_body, end_label);
17            TAC *blk = mk_tac(TAC_LABEL, lbl, NULL, NULL);
18            blk = join_tac(blk, default_body);
19            bodies = bodies ? join_tac(bodies, blk) : blk;
20            next_body_label = lbl;
21        }
22        continue;
23    }
24
25    // 分派：支持直落到后面最近的“有主体”的标签；否则
26    default; 否则 end
    SYM *target = has_body ? lbl

```

```

27         : (next_body_label ? next_body_label
28         : (default_has_body ?
29             default_label : end_label));
30
31     SYM *skip = mk_label(mk_lstr(next_label++));
32     EXP *cmp = do_cmp(TAC_EQ, mk_exp(NULL, tmp, NULL),
33         mk_exp(NULL, k, NULL));
34     TAC *cmp_code = cmp->tac;
35     TAC *ifz = mk_tac(TAC_IFZ, skip, cmp->ret, NULL);
36     ifz->prev = cmp_code;
37     TAC *g_tgt = mk_tac(TAC_GOTO, target, NULL, NULL);
38     g_tgt->prev = ifz;
39     TAC *lab_skip = mk_tac(TAC_LABEL, skip, NULL, NULL);
40     lab_skip->prev = g_tgt;
41     dispatch = dispatch ? join_tac(dispatch, lab_skip) :
42         lab_skip;
43
44     // 有主体的 case: 只生成 “label -> body” (不补 goto end)
45     if (has_body)
46     {
47         rewrite_break_only(bod, end_label);
48         TAC *blk = mk_tac(TAC_LABEL, lbl, NULL, NULL);
49         blk = join_tac(blk, bod);
50         bodies = bodies ? join_tac(bodies, blk) : blk;
51         next_body_label = lbl;
52     }
53
54     // 分派未命中: 跳 default 或 end
55     SYM *fall = default_label ? default_label : end_label;
56     TAC *g_fall = mk_tac(TAC_GOTO, fall, NULL, NULL);
57
58     // 汇总: code(求值) -> dispatch -> g_fall -> bodies -> end
59     code = join_tac(code, dispatch);
60     code = join_tac(code, g_fall);
61     code = join_tac(code, bodies);
62     code = join_tac(code, lab_end);
63     return code;

```

62 }

Listing 第二章.6 tac.c: 构造分派链与主体块, 支持直落与 default 回退

分派链对每个 `case(const)` 进行比较 + 条件跳过 + 命中跳转的逻辑; 无主体的 `case` 不会生成自己的 LABEL, 命中时直落到后面最近的有主体标签; 若无可落处则回退至有主体的 `default`, 否则回退到 `end`。所有比较均未命中时, 统一跳到 `default` 或 `end`。最终把求值 → 分派链 → 未命中跳 → 主体块 → 结束标签拼接为一个连续的 TAC 链。

以上为该功能实现的重要代码片段展示与详细说明。

2.3 for

2.3.1 实现思路

首先词法层添加 token, 语法层在 `mini.y` 中以 `for_statement : FOR '(' for_init ';' for_cond ';' for_iter ')' block` 把三个子句与循环体规约出来, 其中 `for_init/for_iter` 允许赋值或函数调用, 也可为空, `for_cond` 为空时默认为常真常量以实现无限循环;

语义层在 `do_for(init, cond, iter, body)` 中完成降级与拼装: 先为循环分配三个关键标签 `l_start` (条件检查起点)、`l_cont` (迭代点) 与 `l_end` (结束点), 然后按 `for` 的等价执行顺序生成一条连续的三地址码链, 也就是把 `init` 放在最前面作为启动代码; 在 `l_start` 处计算并检查 `cond`, 不满足则跳到 `l_end` 退出; 命中后顺序执行 `body`; 接着落到 `l_cont` 执行 `iter`; 最后无条件跳回 `l_start` 重复, 同时将循环体内的 `TAC_CONTINUE` 重写为跳到 `l_cont`, 将 `TAC_BREAK` 重写为跳到 `l_end`, 以确保语义与 C 一致。

转换为汇编代码时不需要添加新的转换函数, 完全用通用的 `TAC_LABEL/TAC_IFZ/TAC_GOTO` 与已有算术/赋值指令表达控制流。

2.3.2 代码细节

2.3.2.1 mini.y

首先是 `for` 语句的总体产生式。

```
1 for_statement : FOR '(' for_init ';' for_cond ';' for_iter ')'
    block { $$=do_for($3, $5, $7, $9); }
2 ;
```

Listing 第二章.7 mini.y: `for_statement` 规约为 `do_for` 的四个入参

语法层将 `for(init; cond; iter) block` 规约为对语义函数 `do_for(init, cond, iter, body)` 的调用，四个入参分别是初始化代码、条件表达式、迭代代码、循环体代码。

接着是三子句产生式（允许为空）。

```

1 for_init : assignment_statement { $$=$1; } | call_statement {
    $$=$1; } | { $$=NULL; } ;
2 for_cond : expression { $$=$1; } | { $$=mk_exp(NULL, mk_const(1),
    NULL); };
3 for_iter : assignment_statement { $$=$1; } | call_statement {
    $$=$1; } | { $$=NULL; } ;

```

Listing 第二章.8 mini.y: `for_init` / `for_cond` / `for_iter` 的产生式

`for_init` 与 `for_iter` 支持赋值或函数调用，也允许为空；`for_cond` 为空时以常真 1 代替，实现无限循环的语义。

2.3.2.2 tac.c

语义处理函数 `do_for`，通过三个控制标签控制循环。

```

1 TAC *do_for(TAC *init, EXP *cond, TAC *iter, TAC *body)
2 {
3     if (cond == NULL)
4         cond = mk_exp(NULL, mk_const(1), NULL);
5
6     SYM *l_start = mk_label(mk_lstr(next_label++));
7     SYM *l_cont = mk_label(mk_lstr(next_label++));
8     SYM *l_end = mk_label(mk_lstr(next_label++));
9
10    for (TAC *p = body; p != NULL; p = p->prev)
11    {
12        if (p->op == TAC_BREAK)
13        {
14            p->op = TAC_GOTO;
15            p->a = l_end;
16            p->b = p->c = NULL;
17        }
18        if (p->op == TAC_CONTINUE)
19        {
20            p->op = TAC_GOTO;
21            p->a = l_cont;

```

```

22         p->b = p->c = NULL;
23     }
24 }
25
26 TAC *code = mk_tac(TAC_LABEL, l_start, NULL, NULL); // start:
27 code = join_tac(code, cond->tac); // cond
28 TAC *ifz = mk_tac(TAC_IFZ, l_end, cond->ret, NULL); // ifz ->
    end
29 ifz->prev = code;
30 code = ifz;
31
32 code = join_tac(code, body); // body
33 TAC *lab_cont = mk_tac(TAC_LABEL, l_cont, NULL, NULL); // cont:
34 lab_cont->prev = code;
35 code = lab_cont;
36
37 code = join_tac(code, iter); // iter
38 TAC *g_start = mk_tac(TAC_GOTO, l_start, NULL, NULL); // goto
    start
39 g_start->prev = code;
40 code = g_start;
41
42 TAC *lab_end = mk_tac(TAC_LABEL, l_end, NULL, NULL); // end:
43 lab_end->prev = code;
44 code = lab_end;
45
46 // 放在最前面的 init
47 return join_tac(init, code);
48 }

```

Listing 第二章.9 tac.c: do_for 降级为通用标签 + 跳转的控制流

语义层为 for 循环分配三个核心标签：l_start（条件检查起点）、l_cont（迭代点，供 continue 跳转）、l_end（结束点，供 break 跳转）。随后按 for 的等价执行顺序拼接一条连续的 TAC 链：将 init 置于最前作为启动代码；在 l_start 处计算并测试 cond，不满足则 IFZ 跳到 l_end；命中后执行 body，再落到 l_cont 执行 iter，最后 GOTO l_start 回到条件检查，实现循环；同时对循环体内的 TAC_CONTINUE 和 TAC_BREAK 分别重写为跳到 l_cont 与 l_end，确保语义与 C 保持一致。

以上为该功能实现的重要代码片段展示与详细说明。

2.4 while、do_while

2.4.1 实现思路

词法层添加 token，语法层在 mini.y 中用 `while_statement : WHILE '(' expression ')' block` 与 `do_while_statement : DO block WHILE '(' expression ')' ';' ;` 把条件表达式和循环体规约出来。

随后语义层使用 `do_while` 与 `do_do_while` 函数生成三地址码。

对于 `while`，降级到基本控制流语句的方式为：先到起始标签，计算并测试条件，为零则跳到结束标签；命中后顺序执行循环体；体尾无条件跳回起始标签；结束处打一个统一的结束标签，并在进入循环体前重写其中的 `TAC_CONTINUE` 为跳到条件检查点，重写 `TAC_BREAK` 为跳到结束标签，保证与 C 语义一致。

对于 `do-while`，只是将 `while` 的检查条件后置：先直接执行循环体，再计算并测试条件；条件为真则跳回体起始标签继续，否则落到结束标签，使得 `do-while` 至少执行一次循环体。

转换为汇编代码时，两者都不需要额外的转换函数，完全用通用的 `TAC_LABEL` / `TAC_IFZ` / `TAC_GOTO` 与已有的算术、赋值、调用等指令表达控制流；

2.4.2 代码细节

2.4.2.1 mini.y

首先是 `while` 语句产生式。

```
1 while_statement : WHILE '(' expression ')' block { $$=do_while($3,
2 ;               $5); }
```

Listing 第二章.10 mini.y: `while_statement` 规约为 `do_while` 的两个入参

语法层将 `while (expression) block` 规约为语义函数 `do_while(cond, body)` 的调用，其中 `$3` 是条件表达式，`$5` 是循环体。

接着是 `do-while` 语句产生式。

```
1 do_while_statement : DO block WHILE '(' expression ')' ';' {
2 ;                 $$=do_do_while($5, $2); }
```

Listing 第二章.11 mini.y: `do_while_statement` 规约为 `do_do_while`

2.4.2.2 tac.c

do_while 用两个标签 l_start/l_end 拼出 while 的控制图：在 l_start 处计算并测试条件（为零则 IFZ 跳到 l_end），命中后执行循环体，再无条件跳回 l_start；同时把循环体中的 TAC_CONTINUE 重写为跳到条件检查点，把 TAC_BREAK 重写为跳到结束标签，确保与 C 语义一致。

```

1 TAC *do_while(EXP *exp, TAC *stmt)
2 {
3     SYM *l_start = mk_label(mk_lstr(next_label++));
4     SYM *l_end = mk_label(mk_lstr(next_label++));
5
6     // 重写 break/continue
7     for (TAC *p = stmt; p != NULL; p = p->prev)
8     {
9         if (p->op == TAC_BREAK)
10        {
11            p->op = TAC_GOTO;
12            p->a = l_end;
13            p->b = p->c = NULL;
14        }
15        if (p->op == TAC_CONTINUE)
16        {
17            p->op = TAC_GOTO;
18            p->a = l_start;
19            p->b = p->c = NULL;
20        }
21    }
22
23    TAC *code = mk_tac(TAC_LABEL, l_start, NULL, NULL); // start:
24    code = join_tac(code, exp->tac);                    // cond
25    code
26    TAC *ifz = mk_tac(TAC_IFZ, l_end, exp->ret, NULL); // ifz ->
27    end
28    ifz->prev = code;
29    code = ifz;
30
31    code = join_tac(code, stmt);                        // body
32    TAC *g_start = mk_tac(TAC_GOTO, l_start, NULL, NULL); // goto

```

```

    start
31  g_start->prev = code;
32  code = g_start;
33
34  TAC *lab_end = mk_tac(TAC_LABEL, l_end, NULL, NULL); // end:
35  lab_end->prev = code;
36  code = lab_end;
37
38  return code;
39 }

```

Listing 第二章.12 tac.c: do_while 将 while 降级为标签 + 条件跳转 + 回跳

do-while 先执行循环体，再计算条件；为真则回跳体起点继续，否则落到统一结束标签。通过重写 TAC_CONTINUE 为跳回 l_start、TAC_BREAK 为跳到 l_end，保证控制流符合 C 语言的“至少执行一次”与跳出/继续的语义。

```

1  TAC *do_do_while(EXP *exp, TAC *stmt)
2  {
3      SYM *l_start = mk_label(mk_lstr(next_label++));
4      SYM *l_cont = mk_label(mk_lstr(next_label++));
5      SYM *l_end = mk_label(mk_lstr(next_label++));
6
7      for (TAC *p = stmt; p != NULL; p = p->prev)
8      {
9          if (p->op == TAC_BREAK)
10         {
11             p->op = TAC_GOTO;
12             p->a = l_end;
13             p->b = p->c = NULL;
14         }
15         if (p->op == TAC_CONTINUE)
16         {
17             p->op = TAC_GOTO;
18             p->a = l_cont;
19             p->b = p->c = NULL;
20         }
21     }
22
23     TAC *code = mk_tac(TAC_LABEL, l_start, NULL, NULL); //

```

```

    start:
24   code = join_tac(code, stmt);                                // body
25   TAC *lab_cont = mk_tac(TAC_LABEL, l_cont, NULL, NULL); // cont:
26   lab_cont->prev = code;
27   code = lab_cont;
28
29   code = join_tac(code, exp->tac);                                // cond code
30   TAC *ifz = mk_tac(TAC_IFZ, l_end, exp->ret, NULL); // ifz ->
    end
31   ifz->prev = code;
32   code = ifz;
33
34   TAC *g_start = mk_tac(TAC_GOTO, l_start, NULL, NULL); // goto
    start
35   g_start->prev = code;
36   code = g_start;
37
38   TAC *lab_end = mk_tac(TAC_LABEL, l_end, NULL, NULL); // end:
39   lab_end->prev = code;
40   code = lab_end;
41
42   return code;
43 }

```

Listing 第二章.13 tac.c: do_do_while 前执行后判断，至少一次

以上为该功能实现的重要代码片段展示与详细说明。

2.5 char

2.5.1 实现思路

首先添加新的基本数据类型 T_CHAR，在 mini.y 的 type_spec 中遇到 CHAR 即把当前声明类型设为 T_CHAR，使变量、参数、结构体字段与 typedef 能正确携带字节型语义；

词法层在 mini.l 用单引号模式识别字符常量，把解析出的字符写入 yylval.character 并返回 CHARACTER 记号；

语法层在 mini.y 的 expression 里将 CHARACTER 规约为表达式节点（通过 mk_char 构造符号），并在 output_statement 支持 OUTPUT CHARACTER 直接输出

字符；

语义层与中间表示统一把 `char` 当作可计算、可比较的值参与 TAC 流，同时确保尺寸与布局按字节处理——在 `tac.c` 的类型尺寸计算中字符为 1 字节，从而影响数组步长与结构体字段偏移；

后端生成中，`obj.c` 针对 `char` 做完整的访存与 I/O 实现：加载/存储区分字节与整型（本地/全局变量用 LDC/STC，整型用 LOD/STO），字符常量作为立即数零扩展装入寄存器，指针解引用与地址读写也按元素类型选择 LDC/STC 或 LOD/STO，输入输出根据目标/源类型选择字符通道（ITC/OTC）或整型通道（ITI/OTI），并在赋值后写通内存以保证经指针可见。

2.5.2 代码细节

2.5.2.1 mini.l

匹配形如 `'a'` 或 `'\n'` 的字符常量；若为反斜杠开头的转义，按常见集合（`\n, \t, \r, \\, \', \0`）映射为实际字符写入 `yylval.character`，最后返回 `CHARACTER` 记号供语法层使用。

```

1  " "([^\n]|\\.)" " {
2      char c;
3      if (yytext[1] == '\\') {
4          switch (yytext[2]) {
5              case 'n': c = '\n'; break;
6              case 't': c = '\t'; break;
7              case 'r': c = '\r'; break;
8              case '\\': c = '\\'; break;
9              case '\'' : c = '\''; break;
10             case '0': c = '\0'; break;
11             default: c = yytext[2]; break;
12         }
13     } else {
14         c = yytext[1];
15     }
16     yylval.character = c;
17     return CHARACTER;
18 }

```

Listing 第二章.14 mini.l: 识别字符常量与常见转义，返回 `CHARACTER` 记号

2.5.2.2 mini.y

遇到 CHAR 时将 `current_type` 设为 `T_CHAR`，并重置相关类型元数据，确保变量、参数以及结构体字段的字段类型语义能在后续规约与语义阶段正确传播。

```

1 type_spec
2 : INT { current_type = T_INT; current_sdef = NULL;
   current_elem_dtype=T_UNDEF; current_elem_sdef=NULL;
   current_ptr_level=0; current_array_len=0; }
3 | CHAR { current_type = T_CHAR; current_sdef = NULL;
   current_elem_dtype=T_UNDEF; current_elem_sdef=NULL;
   current_ptr_level=0; current_array_len=0; }
4 ;

```

Listing 第二章.15 mini.y: type_spec 中的 CHAR 类型声明

字符常量直接被构造成一个表达式节点：`mk_char` 生成符号表示该字面值，`mk_exp` 将其包装为可参与 TAC 流的表达式，从而与 `int` 常量一致地参与运算、比较与控制流。

```

1 expression
2 ...
3 | CHARACTER { $$=mk_exp(NULL, mk_char((int)$1), NULL); }
4 ...

```

Listing 第二章.16 mini.y: expression 中的 CHARACTER 规约为字符常量表达式

OUTPUT CHARACTER 通过 `mk_char` 直接生成字符符号并走统一的 `do_output` 路径，后端可根据类型选择字符通道输出。

```

1 output_statement : OUTPUT IDENTIFIER { $$=do_output(get_var($2)); }
2 | OUTPUT TEXT { $$=do_output(mk_text($2)); }
3 | OUTPUT CHARACTER { $$=do_output(mk_char((int)$2)); }
4 ;

```

Listing 第二章.17 mini.y: OUTPUT CHARACTER 语句

case 'x': 分支被规约为 TAC_CASE 节点，匹配常量用 `mk_char` 生成；有无主体分别通过 `etc` 指针携带（或为空），语义层在 `do_switch` 中统一展开，行为与整型 case 完全一致。

```

1 case_item : CASE CHARACTER ':' statement_list
2 {

```

```

3     SYM *lbl = mk_label(mk_lstr(next_label++));
4     TAC *body = $4;
5     TAC *case_tac = mk_tac(TAC_CASE, mk_char((int)$2), lbl, NULL);
6     case_tac->etc = (void *)body;
7     $$ = case_tac;
8 }
9 | CASE CHARACTER ':'
10 {
11     SYM *lbl = mk_label(mk_lstr(next_label++));
12     TAC *case_tac = mk_tac(TAC_CASE, mk_char((int)$2), lbl, NULL);
13     case_tac->etc = NULL;
14     $$ = case_tac;
15 }

```

Listing 第二章.18 mini.y: CASE CHARACTER 规约为 TAC_CASE

2.5.2.3 tac.h

SYM_CHAR 区分“字符常量”符号；T_CHAR 标记“字节型”语义；mk_char 用于创建承载字符值的符号，供表达式与语义阶段复用。

```

1 #define SYM_CHAR 6 /* 字符常量符号类型 /
2 ...
3 #define T_CHAR 2 / 基本数据类型: char */
4 ...
5 SYM mk_char(int ch); / 构造字符常量符号 */

```

Listing 第二章.19 tac.h: 符号种类与字符常量构造接口

2.5.2.4 tac.c

字符类型在尺寸计算中被明确为 1 字节，这会影响数组元素步长、结构体字段偏移与整体大小等布局结果。下面展示的是多维数组类型结点的尺寸函数，项目中还存在 size_of_dtype 等通用尺寸函数，它们与此保持一致的字节宽度约定。

```

1 static int size_of_tnode(TNode *tn)
2 {
3     if (!tn)
4         return 0;
5     switch (tn->kind)
6     {

```

```

7   case T_CHAR:
8       return 1;
9   case T_INT:
10      return 4;
11  case T_STRUCT:
12      return tn->sdef ? tn->sdef->size : 0;
13  case T_ARRAY:
14      {
15          int esz = size_of_tnode(tn->elem);
16          if (esz <= 0)
17              esz = 4;
18          return (tn->len > 0 ? tn->len : 0) * esz;
19      }
20  default:
21      return 4;
22  }
23 }

```

Listing 第二章.20 tac.c: 类型结点尺寸计算中 T_CHAR 为 1

2.5.2.5 obj.c

将寄存器中被修改过的变量写回内存时，按数据类型选择存储指令：char 用字节存储 STC，整型用 STO；区分本地与全局地址基址。

```

1  if (rdesc[r].var->scope == 1) /* local var */
2  {
3      if (rdesc[r].var->dtype == T_CHAR)
4          out_str(file_s, "    STC (R%u+%u),R%u\n", R_BP,
5                  rdesc[r].var->offset, r);
6      else
7          out_str(file_s, "    STO (R%u+%u),R%u\n", R_BP,
8                  rdesc[r].var->offset, r);
9  }
10 else /* global var */
11 {
12     out_str(file_s, "    LOD R%u,STATIC\n", R_TP);
13     if (rdesc[r].var->dtype == T_CHAR)
14         out_str(file_s, "    STC (R%u+%u),R%u\n", R_TP,
15                 rdesc[r].var->offset, r);
16 }

```

```

13     else
14         out_str(file_s, "    STO (R%u+%u),R%u\n", R_TP,
15                 rdesc[r].var->offset, r);

```

Listing 第二章.21 obj.c: asm_write_back 中区分 T_CHAR, 使用 STC

字符常量作为立即数零扩展写入寄存器；访问变量时，char 用字节加载 LDC，整型用字加载 LOD，并区分局部/全局基址。

```

1 case SYM_CHAR:
2     /* 字符常量也作为立即数装入寄存器（零扩展） */
3     out_str(file_s, "\tLOD R%u,%u\n", r, (unsigned)(s->value &
4         0xFF));
5     break;
6 case SYM_VAR:
7     if (s->scope == 1) /* local var */
8     {
9         if (s->dtype == T_CHAR)
10        {
11            if ((s->offset) >= 0)
12                out_str(file_s, "    LDC R%u,(R%u+%d)\n", r, R_BP,
13                    s->offset);
14            else
15                out_str(file_s, "    LDC R%u,(R%u-%d)\n", r, R_BP,
16                    -(s->offset));
17        }
18        else
19        {
20            if ((s->offset) >= 0)
21                out_str(file_s, "    LOD R%u,(R%u+%d)\n", r, R_BP,
22                    s->offset);
23            else
24                out_str(file_s, "    LOD R%u,(R%u-%d)\n", r, R_BP,
25                    -(s->offset));
26        }
27    }
28    else /* global var */
29    {
30        out_str(file_s, "    LOD R%u,STATIC\n", R_TP);

```

```

26     if (s->dtype == T_CHAR)
27         out_str(file_s, "    LDC R%u,(R%u+%d)\n", r, R_TP,
                s->offset);
28     else
29         out_str(file_s, "    LOD R%u,(R%u+%d)\n", r, R_TP,
                s->offset);
30 }
31 break;

```

Listing 第二章.22 obj.c: asm_load 中处理 SYM_CHAR 与 T_CHAR 变量

对指针的解引用读取/写入，按目标或值的类型判断是否是字符元素：字符用 LDC/STC，否则用 LOD/STO。这样数组元素、字符指针等都能以 1 字节对齐正确访问。

```

1 static void asm_load_ptr(SYM *dst, SYM *ptr)
2 {
3     int rp = reg_alloc(ptr);          /* 指针值所在寄存器 */
4     int rd = alloc_free_reg_except(rp); /* 目标放到与 rp
        不同的寄存器 */
5     int is_char = (dst && dst->dtype == T_CHAR);
6
7     if (is_char)
8         out_str(file_s, "\tLDC R%u,(R%u+0)\n", rd, rp);
9     else
10        out_str(file_s, "\tLOD R%u,(R%u+0)\n", rd, rp);
11
12    rdesc_fill(rd, dst, MODIFIED);
13 }
14 static void asm_store_ptr(SYM *ptr, SYM *val)
15 {
16     int rp = reg_alloc(ptr);          /* 指针值所在寄存器 */
17     int rv = alloc_free_reg_except(rp); /* 值装入与 rp
        不同的寄存器 */
18    asm_load(rv, val);
19    rdesc_fill(rv, val, UNMODIFIED);
20
21    if (val && val->dtype == T_CHAR)
22        out_str(file_s, "\tSTC (R%u+0),R%u\n", rp, rv);
23    else

```

```

24 out_str(file_s, "\tSTO (R%u+0),R%u\n", rp, rv);
25 ...

```

Listing 第二章.23 obj.c: 指针解引用/存储的字节型通道选择

当左侧/右侧是地址值（如数组下标、字段地址计算）时，同样按元素类型选择字节/整型通道，保证以 1 字节精度访问 char 数据。

```

1  /* 从地址加载：避免目标寄存器与地址相同；按目标类型选 LOD/LDC */
2  static void asm_load_addr(SYM *dst, SYM *addr)
3  {
4      int ra = reg_alloc(addr);
5      int rd = alloc_free_reg_except(ra);
6
7      int is_char = (dst && dst->dtype == T_CHAR);
8      if (is_char)
9          out_str(file_s, "    LDC R%u,(R%u+0)\n", rd, ra);
10     else
11         out_str(file_s, "    LOD R%u,(R%u+0)\n", rd, ra);
12
13     rdesc_fill(rd, dst, MODIFIED);
14 }
15 /* 地址 + 值 存储：避免值装载覆盖地址寄存器；按元素类型选 STO/STC
   */
16 static void asm_store_addr_val(SYM *addr, SYM *val)
17 {
18     /* 先固定地址所在寄存器 */
19     int ra = reg_alloc(addr);
20
21     /* 将值加载到与 ra 不同的寄存器，避免覆盖 ra */
22     int rv = alloc_free_reg_except(ra);
23     asm_load(rv, val);
24     rdesc_fill(rv, val, UNMODIFIED);
25
26     /* 按元素类型发射 store 指令：
27        优先使用地址的 elem_dtype（更可靠），否则根据值的 dtype
28        兜底 */
29     int is_char = (addr && addr->elem_dtype == T_CHAR) || (val &&
        val->dtype == T_CHAR);
30     if (is_char)

```

```

30     out_str(file_s, "\tSTC (R%u+0),R%u\n", ra, rv);
31 else
32     out_str(file_s, "\tSTO (R%u+0),R%u\n", ra, rv);
33 ...

```

Listing 第二章.24 obj.c: 已计算地址的加载与存储, 按元素类型选通道

输入输出通道按类型区分: 字符走 ITC/OTC, 整型走 ITI/OTI, 字符串走 OTS; 字符常量 (SYM_CHAR) 按 8 位零扩展放入 R15 再 OTC 输出。

```

1 case TAC_INPUT:
2     r = reg_alloc(c->a);
3     /* 若目标是 char, 则使用字符输入指令 ITC; 否则使用 ITI */
4     if (c->a && c->a->dtype == T_CHAR)
5         out_str(file_s, "\tITC\n");
6     else
7         out_str(file_s, "\tITI\n");
8     out_str(file_s, "\tLOD R%u,R15\n", r); /* R15 是 VM 约定的 I/O
      数据寄存器 */
9     /* 将输入写通到内存, 避免后续在标签/清缓存后从未初始化内存加载
      */
10    rdesc[r].mod = MODIFIED;
11    asm_write_back(r);
12    return;
13
14 case TAC_OUTPUT:
15     if (c->a->type == SYM_VAR)
16     {
17         r = reg_alloc(c->a);
18         out_str(file_s, "\tLOD R15,R%u\n", r);
19         /* 若变量是 char, 使用字符输出 OTC; 否则整型输出 OTI */
20         if (c->a->dtype == T_CHAR)
21             out_str(file_s, "\tOTC\n");
22         else
23             out_str(file_s, "\tOTI\n");
24     }
25     else if (c->a->type == SYM_TEXT)
26     {
27         r = reg_alloc(c->a);
28         out_str(file_s, "\tLOD R15,R%u\n", r);

```

```

29     out_str(file_s, "\tOTS\n");
30 }
31 else if (c->a->type == SYM_INT)
32 {
33     out_str(file_s, "\tLOD R15,%u\n", (unsigned)c->a->value);
34     out_str(file_s, "\tOTI\n");
35 }
36 else if (c->a->type == SYM_CHAR)
37 {
38     out_str(file_s, "\tLOD R15,%u\n", (unsigned)(c->a->value &
39         0xFF));
40     out_str(file_s, "\tOTC\n");
41 }
42 return;

```

Listing 第二章.25 obj.c: TAC_INPUT/TAC_OUTPUT 对 char 的专用通道

变量分配依据 `size_of_symbol` 的结果推进偏移，char 的大小在类型系统中设定为 1 字节，从而在此能够按字节粒度精确分配（对未定型兜底 4 字节）。

```

1 case TAC_VAR:
2     if (scope)
3     {
4         c->a->scope = 1; /* local var */
5         c->a->offset = tof;
6         int bytes = size_of_symbol(c->a);
7         if (bytes == 0)
8             bytes = 4; /* 兜底至少分配 4 字节 */
9         tof += bytes;
10    }
11    else
12    {
13        c->a->scope = 0; /* global var */
14        c->a->offset = tos;
15        int bytes = size_of_symbol(c->a);
16        if (bytes == 0)
17            bytes = 4;
18        tos += bytes;
19    }
20    return;

```

Listing 第二章.26 obj.c: TAC_VAR 使用 size_of_symbol 进行按字节分配

以上为该功能实现的重要代码片段展示与详细说明。

2.6 pointer

2.6.1 实现思路

指针的实现从词法上只需确保能识别-> 并让、& 作为单字符参与解析（既可作运算也可作解引用/取址）；

语法上要在声明里允许 * 修饰符（变量与形参都支持），在表达式里加入一元（解引用）与 &（取地址），并让下标 [] 与成员访问 ./-> 能够在数组衰变为指针的通用情况下工作，同时提供对应的赋值形态（如 *p=...、a[i]=...、p->x=...），目标是把能取到地址的左值统一降解为地址计算 + 间接访存；

语义/中间表示层要引入指针类型，并携带指向元素类型/层级等信息，使指针算术按元素大小缩放（ptr±int、ptr-ptr），数组在表达式位置自动衰变为指针，字段与下标统一先算元素地址，标量再做一次按宽度的 LOAD/STORE，聚合类型（结构体/多维数组）则直接传递地址以便后续继续选择/索引；

后端则据类型信息选择字节/字宽的访存指令（如 char 用 1 字节，int 用字宽），把取地址实现为基址（栈/静态）+ 偏移的地址计算，间接写后注意寄存器与内存一致性（必要时写通或清寄存器绑定），从而保证通过指针可见最新值。

2.6.2 代码细节

2.6.2.1 mini.l

指针相关的复合记号仅需将 -> 作为保留词返回，其他如 *、&、[、]、. 等均由词法规则的默认分支 . 以字符本身回传给语法即可，十分简洁。

```
1 "->" { return ARROW; }
2 ...
3 . { return *yytext; }
```

Listing 第二章.27 mini.l: ARROW 记号与单字节回退

2.6.2.2 mini.y

在变量、参数、结构体字段声明处显式支持指针（* IDENTIFIER），并在 typedef 中提供指针/数组的别名能力；为后续语义层填充 dtype、elem_dtype、ptr_level

等元信息打下基础。

```

1 %token ARROW
2 field_var_unit
3 : IDENTIFIER { $$ =
   field_from_current($1, 0, 0); }
4 | '*' IDENTIFIER { $$ =
   field_from_current($2, 1, 0); }
5 | IDENTIFIER dims { $$ =
   field_from_current_dl($1, 0, $2); }
6 ;
7
8 declaration
9 : type_spec variable_list ';' { $$ = $2; }
10 | TYPEDEF type_spec IDENTIFIER ';' { typedef_add($3,
   current_type, T_UNDEF, 0, 0, current_sdef, NULL); $$ = NULL; }
11 | TYPEDEF type_spec '*' IDENTIFIER ';' { typedef_add($4,
   T_PTR, current_type, 1, 0, NULL, current_sdef); $$ = NULL; }
12 | TYPEDEF type_spec IDENTIFIER '[' INTEGER ']' ';'
13 { typedef_add($3, T_ARRAY, current_type, 0, atoi($5), NULL,
   current_sdef); $$ = NULL; }
14 | type_spec ';' { $$ = NULL; }
15
16 var_unit
17 : IDENTIFIER { $$ = declare_var($1); }
18 | '*' IDENTIFIER { $$ = declare_var_ptr($2, 1); }
19 | IDENTIFIER dims {
20   int n = dl_count($2);
21   int *lens = (int*)malloc(sizeof(int)*n);
22   dl_to_array($2, lens);
23   $$ = declare_var_array_multi_lens($1, lens, n);
24 }
25 ;

```

Listing 第二章.28 mini.y: 结构体字段/变量/typedef中的指针

统一把一元 * 解引用映射到 `do_deref`, & 将通过 `laddr` 链产出地址; 数组下标与字段访问派发到专门的语义函数, 以便在 TAC 层精确控制地址计算以及取值/存值。

```

1 assignment_statement
2 : IDENTIFIER '=' expression { $$ = do_assign(get_var($1), $3); }
3 | '[' expression '=' expression { $$ = do_assign_deref($2, $4); }
4 | expression '[' expression ']' '=' expression { $$ =
    do_assign_index_exp($1, $3, $6); }
5 | expression '.' IDENTIFIER '=' expression { $$ =
    do_assign_field_select($1, $3, 0, $5); }
6 | expression ARROW IDENTIFIER '=' expression { $$ =
    do_assign_field_select($1, $3, 1, $5); }
7 ;
8
9 expression
10 : ' ' expression %prec UMINUS { $$=do_deref($2); }
11 | '&' laddr %prec UMINUS { $$=$2; }
12 | expression '[' expression ']' { $$=do_index($1, $3); }
13 | expression '.' IDENTIFIER { $$=do_field_select($1, $3, 0); }
14 | expression ARROW IDENTIFIER { $$=do_field_select($1, $3, 1); }
15 | IDENTIFIER %prec LOWER_THAN_POSTFIX {
    $$=exp_from_symbol(get_var($1)); }
16 ;
17
18 laddr
19 : IDENTIFIER { $$ = do_addr_var(get_var($1)); }
20 | laddr '[' expression ']' { $$ = addr_index($1, $3); }
21 | laddr '.' IDENTIFIER { $$ = addr_field_select($1, $3, 1); }
22 | laddr ARROW IDENTIFIER { $$ = addr_field_select($1, $3, 1); }
23 ;

```

Listing 第二章.29 mini.y: 表达式与赋值中的指针操作

laddr 不允许通用 expression，而是受限于“地址链”上的后缀（[]、.、->），这避免了与一般值表达式混淆，保证 & 始终落在“地址”的语义域。

2.6.2.3 tac.h

TAC 层通过 T_PTR 与 TAC_ADDR/TAC_LOAD/TAC_STORE 三个核心操作描述取址/按地址取值/按地址存值，同时在符号上携带元素类型与指针层级，便于后端选择字节/字宽指令并实现指针算术缩放。

```

1 #define T_PTR 3

```

```

2 ...
3 #define TAC_ADDR 28 /* a = &b */
4 #define TAC_LOAD 29 /* a = *b (b 为指针值) */
5 #define TAC_STORE 30 /* *a = b (a 为指针值) */

```

Listing 第二章.30 tac.h: 指针类型与 TAC 操作码

```

1 typedef struct sym {
2 ...
3 int elem_dtype; /* 指针/数组的元素类型 */
4 int ptr_level; /* 指针层级 */
5 int array_len;
6 ...
7 } SYM;

```

Listing 第二章.31 tac.h: 符号上携带指针/数组元信息

```

1 int size_of_dtype(int dtype);
2 TAC *declare_var_ptr(char *name, int level);
3 TAC *declare_para_ptr(char *name, int level);
4 TAC *declare_para_array(char name, int len); /* 形参数组退化为指针 */
5
6 EXP *do_addr_var(SYM *var);
7 EXP *do_deref(EXP *ptr_exp);
8 EXP *do_index(EXP *base, EXP *idx);
9 TAC *do_assign_index_exp(EXP *base_exp, EXP *idx, EXP *rhs);
10 TAC *do_assign_deref(EXP *ptr_exp, EXP *rhs);
11 EXP *exp_from_symbol(SYM s); /* 数组在表达式位置衰变为指针 */
12
13 EXP *addr_field_select(EXP *base, char *fname, int is_arrow);
14 EXP *addr_index(EXP *base, EXP *idx);

```

Listing 第二章.32 tac.h: 指针/数组相关语义接口原型

2.6.2.4 tac.c

指针算术的步长取决于元素类型大小；多级指针的“元素”仍是指针本身。

```

1 int size_of_dtype(int dtype) {
2     switch (dtype) {

```

```

3     case T_CHAR: return 1;
4     case T_INT:
5     case T_PTR: return 4;
6     case T_STRUCT: return 0;
7     default: return 4;
8 }
9 }
10 int size_of_symbol_elem(SYM *s) {
11     if (s && s->dtype == T_ARRAY && s->tnode) {
12         TNode *tn = (TNode *)s->tnode;
13         if (tn && tn->elem) {
14             int esz = size_of_tnode(tn->elem);
15             if (esz > 0) return esz;
16         }
17     }
18     if (s && s->dtype == T_PTR && s->ptr_level > 1) {
19         return size_of_dtype(T_PTR);
20     }
21     switch (s->elem_dtype) {
22         case T_CHAR: return 1;
23         case T_INT:
24         case T_PTR: return 4;
25         case T_STRUCT: return s->elem_sdef ? s->elem_sdef->size :
26             0;
27         default: return 4;
28     }
29 }

```

Listing 第二章.33 tac.c: 基本尺寸与符号尺寸

```

1 EXP *do_bin(int binop, EXP *exp1, EXP exp2) {
2     if ((binop == TAC_ADD || binop == TAC_SUB) &&
3         ((exp1->ret && exp1->ret->dtype == T_PTR) || (exp2->ret &&
4             exp2->ret->dtype == T_PTR))) {
5         /* 归一为 ptr (+|-) int 或 ptr - ptr */
6         ...
7     else if (exp1->ret->dtype == T_PTR && exp2->ret->dtype ==
8         T_PTR && binop == TAC_SUB) {
9         / ptr - ptr => (ptrdiff)/sizeof(elem) */

```

```

8   int esz;
9   if (ptr->ret->ptr_level > 1) esz = size_of_dtype(T_PTR);
10  else if (ptr->ret->tnode && ((TNode *)ptr->ret->tnode)->kind
        == T_ARRAY)
11      esz = size_of_tnode((TNode *)ptr->ret->tnode);
12  else
13      esz = (ptr->ret->elem_dtype == T_STRUCT)
14      ? (ptr->ret->elem_sdef ? ptr->ret->elem_sdef->size : 0)
15      : size_of_dtype((ptr->ret->elem_dtype == T_UNDEF) ? T_INT :
        ptr->ret->elem_dtype);
16  ...
17  }
18  ...
19  }
20  ...
21  }

```

Listing 第二章.34 tac.c: 指针算术 (ptr±int、ptr-ptr)

指针参数与变量通过专用声明 API 设置 dtype、elem_dtype、ptr_level 等；数组名在表达式上下文中自动“衰变”为指针，便于统一后续指针运算。

```

1  TAC *declare_var_ptr(char *name, int level) {
2      SYM *v = mk_var(name);
3      v->dtype = T_PTR; v->ptr_level = level;
4      v->elem_dtype = current_type;
5      v->elem_sdef = (current_type == T_STRUCT) ? current_sdef :
        NULL;
6      v->sdef = NULL;
7      return mk_tac(TAC_VAR, v, NULL, NULL);
8  }
9  EXP *exp_from_symbol(SYM *s) {
10     if (s->dtype == T_ARRAY) {
11         EXP e = do_addr_var(s); /* 数组名作值时 -> 地址 (指针) */
12         if (s->tnode) {
13             e->ret->tnode = ((TNode *)s->tnode)->elem;
14             if (e->ret->tnode && ((TNode *)e->ret->tnode)->kind ==
                T_ARRAY)
15                 e->ret->elem_dtype = T_ARRAY;
16         }

```

```

17     return e;
18 }
19 return mk_exp(NULL, s, NULL);
20 }

```

Listing 第二章.35 tac.c: 指针/数组的声明与表达式衰变

将地址的构造与值的装载/存储分离：先统一求得地址（do_addr_var、addr_index、compute_field_addr），再根据上下文决定 TAC_LOAD 或 TAC_STORE，从而覆盖 *p、p[i]、s.f/p->f 等场景。

```

1 EXP *do_addr_var(SYM *var) {
2     int elem = (var->dtype == T_ARRAY) ? var->elem_dtype :
        var->dtype;
3     SDEF *esd = (elem == T_STRUCT) ? ((var->dtype == T_ARRAY) ?
        var->elem_sdef : var->sdef) : NULL;
4     SYM *ret = mk_tmp_typed(T_PTR, elem, NULL, esd);
5     TAC *decl = mk_tac(TAC_VAR, ret, NULL, NULL);
6     TAC *addr = mk_tac(TAC_ADDR, ret, var, NULL);
7     addr->prev = decl;
8     return mk_exp(NULL, ret, addr);
9 }
10 EXP *do_deref(EXP ptr_exp) {
11     ...
12     if (level > 1) { /* 返回指针值，层数减一 */
13         SYM *ret = mk_tmp_typed(T_PTR, leaf, NULL,
            ptr_exp->ret->elem_sdef);
14         ret->ptr_level = level - 1;
15         ...
16         TAC *ld = mk_tac(TAC_LOAD, ret, ptr_exp->ret, NULL);
17         ...
18     } /* *ptr -> LOAD (多级指针保持指针语义，单级指向标量则取值) */
19     if (leaf == T_STRUCT)
20         error("cannot dereference struct as rvalue; use
            '->field'");
21     SYM *ret = mk_tmp_typed((leaf == T_UNDEF) ? T_INT : leaf,
        T_UNDEF, NULL, NULL);
22     ...
23     TAC *ld = mk_tac(TAC_LOAD, ret, ptr_exp->ret, NULL);
24     ...

```

25 }

Listing 第二章.36 tac.c: 取变量地址与解引用

```

1 EXP *do_index(EXP *base, EXP *idx) {
2     EXP base_ptr = base;
3     if (base->ret->dtype == T_ARRAY) { /* 数组名衰变为指针 */
4         base_ptr = do_addr_var(base->ret);
5         ...
6     }
7     else if (base->ret->dtype != T_PTR) {
8         error("indexing non-pointer/non-array");
9     }
10    /* 计算元素大小 esz (多级指针/多维数组/结构体均处理) */
11    int elem = (base_ptr->ret->elem_dtype == T_UNDEF) ? T_INT :
12              base_ptr->ret->elem_dtype;
13    int esz = ...; if (esz <= 0) esz = 4;
14    /* 计算元素地址 addr = base_ptr + idx*esz */
15    ...
16    /* 若元素为结构体或数组，则返回“地址”，以便后续 . / []
17       使用；否则 LOAD 取值 */
18    if (elem == T_STRUCT || (base_ptr->ret->tnode && ((TNode
19        *)base_ptr->ret->tnode)->kind == T_ARRAY)) {
20        ...
21        return mk_exp(NULL, addr, add);
22    } else {
23        SYM *ret = mk_tmp_typed(elem, T_UNDEF, NULL, NULL);
24        ...
25        TAC *ld = mk_tac(TAC_LOAD, ret, addr, NULL);
26        ...
27    }
28 }

```

Listing 第二章.37 tac.c: 下标访问（计算地址后按需 LOAD/返回地址）

```

1 TAC *do_assign_deref(EXP *ptr_exp, EXP *rhs) {
2     TAC *store = mk_tac(TAC_STORE, ptr_exp->ret, rhs->ret, NULL);
3     store->prev = join_tac(ptr_exp->tac, rhs->tac);
4     return store;
5 }

```

```

6 TAC *do_assign_index_exp(EXP *base_exp, EXP *idx, EXP rhs) {
7     /* 与 do_index 同步地计算 addr 后发出 TAC_STORE */
8     ...
9     TAC *store = mk_tac(TAC_STORE, addr, rhs->ret, NULL);
10    store->prev = join_tac(addr, rhs->tac);
11    return store;
12 }
13 TAC *do_assign_field_select(EXP *base, char *fname, int is_arrow,
14                             EXP *rhs) {
15     EXP *addr = compute_field_addr(base, fname, is_arrow);
16     TAC *store = mk_tac(TAC_STORE, addr->ret, rhs->ret, NULL);
17     store->prev = join_tac(addr->tac, rhs->tac);
18     return store;
19 }

```

Listing 第二章.38 =rhs / 字段存储]tac.c: *ptr=rhs / base[idx]=rhs / 字段存储

```

1 /* 统一先算字段地址，再决定 LOAD 或返回指针 */
2 static EXP *compute_field_addr(EXP *base, char *fname, int
3     is_arrow)
4 {
5 }
6 EXP *do_field_select(EXP *base, char *fname, int is_arrow) {
7     EXP *addr = compute_field_addr(base, fname, is_arrow);
8     FIELD *fld = (FIELD *)addr->ret->etc;
9     if (fld && fld->dtype == T_ARRAY) return addr; /*
10         数组字段作值：返回指针（首元素地址） */
11     ...
12     TAC ld = mk_tac(TAC_LOAD, ret, addr->ret, NULL);
13     ...
14 }
15 /* & 字段 / 下标：直接返回地址表达式 */
16 EXP *addr_field_select(EXP *base, char *fname, int is_arrow) {
17     return compute_field_addr(base, fname, is_arrow);
18 }
19 EXP *addr_index(EXP *base, EXP idx) {
20     ... /* 与 do_index 的地址部分一致 */
21 }

```

Listing 第二章.39 tac.c: 字段访问（值）与“字段地址”统一计算

2.6.2.5 obj.c

区分指针变量与已计算出的地址值两种来源。在 TAC_LOAD/TAC_STORE 分发中，若是指针变量使用 `asm_load_ptr/asm_store_ptr`（以寄存器中的地址作为内存操作基址），否则视为原生地址使用 `asm_load_addr/asm_store_addr_val`。按元素类型选择字节指令（LDC/STC）或字宽指令（LOD/STO），并在间接写后清理寄存器描述，防止把陈旧值回写覆盖最新内存。

```

1 static void asm_addr(SYM *dst, SYM var) {
2     int rd = alloc_free_reg_except(-1);
3     if (var->scope == 1)
4         out_str(file_s, " LOD R%u,R%u\n", rd, R_BP);
5     else
6         out_str(file_s, " LOD R%u,STATIC\n", rd);
7     int ro = alloc_free_reg_except(rd);
8     out_str(file_s, " LOD R%u,%d\n", ro, var->offset);
9     out_str(file_s, " ADD R%u,R%u\n", rd, ro); /* rd = base +
        offset */
10    rdesc_fill(rd, dst, MODIFIED);
11 }
12 ...
13 case TAC_ADDR: asm_addr(c->a, c->b); return;

```

Listing 第二章.40 obj.c: &var → 地址构造（TAC_ADDR）

```

1 case TAC_LOAD:
2     /* b 可能是指针变量或已计算出的地址 */
3     if (c->b && c->b->type == SYM_VAR && c->b->dtype == T_PTR)
4         asm_load_ptr(c->a, c->b);
5     else
6         asm_load_addr(c->a, c->b);
7     break;
8 case TAC_STORE:
9     /* a 可能是指针变量或已计算出的地址 */
10    if (c->a && c->a->type == SYM_VAR && c->a->dtype == T_PTR)
11        asm_store_ptr(c->a, c->b);

```

```

12     else
13         asm_store_addr_val(c->a, c->b);
14     break;

```

Listing 第二章.41 obj.c: LOAD/STORE 的两路分发与字节/字宽选择

```

1 static void asm_load_ptr(SYM *dst, SYM *ptr) {
2     int rp = reg_alloc(ptr);
3     int rd = alloc_free_reg_except(rp);
4     int is_char = (dst && dst->dtype == T_CHAR);
5     if (is_char)
6         out_str(file_s, "\tLDC R%u,(R%u+0)\n", rd, rp);
7     else
8         out_str(file_s, "\tLOD R%u,(R%u+0)\n", rd, rp);
9     rdesc_fill(rd, dst, MODIFIED);
10 }
11 static void asm_store_ptr(SYM *ptr, SYM val) {
12     int rp = reg_alloc(ptr);
13     int rv = alloc_free_reg_except(rp);
14     asm_load(rv, val);
15     if (val && val->dtype == T_CHAR)
16         out_str(file_s, "\tSTC (R%u+0),R%u\n", rp, rv);
17     else
18         out_str(file_s, "\tSTO (R%u+0),R%u\n", rp, rv);
19     /* 间接写后回写并清空, 避免脏覆盖 */
20     for (int r = R_GEN; r < R_NUM; r++)
21     {
22         if (rdesc[r].mod) asm_write_back(r);
23         rdesc_clear(r);
24     }
25 }

```

Listing 第二章.42 obj.c: 指针变量解引用读/写（寄存器里有地址）

```

1 static void asm_load_addr(SYM *dst, SYM *addr) {
2     int ra = reg_alloc(addr), rd = alloc_free_reg_except(ra);
3     int is_char = (dst && dst->dtype == T_CHAR);
4     if (is_char)
5         out_str(file_s, " LDC R%u,(R%u+0)\n", rd, ra);
6     else

```

```

7      out_str(file_s, " LOD R%u,(R%u+0)\n", rd, ra);
8      rdesc_fill(rd, dst, MODIFIED);
9  }
10
11 static void asm_store_addr_val(SYM *addr, SYM *val) {
12     int ra = reg_alloc(addr), rv = alloc_free_reg_except(ra);
13     asm_load(rv, val);
14     int is_char = (addr && addr->elem_dtype == T_CHAR) || (val &&
        val->dtype == T_CHAR);
15     if (is_char)
16         out_str(file_s, "\tSTC (R%u+0),R%u\n", ra, rv);
17     else
18         out_str(file_s, "\tSTO (R%u+0),R%u\n", ra, rv);
19     for (int r = R_GEN; r < R_NUM; r++)
20     {
21         if (rdesc[r].mod) asm_write_back(r);
22         rdesc_clear(r);
23     }
24 }

```

Listing 第二章.43 obj.c: 计算得出的地址读/写（避免覆盖地址寄存器）

字符类型走 LDC/STC（字节访存），其余走 LOD/STO（字宽访存），与 `elem_dtype/dtype` 协同。间接写后清空寄存器描述，防脏回写覆盖最新内存；普通赋值也“写通”以保证随后通过指针可见。

以上为该功能实现的重要代码片段展示与详细说明。

2.7 array

2.7.1 实现思路

首先是词法层，直接将单字符的 `[`、`]`、`.`、`&` 等原样返回。

语法层，先明确数组出现的两类位置：声明（变量与结构体字段）、表达式。声明侧为标识符后接一串 `dims`（如 `a[10][20]`），把每个维度规约到一个链表或栈结构，便于在语义阶段构造类型信息；表达式侧定义下标规则 `base '[' index ']`，并将. 字段访问与 `->` 指针字段访问纳入同一优先级的后缀运算，保证组合性（如 `a[i].x[j]`）。

在语义层，确立数组的核心模型是地址 + 步长：为数组变量/字段建立类型结点（对于多维数组，叶子为基础元素类型，外层逐维包裹），提供统一的数组名在值环境中衰变为指向首元素的指针的规则，从而让所有索引都以基地址 + 索引 ×

元素大小的地址计算展开；元素大小的确定策略为：优先用多维类型结点递归计算块大小，其次根据标量/指针/结构体选择固定宽度或由结构体定义的实际大小，多级指针的元素是指针本身，因此步长取指针大小；当索引或字段选择的目标是结构体或子数组，表达式值应返回元素地址（指针），供后续.或继续[]使用；否则生成按地址取值/存值的TAC（例如ADDR获取地址、LOAD按地址读、STORE按地址写），赋值语义统一为先算地址，再发出STORE。

最后在后端，将TAC翻译为具体指令：变量分配时依据类型结点计算总字节数，在栈或静态区线性分配；取变量地址时以帧指针或静态基址加偏移形成地址寄存器；对LOAD/STORE建立两条路径：如果操作数是“指针变量”（寄存器里存放地址），则以该寄存器为内存基址发起读写；如果是“计算出的地址值”，则先把地址装入一个寄存器，再以该寄存器为基址读写；根据元素类型选择字节型指令还是字宽指令（例如char用字节装载/存储，其他用字宽）

2.7.2 代码细节

2.7.2.1 mini.y

维度链 dims 与用于变量的 var_unit IDENTIFIER dims，以及字段列表中的 field_var_unit IDENTIFIER dims，用于构建数组声明的维度信息。

```

1 dims
2 : '[' INTEGER ']' dims          { $$ = dl_new(atoi($2), $4); }
3 | '[' INTEGER ']'              { $$ = dl_new(atoi($2), NULL); }
4 ;
5 var_unit
6 : IDENTIFIER                    { $$ = declare_var($1); }
7 | '*' IDENTIFIER                { $$ = declare_var_ptr($2, 1); }
8 | IDENTIFIER dims               {
9     int n = dl_count($2);
10    int *lens = (int*)malloc(sizeof(int)*n);
11    dl_to_array($2, lens);
12    $$ = declare_var_array_multi_lens($1, lens, n);
13 }
14 ;
15 field_var_unit
16 : IDENTIFIER                    { $$ =
    field_from_current($1, 0, 0); }
17 | '*' IDENTIFIER                { $$ =
    field_from_current($2, 1, 0); }

```

```

18 | IDENTIFIER dims { $$ =
    field_from_current_dl($1, 0, $2); }
19 ;

```

Listing 第二章.44 mini.y: 变量与字段声明中的数组维度

dims 将类似 [10][20] 的维度串解析为链表，var_unit 在出现 IDENTIFIER dims 时调用 declare_var_array_multi_lens 创建多维数组变量。结构体字段声明也支持 IDENTIFIER dims，交由 field_from_current_dl 构建字段类型结点（多维数组）。

表达式的数组索引，字段选择与索引结合（后缀运算优先级已设置），以及赋值到索引位置。

```

1 %left '.' ARROW '[' ']' /* 后缀运算符统一为同一优先级，左结合 */
2 expression
3 ...
4 | expression '[' expression ']' { $$=do_index($1, $3); }
5 | expression '.' IDENTIFIER { $$=do_field_select($1, $3, 0); }
6 | expression ARROW IDENTIFIER { $$=do_field_select($1, $3, 1); }
7 ...
8
9 assignment_statement
10 ...
11 | expression '[' expression ']' '=' expression
12 {
13 $$ = do_assign_index_exp($1, $3, $6);
14 }

```

Listing 第二章.45 mini.y: 表达式中的数组下标与字段访问

索引读取由 do_index 负责，当左边是索引位置时由 do_assign_index_exp 进行通用的地址计算与存储。字段选择（./->）与数组索引组合时，相关语义由 do_field_select 和后续的索引/赋值组合实现。

地址链（构造 &a[i].x 等的地址），数组索引的地址形式：

```

1 laddr
2 : IDENTIFIER
3 {
4     /* &IDENTIFIER 等价于取该变量地址 */
5     $$ = do_addr_var(get_var($1));
6 }

```

```

7 | laddr '[' expression ']'
8 {
9 /* 在已有地址（指针值）基础上，计算元素地址 */
10 $$ = addr_index($1, $3);
11 }
12 | laddr '.' IDENTIFIER
13 {
14     /* 已有的是地址（指针），对后续 .name 当作指针访问处理（等价于
15        ->） */
16     $$ = addr_field_select($1, $3, 1);
17 }
18 | laddr ARROW IDENTIFIER
19 {
20     /* 仅允许在“地址链”上进行 -> 选择，避免与通用 expression
21        冲突 */
22     $$ = addr_field_select($1, $3, 1);
23 }

```

Listing 第二章.46 mini.y: 地址链与数组下标地址

laddr 为可取地址的链式构造，&var 起点为变量地址，随后支持 [idx] 地址偏移、字段地址的获取等，这使复杂表达式的地址计算成为一条链上累积的指针值，最终用于生成 TAC_ADDR/TAC_ADD 等中间代码。

2.7.2.2 tac.h

```

1 #define T_ARRAY 4
2
3 /* 多维数组维度链表类型 */
4 typedef struct dimlist
5 {
6     int len;
7     struct dimlist *next;
8 } DLIST;
9
10 typedef struct sym
11 {
12     ...
13     /* 新增：复合类型信息 */
14     int elem_dtype; /*

```

```

    指针/数组的元素类型 (T_INT/T_CHAR/.../T_STRUCT) */
15  int ptr_level; /* 指针层级 */
16  int array_len; /* 数组长度 (一维), 参数数组按指针处理可置 0
    */
17
18  /* 新增: 数组类型信息 (用于多维数组) */
19  void *tnode; /* 指向内部类型描述结点, 具体结构在 tac.c 中定义
    */
20 } SYM;
21
22 /* 新增接口 (指针/数组) */
23 int size_of_dtype(int dtype);
24 TAC *declare_var_ptr(char *name, int level);
25 TAC *declare_var_array(char *name, int len);
26 TAC *declare_para_ptr(char *name, int level);
27 TAC *declare_para_array(char *name, int len);
28
29 EXP *do_addr_var(SYM *var);
30 EXP *do_deref(EXP *ptr_exp);
31 EXP *do_index(EXP *base, EXP *idx); /* 读取 base[idx] */
32 TAC *do_assign_index(SYM *base_var, EXP *idx, EXP *rhs);
33 TAC *do_assign_index_exp(EXP *base_exp, EXP *idx, EXP *rhs);
34
35 /* 多维数组支持 */
36 /* 通过维度数组声明多维数组变量 (例如 lens=[10,20,30] 表示
    a[10][20][30]) */
37 TAC *declare_var_array_multi_lens(char *name, int *lens, int n);
38 DLIST *dl_new(int len, DLIST *next);
39 int dl_count(DLIST *p);
40 void dl_to_array(DLIST *p, int *out);

```

Listing 第二章.47 tac.h: 数组类型结点与相关语义接口

T_ARRAY 是数组的语义类型。DLIST 表示多维数组的维度链表。SYM 包含数组及元素类型信息、指针层级以及用于多维数组的 tnode。公开的函数接口涵盖数组声明、数组形参退化、索引读取、索引赋值、以及维度链表的构造与转换。

2.7.2.3 tac.c

多维数组类型结点与大小计算 (递归)。

```

1  /* 多维数组类型结点 */
2  typedef struct type_node
3  {
4      int kind;          /* T_ARRAY / T_INT / T_CHAR / T_STRUCT
5                          */
6      int len;           /* 若为 T_ARRAY，则为该维度长度 */
7      struct type_node *elem; /* 元素类型（下一层） */
8      SDEF *sdef;         /* 若为结构体叶子则指向定义 */
9  } TNode;
10
11 static int size_of_tnode(TNode *tn)
12 {
13     if (!tn)
14         return 0;
15     switch (tn->kind)
16     {
17     case T_CHAR:
18         return 1;
19     case T_INT:
20         return 4;
21     case T_STRUCT:
22         return tn->sdef ? tn->sdef->size : 0;
23     case T_ARRAY:
24     {
25         int esz = size_of_tnode(tn->elem);
26         if (esz <= 0)
27             esz = 4;
28         return (tn->len > 0 ? tn->len : 0) * esz;
29     }
30     default:
31         return 4;
32     }
33 }

```

Listing 第二章.48 tac.c: 多维数组类型结点与大小计算

TNode 描述多维数组类型树，最外层是数组结点包裹元素结点，size_of_tnode 递归计算整个数组块大小，遇到未知尺寸则兜底按 4 字节（整型大小）处理。

维度链表的构造与转换。

```

1 DLIST *dl_new(int len, DLIST *next)
2 {
3     DLIST *p = (DLIST *)malloc(sizeof(DLIST));
4     p->len = len;
5     p->next = next;
6     return p;
7 }
8
9 int dl_count(DLIST *p)
10 {
11     int n = 0;
12     for (; p; p = p->next)
13         n++;
14     return n;
15 }
16
17 void dl_to_array(DLIST *p, int *out)
18 {
19     int n = dl_count(p);
20     int k = n - 1;
21     while (p)
22     {
23         out[k--] = p->len;
24         p = p->next;
25     }
26 }

```

Listing 第二章.49 tac.c: 多维数组维度链表

维度链表是按最内层维度在链表头的构造方式反向拼接的（见语法构造），`dl_to_array` 将其转为外到内的维度数组，便于统一构建多维数组的类型结点树（TNode）。

变量与参数的数组声明与多维数组声明。

```

1 /* 一维数组变量声明 */
2 TAC *declare_var_array(char *name, int len)
3 {
4     SYM *v = mk_var(name);

```

```

5     v->dtype = T_ARRAY;
6     v->elem_dtype = current_type;
7     v->array_len = len;
8     v->elem_sdef = (current_type == T_STRUCT) ? current_sdef :
        NULL;
9     v->sdef = NULL;
10    v->tnode = NULL;
11    return mk_tac(TAC_VAR, v, NULL, NULL);
12 }
13
14 /* 多维数组变量声明: lens 为各维长度 (外到内), n 为维数 */
15 TAC *declare_var_array_multi_lens(char *name, int *lens, int n)
16 {
17     if (!lens || n <= 0)
18         return declare_var_array(name, 0);
19
20     SYM *v = mk_var(name);
21     v->dtype = T_ARRAY;
22     v->sdef = NULL;
23     v->elem_sdef = (current_type == T_STRUCT) ? current_sdef :
        NULL;
24
25     /* 构建类型结点: 从叶子 (基础类型) 向外包裹维度 */
26     TNode *leaf = (current_type == T_STRUCT) ?
        tnode_make_leaf(T_STRUCT, current_sdef)
27         : tnode_make_leaf(current_type, NULL);
28     TNode *tn = leaf;
29     for (int i = 0; i < n; ++i)
30     {
31         tn = tnode_make_array(lens[i], tn);
32     }
33     v->tnode = tn; /* 顶层为数组 */
34
35
36     if (n == 1)
37     {
38         v->elem_dtype = current_type;
39     }

```

```

40     else
41     {
42         v->elem_dtype = T_ARRAY;
43     }
44     return mk_tac(TAC_VAR, v, NULL, NULL);
45 }

```

Listing 第二章.50 tac.c: 数组变量/参数声明与多维数组声明

`declare_var_array` 用于一维数组, `declare_var_array_multi_lens` 构建多维数组类型树并在符号上保存, 用于后续大小计算和索引步长计算。

大小计算: 基本类型、符号元素大小、数组整体大小 (含多维数组)。

```

1  int size_of_dtype(int dtype)
2  {
3      switch (dtype)
4      {
5          case T_CHAR:
6              return 1;
7          case T_INT:
8          case T_PTR:
9              return 4;
10         case T_STRUCT:
11             return 0;
12         default:
13             return 4;
14     }
15 }
16
17 int size_of_symbol_elem(SYM *s)
18 {
19     /* 多维数组支持: 若元素是数组, 使用 tnode 递归计算元素块大小 */
20     if (s && s->dtype == T_ARRAY && s->tnode)
21     {
22         TNode *tn = (TNode *)s->tnode;
23         if (tn && tn->elem)
24         {
25             int esz = size_of_tnode(tn->elem);
26             if (esz > 0)
27                 return esz;

```

```
28     }
29 }
30 /* 多级指针：其“元素”是指针本身（步长为指针大小） */
31 if (s && s->dtype == T_PTR && s->ptr_level > 1)
32 {
33     return size_of_dtype(T_PTR);
34 }
35 switch (s->elem_dtype)
36 {
37 case T_CHAR:
38     return 1;
39 case T_INT:
40 case T_PTR:
41     return 4;
42 case T_STRUCT:
43     return s->elem_sdef ? s->elem_sdef->size : 0;
44 default:
45     return 4;
46 }
47 }
48
49 int size_of_symbol(SYM *s)
50 {
51     if (!s)
52         return 0;
53     switch (s->dtype)
54     {
55 case T_CHAR:
56     return 1;
57 case T_INT:
58 case T_PTR:
59     return 4;
60 case T_STRUCT:
61     return s->sdef ? s->sdef->size : 0;
62 case T_ARRAY:
63     {
64         if (s->tnode)
65         {
```

```

66         int bytes = sizeof_tnode((TNode *)s->tnode);
67         return bytes > 0 ? bytes : 4;
68     }
69     int esz = sizeof_symbol_elem(s);
70     int n = s->array_len > 0 ? s->array_len : 0;
71     int bytes = esz * n;
72     return bytes > 0 ? bytes : 4;
73 }
74 default:
75     return 4;
76 }
77 }

```

Listing 第二章.51 tac.c: 符号元素大小与整体大小计算

字节大小用于栈/静态区分配与索引步长计算, 多维数组和指针层级影响元素尺寸。

数组名在表达式位置的“指针衰变”, 多维数组的元素类型传播。

```

1  /* 数组名在表达式位置衰变为指针 */
2  EXP *exp_from_symbol(SYM *s)
3  {
4      if (s->dtype == T_ARRAY)
5      {
6          EXP *e = do_addr_var(s);
7          if (s->tnode)
8          {
9              e->ret->tnode = ((TNode *)s->tnode)->elem;
10             if (e->ret->tnode && ((TNode *)e->ret->tnode)->kind ==
11                 T_ARRAY)
12                 e->ret->elem_dtype = T_ARRAY;
13             }
14             return e;
15         }
16         return mk_exp(NULL, s, NULL);
17     }

```

Listing 第二章.52 tac.c: 数组名在表达式位置的指针衰变

在表达式中使用数组变量时, 会自动转为指向首元素的指针; 若是多维数组,

则将类型树的“去掉第一维”赋给表达式的返回类型，使后续索引与字段选择具有正确的元素类型信息。

索引读取：do_index，兼容数组与指针，考虑多维数组元素、结构体元素返回地址或值。

```

1  /* base[idx] 读取：数组名自动衰变为指针，计算地址后 LOAD */
2  EXP *do_index(EXP *base, EXP *idx)
3  {
4      EXP *base_ptr = base;
5      if (base->ret->dtype == T_ARRAY)
6      {
7          base_ptr = do_addr_var(base->ret);
8          /* 传播多维数组的类型信息：指向元素类型 */
9          if (base->ret->tnode)
10         {
11             base_ptr->ret->tnode = ((TNode
12                                     *)base->ret->tnode)->elem;
13             if (base_ptr->ret->tnode && ((TNode
14                                     *)base_ptr->ret->tnode)->kind == T_ARRAY)
15                 base_ptr->ret->elem_dtype = T_ARRAY;
16         }
17     }
18     else if (base->ret->dtype != T_PTR)
19     {
20         error("indexing non-pointer/non-array");
21     }
22
23     int elem = (base_ptr->ret->elem_dtype == T_UNDEF) ? T_INT :
24               base_ptr->ret->elem_dtype;
25     int esz;
26     if (base_ptr->ret->ptr_level > 1)
27         esz = size_of_dtype(T_PTR);
28     else if (base_ptr->ret->tnode && ((TNode
29                                     *)base_ptr->ret->tnode)->kind == T_ARRAY)
30         esz = size_of_tnode((TNode *)base_ptr->ret->tnode);
31     else
32         esz = (elem == T_STRUCT) ? (base_ptr->ret->elem_sdef ?
33                                     base_ptr->ret->elem_sdef->size : 0)
34                                     : size_of_dtype(elem);

```

```

30  if (esz <= 0)
31      esz = 4;
32
33  SYM *t_mul = mk_tmp();
34  TAC *d_mul = mk_tac(TAC_VAR, t_mul, NULL, NULL);
35  TAC *m = mk_tac(TAC_MUL, t_mul, idx->ret, mk_const(esz));
36  m->prev = d_mul;
37
38  SYM *addr = mk_tmp_typed(T_PTR, elem, NULL,
39      base_ptr->ret->elem_sdef);
40  TAC *d_addr = mk_tac(TAC_VAR, addr, NULL, NULL);
41  d_addr->prev = join_tac(base_ptr->tac, idx->tac);
42  TAC *add = mk_tac(TAC_ADD, addr, base_ptr->ret, t_mul);
43  add->prev = join_tac(d_addr, m);
44
45  /* 元素为结构体或数组：返回元素地址（指针），供后续 . / []
46     使用；否则正常 LOAD */
47  if (elem == T_STRUCT || (base_ptr->ret->tnode && ((TNode
48      *)base_ptr->ret->tnode)->kind == T_ARRAY))
49  {
50      if (base_ptr->ret->tnode && ((TNode
51          *)base_ptr->ret->tnode)->kind == T_ARRAY)
52      {
53          addr->tnode = ((TNode *)base_ptr->ret->tnode)->elem;
54          if (addr->tnode && ((TNode *)addr->tnode)->kind ==
55              T_ARRAY)
56              addr->elem_dtype = T_ARRAY;
57      }
58      return mk_exp(NULL, addr, add);
59  }
60  else
61  {
62      SYM *ret = mk_tmp_typed(elem, T_UNDEF, NULL, NULL);
63      TAC *d_ret = mk_tac(TAC_VAR, ret, NULL, NULL);
64      d_ret->prev = add;
65      TAC *ld = mk_tac(TAC_LOAD, ret, addr, NULL);
66      ld->prev = d_ret;
67      return mk_exp(NULL, ret, ld);

```

```

63     }
64 }

```

Listing 第二章.53 tac.c: 数组下标读取

索引读取会先把数组衰变为指针，计算偏移（乘元素字节数），得到元素地址。若元素是结构体或仍是数组（多维），返回地址以供后续字段访问或进一步索引；若是标量则直接生成 TAC_LOAD 读值。

索引赋值：变量数组 do_assign_index 与通用表达式基址 do_assign_index_exp。

```

1 TAC *do_assign_index(SYM *base_var, EXP *idx, EXP *rhs)
2 {
3     EXP *base = mk_exp(NULL, base_var, NULL);
4     if (base_var->dtype == T_ARRAY)
5         base = do_addr_var(base_var);
6     else if (base_var->dtype != T_PTR)
7         error("indexing non-pointer/non-array");
8
9     int elem = (base->ret->elem_dtype == T_UNDEF) ? T_INT :
10        base->ret->elem_dtype;
11     int esz = (elem == T_STRUCT) ? (base->ret->elem_sdef ?
12        base->ret->elem_sdef->size : 0)
13        : sizeof_dtype(elem);
14
15     if (esz <= 0)
16         esz = 4;
17
18     SYM *t_mul = mk_tmp();
19     TAC *d_mul = mk_tac(TAC_VAR, t_mul, NULL, NULL);
20     TAC *m = mk_tac(TAC_MUL, t_mul, idx->ret, mk_const(esz));
21     m->prev = d_mul;
22
23     SYM *addr = mk_tmp_typed(T_PTR, elem, NULL,
24        base->ret->elem_sdef);
25     TAC *d_addr = mk_tac(TAC_VAR, addr, NULL, NULL);
26     d_addr->prev = join_tac(base->tac, idx->tac);
27     TAC *add = mk_tac(TAC_ADD, addr, base->ret, t_mul);
28     add->prev = join_tac(d_addr, m);
29
30     TAC *store = mk_tac(TAC_STORE, addr, rhs->ret, NULL);
31     store->prev = join_tac(add, rhs->tac);

```

```

28     return store;
29 }
30
31 TAC *do_assign_index_exp(EXP *base_exp, EXP *idx, EXP *rhs)
32 {
33     EXP *base_ptr = base_exp;
34     if (base_exp->ret->dtype == T_ARRAY)
35     {
36         base_ptr = do_addr_var(base_exp->ret);
37         if (base_exp->ret->tnode)
38         {
39             base_ptr->ret->tnode = ((TNode
40                                     *)base_exp->ret->tnode)->elem;
41             if (base_ptr->ret->tnode && ((TNode
42                                     *)base_ptr->ret->tnode)->kind == T_ARRAY)
43                 base_ptr->ret->elem_dtype = T_ARRAY;
44         }
45     }
46     else if (base_exp->ret->dtype != T_PTR)
47     {
48         error("indexing non-pointer/non-array");
49     }
50
51     int elem = (base_ptr->ret->elem_dtype == T_UNDEF) ? T_INT :
52               base_ptr->ret->elem_dtype;
53     int esz;
54     if (base_ptr->ret->ptr_level > 1)
55         esz = size_of_dtype(T_PTR);
56     else if (base_ptr->ret->tnode && ((TNode
57                                     *)base_ptr->ret->tnode)->kind == T_ARRAY)
58         esz = size_of_tnode((TNode *)base_ptr->ret->tnode);
59     else
60         esz = (elem == T_STRUCT) ? (base_ptr->ret->elem_sdef ?
61                                     base_ptr->ret->elem_sdef->size : 0)
62               : size_of_dtype(elem);
63     if (esz <= 0)
64         esz = 4;

```

```

61  SYM *t_mul = mk_tmp();
62  TAC *d_mul = mk_tac(TAC_VAR, t_mul, NULL, NULL);
63  TAC *m = mk_tac(TAC_MUL, t_mul, idx->ret, mk_const(esz));
64  m->prev = d_mul;
65
66  SYM *addr = mk_tmp_typed(T_PTR, elem, NULL,
        base_ptr->ret->elem_sdef);
67  TAC *d_addr = mk_tac(TAC_VAR, addr, NULL, NULL);
68  d_addr->prev = join_tac(base_ptr->tac, idx->tac);
69  TAC *add = mk_tac(TAC_ADD, addr, base_ptr->ret, t_mul);
70  add->prev = join_tac(d_addr, m);
71
72  TAC *store = mk_tac(TAC_STORE, addr, rhs->ret, NULL);
73  store->prev = join_tac(add, rhs->tac);
74  return store;
75 }

```

Listing 第二章.54 tac.c: 数组下标赋值

两者都计算元素地址并发射 TAC_STORE。do_assign_index_exp 更通用，支持复杂的左值基（如结构体字段为数组，或表达式结果为指针）。

地址形式的索引：addr_index，返回元素地址指针（与 laddr 配合）。

```

1  /* 地址形式：& (base[idx]) */
2  EXP *addr_index(EXP *base, EXP *idx)
3  {
4      EXP *base_ptr = base;
5      if (base->ret->dtype == T_ARRAY)
6      {
7          base_ptr = do_addr_var(base->ret);
8          if (base->ret->tnode)
9          {
10             base_ptr->ret->tnode = ((TNode
                *)base->ret->tnode)->elem;
11             if (base_ptr->ret->tnode && ((TNode
                *)base_ptr->ret->tnode)->kind == T_ARRAY)
12                 base_ptr->ret->elem_dtype = T_ARRAY;
13         }
14     }
15     else if (base->ret->dtype != T_PTR)

```

```

16 {
17     error("indexing non-pointer/non-array");
18 }
19
20 int elem = (base_ptr->ret->elem_dtype == T_UNDEF) ? T_INT :
    base_ptr->ret->elem_dtype;
21 int esz;
22 if (base_ptr->ret->ptr_level > 1)
23     esz = size_of_dtype(T_PTR);
24 else if (base_ptr->ret->tnode && ((TNode
    *)base_ptr->ret->tnode)->kind == T_ARRAY)
25     esz = size_of_tnode((TNode *)base_ptr->ret->tnode);
26 else
27     esz = (elem == T_STRUCT) ? (base_ptr->ret->elem_sdef ?
    base_ptr->ret->elem_sdef->size : 0)
28         : size_of_dtype(elem);
29 if (esz <= 0)
30     esz = 4;
31
32 SYM *t_mul = mk_tmp();
33 TAC *d_mul = mk_tac(TAC_VAR, t_mul, NULL, NULL);
34 TAC *m = mk_tac(TAC_MUL, t_mul, idx->ret, mk_const(esz));
35 m->prev = d_mul;
36
37 SYM *addr = mk_tmp_typed(T_PTR, elem, NULL,
    base_ptr->ret->elem_sdef);
38 TAC *d_addr = mk_tac(TAC_VAR, addr, NULL, NULL);
39 d_addr->prev = join_tac(base_ptr->tac, idx->tac);
40 TAC *add = mk_tac(TAC_ADD, addr, base_ptr->ret, t_mul);
41 add->prev = join_tac(d_addr, m);
42
43 if (base_ptr->ret->tnode && ((TNode
    *)base_ptr->ret->tnode)->kind == T_ARRAY)
44 {
45     addr->tnode = ((TNode *)base_ptr->ret->tnode)->elem;
46     if (addr->tnode && ((TNode *)addr->tnode)->kind == T_ARRAY)
47         addr->elem_dtype = T_ARRAY;
48 }

```

```

49     return mk_exp(NULL, addr, add);
50 }

```

Listing 第二章.55 tac.c: 数组下标地址计算

与 `do_index` 类似，但目的不是读值而是取元素地址（用于 `&base[idx]` 和地址链），并保持多维数组元素类型传播。

2.7.2.4 obj.c

针对通过地址存储时的寄存器管理与写通策略，避免数组/指针写影响后造成错误回写。

```

1  /* 地址 + 值 存储：避免值装载覆盖地址寄存器；按元素类型选 STQ/STC
   */
2  static void asm_store_addr_val(SYM *addr, SYM val)
3  {
4      ...
5      /* 间接写会导致缓存失效：清空所有寄存器描述，
6       避免后续将脏旧值回写覆盖正确内存 */
7      for (int r = R_GEN; r < R_NUM; r++)
8      {
9          if (rdesc[r].mod)
10             asm_write_back(r);
11             rdesc_clear(r);
12     }
13 }

```

Listing 第二章.56 obj.c: 数组/指针存储后的寄存器管理

当进行 `addr = val` 存储时，为避免后续寄存器回写覆盖刚通过指针更新过的内存，清空寄存器描述并仅回写真正被修改的绑定，确保数组/指针赋值的可见性与正确性。

以上为该功能实现的重要代码片段展示与详细说明。

2.8 struct

2.8.1 实现思路

从词法层面，先添加结构相关关键字与符号（“`struct`” “`typedef`” “`->`” 以及标识符与基本常量），保证后续语法能区分标签式结构体、匿名结构与指针访问；

语法层面以统一的后缀优先级解析“.”、“->”与“[]”，在类型说明中支持“struct 标记 字段列”“struct 字段列”“struct 标记（前向/引用）”与 typedef 名称回填完整类型元信息；字段声明规则要在类型说明 + 字段变量列上支持标量、指针与（多维）数组，并保持与一般变量声明一致的维度表达，另通过 laddr 链式地址构造统一解析诸如“&a[i].x”“&p->x”等复合取址。

语义层面以单独的结构体定义表承载 SDEF（结构体定义）并在定义时顺序计算各 FIELD 的 offset 与整体 size（不引入对齐复杂度），允许前向声明先占位；同时维护 typedef 表携带 dtype/elem_dtype/ptr_level/array_len/sdef/elem_sdef 等完整元信息，让 typedef 到指针/数组/结构体的别名在任意声明与表达式处都能还原精确类型；变量与字段的符号（SYM/FIELD）统一记录“自身类型 + 元素类型 + 指针层级 + 数组长度 + 多维类型树 tnode”，从而在数组衰变、指针算术与索引步长时可用 size-of 信息（结构体元素用 sdef->size）；表达式求值时对“.”/“->”统一降解为“计算字段地址 = 基址指针 + 常量偏移”，将“数组字段”视为“指向首元素的指针”返回以便后续继续“[]/.”链式访问，对“标量字段/指针字段”分别发射按字节宽度的 LOAD；整体结构体作为右值不直接支持以避免大对象拷贝，写字段则以“计算地址 + STORE”实现；数组名在表达式位置自动衰变为指向首元素的指针，多维数组通过 tnode 传播去掉一维后的元素类型，指针算术与索引统一按元素大小（包括结构体大小或剩余维度块大小）进行位移。

后端生成中间码到目标码时，TAC_ADDR/TAC_ADD/TAC_LOAD/TAC_STORE 构成结构访问的核心：字段访问与数组索引都转为基址寄存器 + 常量/变长偏移形式的加载与存储，按元素类型选择字/字节指令；局部/全局结构体对象的空间在变量声明处据 size_of_symbol 分配；经由指针的写操作后清理/写回寄存器描述，避免寄存器缓存把通过指针更新过的内存又被旧值覆盖，从而保证通过“p->x”“s.arr[i]”之类路径更新的可见性与一致性；

2.8.2 代码细节

2.8.2.1 mini.l

词法分析上，要对关键字以及关系连接符进行识别，由于还添加 typedef 的功能，所以除开 struct 与->（用于结构体指针访问成员），还有 typedef 的 token 识别。

```
1 "struct"    { return STRUCT; }
2 "typedef"   { return TYPEDEF; }
3 "->"       { return ARROW; }
```

Listing 第二章.57 mini.l: 结构体相关关键字与符号识别

同时，由于 `typedef` 的存在，标识符可能是之前已经定义过的某种类型，所以在标识符识别的部分需要进行更改，如果之前已定义过则返回不同的 `token`，提前说明，所有直接定义的 `struct` 与 `typedef` 定义的 `struct` 都会通过不同的链表存储，下面用到的方法其实就是在 `typedef` 链表中查找该名字。

```

1 {ID} {
2     yylval.string = strdup(yytext);
3     if (is_typedef_name(yytext)) return TYPEID;
4     return IDENTIFIER;
5 }
```

Listing 第二章.58 mini.l: 标识符识别与 `typedef` 区分

2.8.2.2 mini.y

首先是类型说明，在 `mini.y` 中，扩展 `type_spec`，包括直接声明结构体，匿名结构（用于 `typedef`），之前声明并描述其结构后进行引用以及 `typedef` 过的类型声明。

```

1 type_spec
2 : INT
3 {
4     current_type = T_INT;
5     current_sdef = NULL;
6     current_elem_dtype=T_UNDEF;
7     current_elem_sdef=NULL;
8     current_ptr_level=0;
9     current_array_len=0;
10 }
11 | CHAR
12 {
13     current_type = T_CHAR;
14     current_sdef = NULL;
15     current_elem_dtype=T_UNDEF;
16     current_elem_sdef=NULL;
17     current_ptr_level=0;
18     current_array_len=0;
19 }
20 /* struct S { ... } */
21 | STRUCT IDENTIFIER '{' field_decl_list '}'
22 {
```

```

23     SDEF *d = struct_define($2, $4);
24     current_type = T_STRUCT;
25     current_sdef = d; current_elem_dtype=T_UNDEF;
26     current_elem_sdef=NULL;
27     current_ptr_level=0;
28     current_array_len=0;
29 }
30 | STRUCT '{' field_decl_list '}' /* 匿名结构 */
31 {
32     SDEF *d = struct_define(NULL, $3);
33     current_type = T_STRUCT;
34     current_sdef = d;
35     current_elem_dtype=T_UNDEF;
36     current_elem_sdef=NULL;
37     current_ptr_level=0;
38     current_array_len=0;
39 }
40 | STRUCT IDENTIFIER /* 前向/引用标签 */
41 {
42     SDEF *d = struct_forward($2);
43     current_type = T_STRUCT;
44     current_sdef = d;
45     current_elem_dtype=T_UNDEF;
46     current_elem_sdef=NULL;
47     current_ptr_level=0;
48     current_array_len=0;
49 }
50 | TYPEID /* typedef 名称: 带回完整元信息 */
51 {
52     int dt, edt, pl, al; SDEF *sd, *esd;
53     if (!typedef_get($1, &dt, &edt, &pl, &al, &sd, &esd))
54         yyerror("unknown typedef");
55     current_type = dt; current_sdef = sd;
56     current_elem_dtype = edt; current_elem_sdef = esd;
57     current_ptr_level = pl; current_array_len = al;
58 }
59 ;

```

Listing 第二章.59 mini.y: 结构体类型说明

结构体字段声明，字段方面，可以声明的类型有基本数据类型，指针，数组（可以是多维），还有嵌套结构体

```
1  /* ----- 结构体字段声明 ----- */
2  field_decl_list
3  : field_decl
4  {
5      $$ = $1;
6  }
7  | field_decl_list field_decl
8  {
9      $$ = field_join($1, $2);
10 }
11 ;
12
13 field_decl
14 : type_spec field_var_list ';'
15 {
16     $$ = $2;
17 }
18 ;
19
20 field_var_list
21 : field_var_unit
22 {
23     $$ = $1;
24 }
25 | field_var_list ',' field_var_unit
26 {
27     $$ = field_join($1, $3);
28 }
29 ;
30
31 field_var_unit
32 : IDENTIFIER
33 {
34     $$ = field_from_current($1, 0, 0);
35 }
36 | '*' IDENTIFIER
```

```

37 {
38     $$ = field_from_current($2, 1, 0);
39 }
40 | IDENTIFIER dims
41 {
42     $$ = field_from_current_d1($1, 0, $2);
43 }
44 ;

```

Listing 第二章.60 mini.y: 结构体字段声明

typedef 声明, 支持 typedef 创建类型别名:

```

1  /* ----- 变量/函数声明 ----- */
2  declaration
3  : type_spec variable_list ';'
4  {
5      $$ = $2;
6  }
7  | TYPEDEF type_spec IDENTIFIER ';'
8  {
9      typedef_add($3, current_type, T_UNDEF, 0, 0,
10         current_sdef, NULL);
11     $$ = NULL;
12 }
13 | TYPEDEF type_spec '*' IDENTIFIER ';'
14 {
15     typedef_add($4, T_PTR, current_type, 1, 0, NULL,
16         current_sdef);
17     $$ = NULL;
18 }
19 | TYPEDEF type_spec IDENTIFIER '[' INTEGER ']' ';'
20 {
21     typedef_add($3, T_ARRAY, current_type, 0, atoi($5), NULL,
22         current_sdef);
23     $$ = NULL;
24 }
25 | type_spec ';'
26 {
27     $$ = NULL;

```

```
28 }
```

Listing 第二章.61 mini.y: typedef 声明

成员访问表达式, 支持. 和-> 运算符访问结构体成员:

```
1 /* ----- 表达式 (含字段访问) ----- */
2 expression
3 : expression '.' IDENTIFIER
4 {
5     $$=do_field_select($1, $3, 0);
6 }
7 | expression ARROW IDENTIFIER
8 {
9     $$=do_field_select($1, $3, 1);
10 }
```

Listing 第二章.62 mini.y: 结构体成员访问表达式

成员赋值, 支持对结构体成员进行赋值:

```
1 /* ----- 赋值 (含结构体字段) ----- */
2 assignment_statement
3 : expression '.' IDENTIFIER '=' expression
4 {
5     $$ = do_assign_field_select($1, $3, 0, $5);
6 }
7 | expression ARROW IDENTIFIER '=' expression
8 {
9     $$ = do_assign_field_select($1, $3, 1, $5);
10 }
11 ;
```

Listing 第二章.63 mini.y: 结构体成员赋值

2.8.2.3 tac.h

结构体基本数据类型添加, 结构体字段以及结构体定义的数据结构设计。

```
1 #define T_STRUCT 5
2
3 /* 结构体字段与定义类型 (供语法/语义层使用) */
4 typedef struct field
5 {
6     char *name;
7     int dtype; /*
8     T_INT/T_CHAR/T_PTR/T_ARRAY/T_STRUCT */
```

```

8      int elem_dtype;          /* 指针/数组元素类型 */
9      int ptr_level;          /* 指针层级 */
10     void *tnode;             /*
        若为多维数组，指向类型结点 (TNode) */
11     int array_len;           /* 数组长度 (0 代表未定长或按 0
        处理) */
12     struct struct_def *sdef;  /* 若字段自身为结构体，指向定义
        */
13     struct struct_def *elem_sdef; /*
        若字段为指针/数组且元素为结构体 */
14     int offset;              /* 字段偏移 (定义完成后填充) */
15     struct field *next;
16 } FIELD;
17
18 typedef struct struct_def
19 {
20     char *name;              /* 标签名 (可为匿名，内部生成) */
21     FIELD *fields;           /* 字段链表 */
22     int size;                /* 结构体总大小 (简单顺序打包，无对齐) */
23     struct struct_def *next;
24 } SDEF;
25
26 typedef struct sym
27 {
28     ...
29     /* 新增：结构体信息 */
30     SDEF *sdef;              /* 若自身是 T_STRUCT，则指向结构体定义 */
31     SDEF *elem_sdef;         /*
        若元素类型是结构体 (指针/数组)，指向元素结构体定义 */
32     ...
33 } SYM;

```

Listing 第二章.64 tac.h: 结构体相关数据结构

2.8.2.4 tac.c

结构体定义，对于结构体的定义，包括直接定义和 `typedef` 定义两种，同时也因为要管理这两种结构体创建两个表用于查找，插入，`typedef` 因为可以声明出除结构以外的其他类型，所以比起普通的 SDEF 要多出一些信息需要存储，如果是

结构的定义则将定义放在 SDEF 指针中，如果是指向结构的数组，指针，则把结构的定义放在 elem_sdef 中。

```

1 // 结构体定义与 typedef 表
2 static SDEF *sdef_tab = NULL;
3
4 typedef struct typedef_entry
5 {
6     char *name;
7     int dtype;
8     int elem_dtype;
9     int ptr_level;
10    int array_len;
11    SDEF *sdef;
12    SDEF *elem_sdef;
13    struct typedef_entry *next;
14 } TDEF;
15
16 static TDEF *typedef_tab = NULL;

```

Listing 第二章.65 tac.c: typedef 定义与 typedef、结构体表

结构体定义函数，先查找之前有无声明过，没有就申请空间，然后把字段链表填入，计算总大小，对每一个字段进行类型的判断，不同类型做不同处理，最后把大小填入完成声明。

```

1 SDEF *struct_define(char *tag, FIELD *fields)
2 {
3     SDEF *def = tag ? struct_lookup(tag) : NULL;
4     if (!def)
5     {
6         def = (SDEF *)malloc(sizeof(SDEF));
7         def->name = tag ? strdup(tag) :
8             strdup(mk_lstr(next_label++));
9         def->fields = NULL;
10        def->size = 0;
11        def->next = sdef_tab;
12        sdef_tab = def;
13    }

```

```
14  /* 直接 在原字段链表上计算并写入 offset，避免复制时丢失偏移 */
15  int off = 0;
16  for (FIELD *f = fields; f; f = f->next)
17  {
18      int bytes = 0;
19      if (f->dtype == T_ARRAY)
20      {
21          /* 多维数组：若字段带有
22             tnode，优先使用类型结点计算整个块大小 */
23          if (f->tnode)
24          {
25              bytes = size_of_tnode((TNode *)f->tnode);
26          }
27          else
28          {
29              int esz = (f->elem_dtype == T_STRUCT) ?
30                  (f->elem_sdef ? f->elem_sdef->size : 0)
31                  : size_of_dtype(f->elem_dtype);
32              bytes = esz * ((f->array_len > 0) ? f->array_len :
33                  0);
34          }
35      }
36      else if (f->dtype == T_STRUCT)
37      {
38          bytes = f->sdef ? f->sdef->size : 0;
39      }
40      else
41      {
42          bytes = size_of_dtype(f->dtype);
43      }
44      if (bytes <= 0)
45          bytes = 4;
46      f->offset = off; /* 写回原结点 */
47      off += bytes;
48  }
49  def->fields = fields; /* 直接挂原链表 */
50  def->size = off;
51  return def;
```

```
50 }
```

Listing 第二章.66 tac.c: 结构体定义函数

字段查找函数，用于赋值，读取时查找某个结构体的字段进行操作。

```
1 static FIELD *find_field(SDEF *def, const char *name)
2 {
3     for (FIELD *f = def ? def->fields : NULL; f; f = f->next)
4         if (strcmp(f->name, name) == 0)
5             return f;
6     return NULL;
7 }
```

Listing 第二章.67 tac.c: 结构体字段查找函数

成员访问语义处理, 处理. 和-> 运算符的语义动作，对于字段不同的类型生成不同的 symbol，最后返回表达式。

```
1 EXP *do_field_select(EXP *base, char *fname, int is_arrow)
2 {
3     EXP *addr = compute_field_addr(base, fname, is_arrow);
4     FIELD *fld = (FIELD *)addr->ret->etc;
5
6     int val_dtype = T_UNDEF, val_elem_dtype = T_UNDEF;
7     SDEF *val_sdef = NULL, *val_elem_sdef = NULL;
8
9     if (fld)
10    {
11        if (fld->dtype == T_ARRAY)
12        {
13            /* 数组字段：返回指向该数组首元素的指针值，
14             * 供后续索引/指针算术使用。
15             * 在赋值到标量时，由 do_assign
16             * 进行自动解引用为 a[0]。 */
17            return addr;
18        }
19        else if (fld->dtype == T_PTR)
20        {
21            /* 指针字段：保存其指向类型，支持后续 q->v
22             * */
23        }
24    }
25 }
```

```

21         val_dtype = T_PTR;
22         val_elem_dtype = fld->elem_dtype;
23         if (val_elem_dtype == T_STRUCT)
24             val_elem_sdef = fld->elem_sdef;
25     }
26     else if (fld->dtype == T_STRUCT)
27     {
28         error("cannot use whole struct as rvalue");
29     }
30     else
31     {
32         val_dtype = fld->dtype; /* 标量 */
33     }
34 }
35 else
36 {
37     int elem = addr->ret->elem_dtype;
38     if (elem == T_STRUCT)
39         error("cannot use whole struct as rvalue");
40     val_dtype = elem;
41 }
42
43 SYM *ret = mk_tmp_typed(val_dtype, val_elem_dtype,
44                         val_sdef, val_elem_sdef);
45 TAC *d = mk_tac(TAC_VAR, ret, NULL, NULL);
46 d->prev = addr->tac;
47 TAC *ld = mk_tac(TAC_LOAD, ret, addr->ret, NULL);
48 ld->prev = d;
49 return mk_exp(NULL, ret, ld);
}

```

Listing 第二章.68 tac.c: 结构体成员访问语义处理

成员赋值语义处理，先找地址，然后 store。

```

1 TAC *do_assign_field_select(EXP *base, char *fname, int is_arrow,
2   EXP *rhs)
3 {
4     EXP *addr = compute_field_addr(base, fname, is_arrow);
5     TAC *store = mk_tac(TAC_STORE, addr->ret, rhs->ret, NULL);

```

```

5     store->prev = join_tac(addr->tac, rhs->tac);
6     return store;
7 }

```

Listing 第二章.69 tac.c: 结构体成员赋值语义处理

地址计算核心函数，最麻烦的一个函数，计算字段的地址。

首先根据 `is_arrow` 分两条路径校验与归一化基址：

对于点号`.`：若 `base` 是结构体变量 (`dtype=T_STRUCT` 且 `sdef`)，要求它确实是变量 (`type==SYM_VAR`)，然后通过 `do_addr_var` 取其地址，得到 `base_ptr`。若 `base` 是指向结构体的指针，放宽语义允许用 `.` 当作 `->`，直接把 `base` 当指针使用。若 `base` 是“指向数组”的指针，且该数组的叶子元素是结构体（通过 `tnode` 连跳数组结点直到叶子，得到 `leaf_kind==T_STRUCT`），则认为 `base` 指向结构体数组元素的地址，亦可用 `.` 访问字段。其他情况报错：“``.` base must be a struct object`”。

对于箭头`->`：必须是指向结构体的指针 (`dtype=T_PTR` 且 `elem_dtype=T_STRUCT` 且 `elem_sdef` 非空)，否则报错并打印调试信息。确认后把 `def` 设为结构体定义，`base_ptr` 就是 `base` 本身。接着用 `find_field(def, fname)` 找字段描述，生成一段指令：`addr = base_ptr + offset (mk_tac(TAC_ADD))`，其中 `offset` 来自字段在结构体内的偏移。这里返回的是字段的地址。关键之处在于为 `addr` 补全 `elem` 类型信息，供后续 `LOAD/STORE` 决定按字节/字宽访问以及支持继续的类型推断：

字段是数组 (`T_ARRAY`)：若字段是多维数组 (`fld->tnode` 存在)，实现指针衰变：丢弃最外层一维，把 `addr->tnode` 设为剩余维度的 `tnode`，并据叶子结点更新 `elem_dtype/elem_sdef` (数组/结构体/标量)。单维数组则把 `elem_dtype` 直接设为元素类型，若元素是结构体则传递 `elem_sdef`。

字段是结构体 (`T_STRUCT`)：将 `elem_dtype` 设为 `T_STRUCT`，并传递字段的 `sdef`。

字段是指针 (`T_PTR`)： `elem_dtype` 设为 `T_PTR` (用于选择指针字宽)。

其他标量： `elem_dtype` 设为具体标量类型， `elem_sdef` 置空。

此外， `addr->etc` 直接挂上 `FIELD` 指针，方便后续阶段获取字段的完整元信息。最终返回 `mk_exp(NULL, addr, add)`，即值为该字段地址、代码链为基址求值与地址计算的表达式。

```

1  /* 取字段地址： base(.|->)field -> 统一得到指针值 */
2  static EXP *compute_field_addr(EXP *base, char *fname, int
   is_arrow)
3  {

```

```

4      SDEF *def = NULL;
5      EXP *base_ptr = NULL;
6
7      if (!is_arrow)
8      {
9          if (!base->ret)
10             error("'.' base must be a struct object");
11
12         /* 辅助：提取 tnode 叶子类型（跳过多维数组层） */
13         TNode *tn = (TNode *)base->ret->tnode;
14         while (tn && tn->kind == T_ARRAY)
15             tn = tn->elem;
16         int leaf_kind = tn ? tn->kind : T_UNDEF;
17
18         if (base->ret->dtype == T_STRUCT &&
19             base->ret->sdef)
20         {
21             /* 结构体变量：按取地址处理 */
22             def = base->ret->sdef;
23             if (base->ret->type != SYM_VAR)
24                 error("'.' base must be a variable");
25             base_ptr = do_addr_var(base->ret);
26         }
27         else if (base->ret->dtype == T_PTR &&
28                 base->ret->elem_dtype == T_STRUCT &&
29                 base->ret->elem_sdef)
30         {
31             /* 指向结构体的指针，允许使用 '.' 视作
32              * '-'>' */
33             def = base->ret->elem_sdef;
34             base_ptr = base;
35         }
36         else if (base->ret->dtype == T_PTR &&
37                 base->ret->elem_dtype == T_ARRAY && leaf_kind
38                 == T_STRUCT)
39         {
40             /*
41              * 指向结构体数组元素的地址：索引后得到的指针，

```

```

35         其 elem_dtype 可能是 T_ARRAY, tnode 叶子为
           struct */
36         def = tn ? tn->sdef : NULL;
37         if (!def)
38             error("'.' base struct array element
                   unresolved");
39         base_ptr = base;
40     }
41     else
42     {
43         error("'.' base must be a struct object");
44     }
45 }
46 else
47 {
48     if (!base->ret || base->ret->dtype != T_PTR ||
        base->ret->elem_dtype != T_STRUCT ||
        !base->ret->elem_sdef)
49     {
50         fprintf(stderr, "debug: '->' base name=%s
                           dtype=%d elem_dtype=%d elem_sdef=%p\n",
51                 base->ret ? base->ret->name : "(null)",
52                 base->ret ? base->ret->dtype : -1,
53                 base->ret ? base->ret->elem_dtype : -1,
54                 base->ret ? (void *)base->ret->elem_sdef :
                           NULL);
55         error("'-' base must be a pointer to
               struct");
56     }
57     def = base->ret->elem_sdef;
58     base_ptr = base;
59 }
60
61 FIELD *fld = find_field(def, fname);
62 if (!fld)
63     error("unknown field: %s", fname);
64
65 SYM *addr = mk_tmp_typed(T_PTR, T_UNDEF, NULL, NULL);

```

```

66     TAC *d_addr = mk_tac(TAC_VAR, addr, NULL, NULL);
67     TAC *code = join_tac(base_ptr->tac, d_addr);
68     TAC *add = mk_tac(TAC_ADD, addr, base_ptr->ret,
        mk_const(fld->offset));
69     add->prev = code;
70
71     /* 供 LOAD/STORE 使用的按字节/字宽选择 */
72     if (fld->dtype == T_ARRAY)
73     {
74         addr->elem_sdef = (fld->elem_dtype == T_STRUCT) ?
            fld->elem_sdef : NULL;
75         /* 若是多维数组(tnode存在),
76            指针衰变应指向首元素: 丢弃最外层一维 */
77         if (fld->tnode)
78         {
79             TNode *full = (TNode *)fld->tnode; /* full
            描述完整多维 */
80             TNode *elem = full->elem;           /*
            首元素类型(去掉第一维) */
81             addr->tnode = elem;                 /*
            指针的 tnode 指向剩余维度 */
82             if (elem && elem->kind == T_ARRAY)
83             {
84                 addr->elem_dtype = T_ARRAY; /*
            仍然是数组元素(还有多维) */
85             }
86             else if (elem && elem->kind == T_STRUCT)
87             {
88                 addr->elem_dtype = T_STRUCT;
89                 addr->elem_sdef = elem->sdef;
90             }
91             else if (elem)
92             {
93                 addr->elem_dtype = elem->kind; /*
            标量叶子 */
94             }
95             else
96             {

```

```

97             /* 退化：没有
                elem（不应发生），保持原
                elem_dtype */
98             addr->elem_dtype = fld->elem_dtype;
99         }
100     }
101     else
102     {
103         /* 单维数组：指针指向标量/结构体元素 */
104         addr->elem_dtype = fld->elem_dtype;
105     }
106 }
107 else if (fld->dtype == T_STRUCT)
108 {
109     addr->elem_dtype = T_STRUCT;
110     addr->elem_sdef = fld->sdef;
111 }
112 else if (fld->dtype == T_PTR)
113 {
114     addr->elem_dtype = T_PTR;
115 }
116 else
117 {
118     addr->elem_dtype = fld->dtype; /* 标量 */
119     addr->elem_sdef = NULL;
120 }
121 addr->etc = (void *)fld; /* 关键：把字段描述传下去 */
122
123 return mk_exp(NULL, addr, add);
124 }

```

Listing 第二章.70 tac.c: 结构体字段地址计算

typedef 管理，往链表加新的 typedef 定义，查看是不是 typedef 过的类型。

```

1 void typedef_add(const char *name, int dtype, int elem_dtype, int
  ptr_level, int array_len, SDEF *sdef, SDEF *elem_sdef)
2 {
3     TDEF *e = (TDEF *)malloc(sizeof(TDEF));
4     e->name = strdup(name);

```

```
5     e->dtype = dtype;
6     e->elem_dtype = elem_dtype;
7     e->ptr_level = ptr_level;
8     e->array_len = array_len;
9     e->sdef = sdef;
10    e->elem_sdef = elem_sdef;
11    e->next = typedef_tab;
12    typedef_tab = e;
13 }
14
15 int is_typedef_name(const char *name)
16 {
17     for (TDEF *p = typedef_tab; p; p = p->next)
18         if (strcmp(p->name, name) == 0)
19             return 1;
20     return 0;
21 }
```

Listing 第二章.71 tac.c: typedef 管理

以上为该功能实现的重要代码片段展示与详细说明。

2.9 function

2.9.1 实现思路

老师的编译器已有基本的函数功能，但是需要添加形参数据类型声明以及返回值类型检查，在这里就从到位说一遍实现的大体思路。

在词法层面，识别函数相关的关键字、标识符和分隔符（如基本类型 `int/char`、`return`、小括号与逗号、花括号）；语法层面将类型说明 + 函数名 + 参数列表 + 语句块作为一个完整的函数定义单元，并允许把类型说明与声明分离（便于前向声明和多源声明），参数列表的语法要容纳标量、指针以及数组形参（语义上退化为指针），同时在语句层面纳入 `return`、分支与循环等控制结构；

语义层面将函数名在全局符号表中宣告为可调用实体，为其记录返回类型与（在调用处可选的）形参与调用约定，并在函数进入时切换到局部作用域建立帧布局，把形式参数与局部变量映射为栈帧中的偏移，随后把语句块解析为中间表示（TAC），其中调用函数的表达统一编码为实参压入（按照约定的方向）+ `CALL` + 可选返回值接收，`return` 则编码为设置返回值 + 回到返回标签/恢复基址，并在语

义阶段对返回类型与实/形参类型进行一致性校验（如数组形参作为指针检查元素类型）；

后端层面在目标代码生成时为函数入口发射“序言”（建立栈帧、保存旧基址与返回地址），将形式参数和局部变量按照前面确定的偏移布置在帧内，调用方按约定把实参写入新帧区域，再跳转到被调函数入口，返回时恢复寄存器与基址并跳回调用点，同时对 I/O 寄存器、通用寄存器与写回策略做保守处理，确保跨调用边界的数据一致性与无副作用的寄存器污染；

2.9.2 代码细节

2.9.2.1 mini.y

最外层把函数声明（定义）和普通声明作为同级构件归纳，并在语法结束时通过 `tac_complete()` 将倒置的三地址码链表反向成正序。函数定义与全局变量声明共存。

```

1 program : function_declaration_list
2 {
3     tac_last=$1;
4     tac_complete();
5 }
6 ;
7
8 function_declaration_list : function_declaration
9 | function_declaration_list function_declaration
10 {
11     $$=join_tac($1, $2);
12 }
13 ;
14
15 function_declaration : function
16 | declaration
17 ;

```

Listing 第二章.72 mini.y: 函数定义与全局声明

函数定义的核心语法。`function_head` 把标识符声明成全局函数符号并切换到局部作用域；整个函数单元以“类型说明 + 名称 + 形参 + 语句块”的形式，被语义组装为 `do_func(...)` 的中间表示，同时把当前函数的返回类型（如结构体返回的 `sdef`）记录在符号上，函数体完成后重置作用域与本地表。

```

1  /* ----- 函数 ----- */
2  function
3  : type_spec function_head '(' parameter_list ')' block
4  {
5      current_function = $2;
6      current_function->dtype = current_type;          /* 设置返回类型
7      */
8      current_function->sdef = (current_type == T_STRUCT) ?
9      current_sdef : NULL;
10     $$ = do_func($2, $4, $6);
11     scope = 0; sym_tab_local = NULL; current_function = NULL;
12 }
13 | error
14 {
15     error("Bad function syntax");
16     $$ = NULL;
17 }
18 ;
19
20 function_head : IDENTIFIER
21 {
22     $$=declare_func($1);
23     scope=1; /* Enter local scope. */
24     sym_tab_local=NULL; /* Init local symbol table. */
25 }
26 ;

```

Listing 第二章.73 mini.y: 函数定义核心语法

形参列表既支持标量参数，也支持指针与数组形参（数组在语义上退化为指针）。每个形参通过 `declare_para*` 系列函数产生 `TAC_FORMAL`，并在后端映射到新帧的参数区域（由帧偏移统一管理）。

```

1  parameter_list
2  : /* empty */ { $$ = NULL; }
3  | param_seq
4  ;
5
6  param_seq

```

```

7 : type_spec IDENTIFIER { $$ =
    declare_para($2); }
8 | type_spec '*' IDENTIFIER { $$ =
    declare_para_ptr($3, 1); }
9 | type_spec IDENTIFIER '[' ']' { $$ =
    declare_para_array($2, 0); } /* 形参数组退化为指针 */
10 | param_seq ',' type_spec IDENTIFIER { $$ =
    join_tac($1, declare_para($4)); }
11 | param_seq ',' type_spec '*' IDENTIFIER { $$ =
    join_tac($1, declare_para_ptr($5, 1)); }
12 | param_seq ',' type_spec IDENTIFIER '[' ']' { $$ =
    join_tac($1, declare_para_array($4, 0)); }
13 ;

```

Listing 第二章.74 mini.y: 形参列表语法

实参列表按逗号分隔的表达式聚合，并通过链表的 next 指针从右向左串接，后续在 do_call/do_call_ret 中按约定顺序生成 TAC_ACTUAL。

```

1 argument_list : /* empty */ { $$=NULL; } | expression_list ;
2
3 expression_list : expression
4 | expression_list ',' expression { $3->next=$1; $$=$3; }
5 ;

```

Listing 第二章.75 mini.y: 实参列表语法

函数调用既可以是语句（无关返回值），也可以是表达式（需要一个临时接收返回值）。分别由 do_call 和 do_call_ret 生成相应的 TAC_ACTUAL + TAC_CALL 序列，并在表达式场景下分配承载返回值的临时。

```

1 call_statement : IDENTIFIER '(' argument_list ')' { $$=do_call($1,
    $3); }
2 ;
3 call_expression : IDENTIFIER '(' argument_list ')' {
    $$=do_call_ret($1, $3); }
4 ;

```

Listing 第二章.76 mini.y: 函数调用语法

返回语句会对返回类型进行静态检查（与当前函数的返回类型比较），并把返回值表达式生成的代码与 TAC_RETURN 串接起来，交给后端进一步生成返回值

存放与函数返回序言。

```

1 return_statement : RETURN expression
2 {
3     if (current_function && current_function->dtype !=
4         $2->ret->dtype)
5         error("return type mismatch");
6     TAC *t=mk_tac(TAC_RETURN, $2->ret, NULL, NULL);
7     t->prev=$2->tac;
8     $$=t;
9 }

```

Listing 第二章.77 mini.y: 返回语句语法

2.9.2.2 tac.h

符号种类包含 `SYM_FUNC`，用于全局函数条目的标识。函数名符号与变量符号共用表，在不同作用域与语义阶段进行区分与检查。

```

1 #define SYM_FUNC 2

```

Listing 第二章.78 tac.h: 函数符号种类

函数相关的三地址码操作包括函数开始/结束标记、形参与实参、调用与返回、以及为函数入口生成的标签。这些是后端翻译与帧管理的直接依据。

```

1 /* type of tac /
2 #define TAC_UNDEF 0
3 ...
4 #define TAC_GOTO 13
5 #define TAC_IFZ 14
6 #define TAC_BEGINFUNC 15 / function begin /
7 #define TAC_ENDFUNC 16 / function end /
8 #define TAC_LABEL 17 / label a /
9 #define TAC_VAR 18 / int a /
10 #define TAC_FORMAL 19 / formal a /
11 #define TAC_ACTUAL 20 / actual a /
12 #define TAC_CALL 21 / a=call b /
13 #define TAC_RETURN 22 / return a */
14 ...

```

Listing 第二章.79 tac.h: 函数相关三地址码操作

函数相关的接口包括函数声明、形参声明、中间码封装函数体、调用（语句、表达式两种）等；它们共同串起解析到目标码的整条流水线。

```

1  /* function */
2  void tac_init();
3  void tac_complete();
4  TAC *join_tac(TAC *c1, TAC *c2);
5  ...
6  SYM *mk_label(char *name);
7  SYM *mk_tmp(void);
8  ...
9  SYM *declare_func(char *name);
10 TAC *declare_para(char *name);
11 TAC *do_func(SYM *name, TAC *args, TAC *code);
12 ...
13 TAC *do_call(char *name, EXP *arglist);
14 EXP *do_call_ret(char *name, EXP *arglist);

```

Listing 第二章.80 tac.h: 函数相关接口

全局编译状态中，scope 控制当前是否处在函数体内部的局部作用域；current_function 保存当前正在构建的函数，使得返回类型检查等成为可能。

```

1  /* global var */
2  extern int yylineno, scope, next_tmp, next_label;
3  extern SYM *sym_tab_global, *sym_tab_local;
4  extern TAC *tac_first, tac_last;
5  ...
6  / 新增：当前声明类型与当前函数（用于类型检查） /
7  extern int current_type; / T_INT/T_CHAR */
8  extern SYM *current_function;

```

Listing 第二章.81 tac.h: 函数相关全局状态

2.9.2.3 tac.c

将函数名登记到全局符号表，防止重复声明或与其他实体重名。成功返回的 SYM 被用作函数入口标签等后续生成的锚点。

```

1  SYM *declare_func(char *name)
2  {
3      SYM *sym = NULL;

```

```
4
5     sym = lookup_sym(sym_tab_global, name);
6
7     if (sym != NULL)
8     {
9         if (sym->type == SYM_FUNC)
10        {
11            error("func already declared");
12            return NULL;
13        }
14
15        if (sym->type != SYM_UNDEF)
16        {
17            error("func name already used");
18            return NULL;
19        }
20
21        return sym;
22    }
23
24    sym = mk_sym();
25    sym->type = SYM_FUNC;
26    sym->name = name;
27    sym->address = NULL;
28
29    insert_sym(&sym_tab_global, sym);
30    return sym;
31 }
```

Listing 第二章.82 tac.c: 函数声明

将函数整体封装为“入口标签 + 序言标记 + 形参声明 + 函数体代码 + 终结标记”的一条三地址码链。后端据此完成帧建立、参数布局与返回收尾。

```
1 TAC *do_func(SYM *func, TAC *args, TAC *code)
2 {
3     TAC *tlist; /* The backpatch list */
4
5     TAC *tlab; /* Label at start of function */
6     TAC *tbegin; /* BEGINFUNC marker */
```

```

7   TAC *tend;    /* ENDFUNC marker */
8
9   tlab = mk_tac(TAC_LABEL, mk_label(func->name), NULL, NULL);
10  tbegin = mk_tac(TAC_BEGINFUNC, NULL, NULL, NULL);
11  tend = mk_tac(TAC_ENDFUNC, NULL, NULL, NULL);
12
13  tbegin->prev = tlab;
14  code = join_tac(args, code);
15  tend->prev = join_tac(tbegin, code);
16
17  return tend;
18 }

```

Listing 第二章.83 tac.c: 函数封装

形参（标量）在语义上与局部变量类似，但用 TAC_FORMAL 区分，后端会把它们按既定偏移（从 FORMAL_OFF 向下）布局到被调栈帧中。

```

1   TAC *declare_para(char *name)
2   {
3       SYM *v = mk_var(name);
4       return mk_tac(TAC_FORMAL, v, NULL, NULL);
5   }

```

Listing 第二章.84 tac.c: 形参声明

指针形参与数组形参（退化为指针）在类型信息上记录指向元素的类型（包括结构体元素时的 sdef 指针），从而在体内支持解引用、成员访问与索引的类型正确性与步长计算。

```

1   /* 形式参数：指针 */
2   TAC *declare_para_ptr(char *name, int level)
3   {
4       SYM *v = mk_var(name);
5       v->dtype = T_PTR;
6       v->ptr_level = level;
7       v->elem_dtype = current_type;
8       v->elem_sdef = (current_type == T_STRUCT) ? current_sdef :
9                   NULL;
9       v->sdef = NULL;
10      return mk_tac(TAC_FORMAL, v, NULL, NULL);

```

```

11 }
12
13 /* 形式参数：数组（按指针退化处理） */
14 TAC *declare_para_array(char *name, int len)
15 {
16     SYM *v = mk_var(name);
17     v->dtype = T_PTR; /* 参数数组等价于指针 */
18     v->ptr_level = 1;
19     v->elem_dtype = current_type;
20     v->array_len = len;
21     v->elem_sdef = (current_type == T_STRUCT) ? current_sdef :
        NULL;
22     v->sdef = NULL;
23     return mk_tac(TAC_FORMAL, v, NULL, NULL);
24 }

```

Listing 第二章.85 tac.c: 指针与数组形参声明

作为语句的调用会先串接所有实参表达式的代码，再从左到右生成 TAC_ACTUAL（最终相当于把第一个形参放在最后，为后端栈布局相匹配），最后发出 TAC_CALL 跳转。因为不关心返回值，所以 TAC_CALL 的 a=NULL。

```

1 TAC *do_call(char *name, EXP *arglist)
2 {
3     EXP *alt; /* For counting args */
4     TAC *code; /* Resulting code */
5     TAC *temp; /* Temporary for building code */
6
7     code = NULL;
8     for (alt = arglist; alt != NULL; alt = alt->next)
9         code = join_tac(code, alt->tac);
10
11     /* 按语法从左到右生成
        ACTUAL，保持最后一个为第一个形参（与现有帧布局匹配） */
12     while (arglist != NULL)
13     {
14         temp = mk_tac(TAC_ACTUAL, arglist->ret, NULL, NULL);
15         temp->prev = code;
16         code = temp;
17         arglist = arglist->next;

```

```

18     }
19
20     temp = mk_tac(TAC_CALL, NULL, (SYM *)strdup(name), NULL);
21     temp->prev = code;
22     code = temp;
23
24     return code;
25 }

```

Listing 第二章.86 tac.c: 作为语句的函数调用

作为表达式的调用除了 ACTUAL + CALL 外，先为返回值分配临时并以返回类型（从被调函数符号中获取）标注，TAC_CALL 的 a=ret 表示返回值需要被填充，后端按调用约定把返回值从专用寄存器/位置搬入该临时。

```

1 EXP *do_call_ret(char *name, EXP *arglist)
2 {
3     EXP *alt;
4     SYM *ret;
5     TAC *code;
6     TAC *temp;
7
8     /* 根据被调函数设置返回值类型 */
9     SYM *f = lookup_sym(sym_tab_global, name);
10    int rtype = T_INT;
11    if (f && f->type == SYM_FUNC)
12        rtype = f->dtype;
13
14    ret = mk_tmp();
15    ret->dtype = rtype;
16
17    code = mk_tac(TAC_VAR, ret, NULL, NULL);
18
19    for (alt = arglist; alt != NULL; alt = alt->next)
20        code = join_tac(code, alt->tac);
21
22    while (arglist != NULL)
23    {
24        temp = mk_tac(TAC_ACTUAL, arglist->ret, NULL, NULL);
25        temp->prev = code;

```

```

26     code = temp;
27     arglist = arglist->next;
28 }
29
30 temp = mk_tac(TAC_CALL, ret, (SYM *)strdup(name), NULL);
31 temp->prev = code;
32 code = temp;
33
34 return mk_exp(NULL, ret, code);
35 }

```

Listing 第二章.87 tac.c: 作为表达式的函数调用

2.9.2.4 obj.h

寄存器约定中，R_BP 为基址指针，R_TP 被本项目用于函数返回值寄存器。帧布局从 LOCAL_OFF (8) 开始存放局部变量，OBP_OFF 和 RET_OFF 分别是旧基址与返回地址，形参从 FORMAL_OFF (-4) 向负地址扩展。

```

1  /* register */
2  #define R_UNDEF -1
3  #define R_FLAG 0
4  #define R_IP 1
5  #define R_BP 2
6  #define R_JP 3
7  #define R_TP 4
8  #define R_GEN 5
9  #define R_NUM 16
10
11 /* frame /
12 #define FORMAL_OFF -4 / first formal parameter /
13 #define OBP_OFF 0 / dynamic chain /
14 #define RET_OFF 4 / ret address /
15 #define LOCAL_OFF 8 / local var */

```

Listing 第二章.88 obj.h: 函数相关寄存器与帧布局

2.9.2.5 obj.c

调用方序列。先回写并清理一般寄存器，然后把旧基址与返回地址压入当前帧顶部的调用区域（由 tof+oon 指定），再把 R2（bp）提升到新帧基址并跳转到被调函数的入口标签。返回点（标签）处，如果调用需要返回值（a!=NULL，即表达式调用），就从 R_TP 取回返回值装入目标寄存器绑定到那个临时。oon 是“下一次调用临时区域”的累计偏移，调用结束清零。

```

1 void asm_call(SYM *a, SYM *b)
2 {
3     int r;
4     for (int r = R_GEN; r < R_NUM; r++)
5         asm_write_back(r);
6     for (int r = R_GEN; r < R_NUM; r++)
7         rdesc_clear(r);
8     out_str(file_s, "\tSTO (R2+%d),R2\n", tof + oon); /* store old
9         bp */
10    oon += 4;
11    static unsigned ret_lbl_seq = 0;
12    char retlbl[32];
13    /* 生成仅含 L+数字 的标签名，兼容汇编器对立即数标签的语法限制
14       */
15    sprintf(retlbl, "L%u", 90000u + (++ret_lbl_seq));
16    out_str(file_s, "\tLOD R4,%s\n", retlbl); /* return
17        addr: 标签地址 */
18    out_str(file_s, "\tSTO (R2+%d),R4\n", tof + oon); /* store
19        return addr */
20    oon += 4;
21    out_str(file_s, "\tLOD R2,R2+%d\n", tof + oon - 8); /* load
22        new bp */
23    out_str(file_s, "\tJMP %s\n", (char *)b); /* jump to
24        new func */
25    out_str(file_s, "%s:\n", retlbl); /* return
26        label */
27    if (a != NULL)
28    {
29        r = reg_alloc(a);
30        out_str(file_s, "\tLOD R%u,R%u\n", r, R_TP);
31        rdesc[r].mod = MODIFIED;

```

```

25     }
26     oon = 0;
27 }

```

Listing 第二章.89 obj.c: 函数调用生成

被调函数返回序列。若有返回值，将其载入 R_TP（调用约定的返回寄存器）以供调用方在返回点读取；随后弹出返回地址到 R3 并恢复基址 R2，最后跳转到返回地址。

```

1 void asm_return(SYM *a)
2 {
3     for (int r = R_GEN; r < R_NUM; r++)
4         asm_write_back(r);
5     for (int r = R_GEN; r < R_NUM; r++)
6         rdesc_clear(r);
7     if (a != NULL) /* return value */
8     {
9         asm_load(R_TP, a);
10    }
11
12    out_str(file_s, "    LOD R3,(R2+4)\n"); /* return address */
13    out_str(file_s, "    LOD R2,(R2)\n");    /* restore bp */
14    out_str(file_s, "    JMP R3\n");        /* return */
15 }

```

Listing 第二章.90 obj.c: 函数返回生成

函数相关的三地址码在后端的翻译：TAC_ACTUAL 把实参（寄存器中的值）按顺序压入当前帧顶部的调用工作区；TAC_BEGINFUNC 重置帧偏移（tof 本地起点 8，oof 形参起点 -4）并进入局部作用域；TAC_FORMAL 自高到低为每个形参分配偏移；TAC_CALL 调用被调序列；TAC_RETURN/TAC_ENDFUNC 完成返回与作用域退出；TAC_LABEL 作为函数入口与中间跳转标签，在生成前清理与回写寄存器，保证跨边界一致性。

```

1 case TAC_LABEL:
2     for (int r = R_GEN; r < R_NUM; r++)
3         asm_write_back(r);
4     for (int r = R_GEN; r < R_NUM; r++)
5         rdesc_clear(r);
6     out_str(file_s, "%s:\n", c->a->name);

```

```
7     return;
8
9     case TAC_ACTUAL:
10         r = reg_alloc(c->a);
11         out_str(file_s, "    STO (R2+%d),R%u\n", tof + oon, r);
12         oon += 4;
13         return;
14
15     case TAC_CALL:
16         asm_call(c->a, c->b);
17         return;
18
19     case TAC_BEGINFUNC:
20         /* We reset the top of stack, since it is currently empty
21            apart from the link information. */
22         scope = 1;
23         tof = LOCAL_OFF;
24         oof = FORMAL_OFF;
25         oon = 0;
26         return;
27
28     case TAC_FORMAL:
29         c->a->scope = 1; /* parameter is special local var */
30         c->a->offset = oof;
31         oof -= 4;
32         return;
33
34     case TAC_RETURN:
35         asm_return(c->a);
36         return;
37
38     case TAC_ENDFUNC:
39         asm_return(NULL);
40         scope = 0;
41         return;
```

Listing 第二章.91 obj.c: 函数相关三地址码翻译

程序入口序言。初始化基址寄存器 R2 指向 STACK，设置动态链与返回地址，

然后无条件跳往 `main` 的入口标签，作为顶层函数启动点。与函数机制配合，使 `main` 像普通函数一样执行。

```
1 void asm_head()  
2 {  
3     char head[] =  
4         "    # head\n"  
5         "    LOD R2,STACK\n"  
6         "    STO (R2),0\n"  
7         "    LOD R4,EXIT\n"  
8         "    STO (R2+4),R4\n"  
9         "    JMP main\n"; // 入口跳到 main  
10    out_str(file_s, "%s", head);  
11 }
```

Listing 第二章.92 obj.c: 程序入口序言生成

以上为该功能实现的重要代码片段展示与详细说明。

第三章 性能优化部分的实现

本章将详细介绍对于基础版本编译器进行性能优化采用的各类优化方法的实现思路及代码细节。

本章采用的各类优化方法的实现都建立在对源程序生成的三地址码进行程序控制流图构建以及数据流分析的基础上，因此，这里首先介绍对于程序控制流图构建的实现以及用到的三种数据流分析方法。

3.1 程序控制流图

3.1.1 实现思路

本项目的控制流图构建以线性三地址码为输入，先在语义上将顺序代码划分为基本块，再把这些块用显式的控制转移关系连成图。具体做法是：首先进行一次顺序扫描，借助块头与块尾的语义规则切分基本块：凡是函数入口或出现显式标签的位置作为块头，遇到无条件跳转、条件跳转、返回或函数结束等控制转移指令则作为块尾；在切分的同时把每个块起始处出现的标签名与该块的编号建立映射，供后续解析跳转目标使用。

随后进行一次连边过程：考察每个基本块最后一条指令，根据其控制转移语义确定后继，像无条件跳转将后继指向对应标签的目标块，条件跳转形成两条边（分支目标与顺序落下的后继），返回/函数结束没有后继，而普通语句按顺序落下连接到下一个块；这里对条件的落下语义通过块的物理顺序自然表达。

完成后再从所有后继关系反向构建前驱集合，得到一个既有前驱也有后继的信息完备的图。

整个过程不复制或重排 TAC 序列，只以 `first/last` 指针把原有线性指令区间投影为基本块，保证构建轻量且与优化器共享同一份 IR；图以整数块编号和简洁链表表示，用以做数据流分析，并生成可读的 `.cfg.txt` 用于校对与调试。

3.1.2 代码细节

3.1.2.1 `cfg.h`

定义供优化器只读访问的基本块结构与前驱列表，并提供 CFG 构建与导出接口。

```
1 /* 公开 CFG 结构，便于优化器遍历基本块 */
2 typedef struct Pred Pred;
```

```

3 struct Pred
4 {
5     int id;           /* 前驱块的编号 */
6     struct Pred *next; /* 链表下一个前驱 */
7 };
8
9 typedef struct Block Block;
10 struct Block
11 {
12     int id;           /* 基本块编号 */
13     TAC *first;       /* 块起始三地址码 */
14     TAC *last;       /* 块结束三地址码 */
15     int succ1;        /* 第一后继块编号 (-1 表示无) */
16     int succ2;        /* 第二后继块编号 (-1 表示无) */
17     char *labelName; /* 若以标签开始, 记录标签名 */
18     Pred *preds;      /* 前驱块链表 */
19     Block *next;      /* 下一基本块 */
20 };
21
22 /* 构建并导出 CFG 文本到 .cfg.txt 文件 */
23 void cfg_build_and_dump_txt(const char *input_m_filename);
24
25 /* 仅构建 CFG (不输出), 供优化器使用 */
26 void cfg_build_only(void);
27
28 /* 获取基本块链表头指针 (只读遍历用) */
29 Block *cfg_blocks_head(void);

```

Listing 第三章.1 cfg.h: 控制流图基本块结构与接口

Block: 保存基本块的 id、首末三地址码指针、两个后继编号、可选标签名、前驱链表与顺序上的下一个块;

Pred: 采用链表记录前驱块编号, 便于可变数量的前驱维护 (如分支汇合)。

3.1.2.2 cfg.c

通过 LabelMap 将标签名关联到基本块编号, 便于第二趟连边时解析跳转目标。blocks_head/blocks_tail 维护块链表, block_count 连续分配块 ID。

```

1 typedef struct LabelMap LabelMap;

```

```

2 struct LabelMap
3 {
4     char *name;
5     /*
6      * 标签映射（名称 -> 基本块 id）
7      * 在构块第一趟中建立，供跳转目标解析使用。
8      */
9     int blockId;
10    LabelMap *next;
11 };
12 static Block *blocks_head = NULL, *blocks_tail = NULL;
13 static int block_count = 0;
14 static LabelMap *labels = NULL;

```

Listing 第三章.2 cfg.c: 控制流图全局状态

函数 `is_leader` 用以判断 `TAC_LABEL` 与 `TAC_BEGINFUNC` 为新块起点。标签映射采用简单链表，足以应对常规模块数量。

```

1 static int is_leader(TAC *t)
2 {
3     if (!t)
4         return 0;
5     /*
6      * 判断一条 TAC 是否为基本块 leader:
7      * - 标签 (TAC_LABEL);
8      * - 函数起点 (TAC_BEGINFUNC)。
9      */
10    if (t->op == TAC_LABEL)
11        return 1;
12    if (t->op == TAC_BEGINFUNC)
13        return 1;
14    return 0;
15 }
16
17 static void labelmap_put(char *name, int blockId)
18 {
19     LabelMap *m = (LabelMap *)malloc(sizeof(LabelMap));
20     m->name = name;
21     /* 将标签名与块 id 关联（头插链表） */

```

```

22     m->blockId = blockId;
23     m->next = labels;
24     labels = m;
25 }
26
27 static int labelmap_get(char *name)
28 {
29     for (LabelMap *m = labels; m; m = m->next)
30     {
31         /* 通过标签名查找对应块 id, 找不到返回 -1 */
32         if (m->name && name && strcmp(m->name, name) == 0)
33             return m->blockId;
34     }
35     return -1;
36 }

```

Listing 第三章.3 cfg.c: 块头判断与标签映射管理

基本块创建与第一趟构块, 构块规则为遇终止指令或下一条是 leader 时切块。块的 labelName 仅在首指令为标签时记录, 用于后续跳转解析。succ1/succ2 在这里尚未赋值, 留到第二趟。

```

1 static Block *new_block(TAC *first)
2 {
3     Block *b = (Block *)calloc(1, sizeof(Block));
4     b->id = block_count++;
5     /* 新建一个以 first 为首指令的基本块, 并接到块链表尾部 */
6     b->first = first;
7     b->last = first;
8     b->succ1 = -1;
9     b->succ2 = -1;
10    b->labelName = NULL;
11    b->preds = NULL;
12    b->next = NULL;
13    if (!blocks_head)
14        blocks_head = b;
15    else
16        blocks_tail->next = b;
17    blocks_tail = b;
18    return b;

```

```
19 }
20
21 static void build_blocks(void)
22 {
23     blocks_head = blocks_tail = NULL;
24     block_count = 0;
25     /*
26      * 第一趟：按 leader/终止条件切分基本块，并记录起始标签到块 id
27      * 的映射。
28      * 终止条件包括：GOTO、IFZ、RETURN、ENDFUNC；
29      * 下一条指令是 leader 也会开新块。
30      */
31     labels = NULL;
32
33     if (!tac_first)
34         return;
35     TAC *cur = tac_first;
36     Block *currBlock = new_block(cur);
37     for (; cur; cur = cur->next)
38     {
39         if (cur == currBlock->first && cur->op == TAC_LABEL &&
40             cur->a && cur->a->name)
41         {
42             currBlock->labelName = cur->a->name;
43             labelmap_put(cur->a->name, currBlock->id);
44         }
45
46         currBlock->last = cur;
47
48         int terminates = 0;
49         switch (cur->op)
50         {
51             case TAC_GOTO:
52             case TAC_IFZ:
53             case TAC_RETURN:
54             case TAC_ENDFUNC:
55                 terminates = 1;
56                 break;
```

```

55     default:
56         break;
57     }
58
59     TAC *nxt = cur->next;
60     int next_is_leader = is_leader(nxt);
61     if (terminates || next_is_leader)
62     {
63         if (nxt)
64         {
65             currBlock = new_block(nxt);
66         }
67     }
68 }
69 }

```

Listing 第三章.4 cfg.c: 基本块创建与第一趟构块

块检索与顺序访问，块以链表顺序排列，`get_next_block` 表示语义上的顺序落下块，用于默认 `fallthrough` 连边。

```

1 static Block *get_block_by_id(int id)
2 {
3     for (Block *b = blocks_head; b; b = b->next)
4         if (b->id == id)
5             /* 按 id 在线性块链表中查找块 */
6             return b;
7     return NULL;
8 }
9
10 static Block *get_next_block(Block *b)
11 {
12     return b ? b->next : NULL;
13 }
14 /* 获取顺序意义上的下一个块 */

```

Listing 第三章.5 cfg.c: 块检索与顺序访问

第二趟后继连边，采用双后继位覆盖最常见情况：无条件跳转、条件跳转（分支 + 落下）、顺序落下。复杂控制转移可延伸此模型或引入后继列表。

```
1 static void wire_successors(void)
2 {
3     for (Block *b = blocks_head; b; b = b->next)
4     {
5         /*
6          * 第二趟：根据每个块最后一条指令连好后继边。
7          * - GOTO: succ1 指向标签目标；
8          * - IFZ: succ1 指向分支目标，succ2 指向顺序落下块；
9          * - RETURN/ENDFUNC: 无后继；
10         * - 其他: succ1 指向顺序落下块。
11         */
12         TAC *last = b->last;
13         if (!last)
14             continue;
15         switch (last->op)
16         {
17             case TAC_GOTO:
18                 if (last->a && last->a->name)
19                     b->succ1 = labelmap_get(last->a->name);
20                 break;
21             case TAC_IFZ:
22                 if (last->a && last->a->name)
23                     b->succ1 = labelmap_get(last->a->name);
24                 if (get_next_block(b))
25                     b->succ2 = get_next_block(b)->id;
26                 break;
27             case TAC_RETURN:
28             case TAC_ENDFUNC:
29                 break;
30             default:
31                 if (get_next_block(b))
32                     b->succ1 = get_next_block(b)->id;
33                 break;
34         }
35     }
36 }
```

Listing 第三章.6 cfg.c: 第二趟后继连边

第三趟前驱反向构建，为防止重复前驱条目，add_pred 在插入前进行去重。第三趟先释放旧链表，确保数据与最新连边一致。

```
1 static void add_pred(int toId, int fromId)
2 {
3     Block *to = get_block_by_id(toId);
4     if (!to)
5         return;
6     /* 给目标块增加一个前驱记录（去重） */
7     Pred *p = (Pred *)malloc(sizeof(Pred));
8     for (Pred *q = to->preds; q; q = q->next)
9         if (q->id == fromId)
10             {
11                 free(p);
12                 return;
13             }
14     p->id = fromId;
15     p->next = to->preds;
16     to->preds = p;
17 }
18
19 static void build_predecessors(void)
20 {
21     for (Block *b = blocks_head; b; b = b->next)
22     {
23         Pred *p = b->preds;
24         /*
25          * 第三趟：先清空每个块的前驱链表，再依据 succ1/succ2
26          * 反向构建 preds。
27          */
28         while (p)
29         {
30             Pred *n = p->next;
31             free(p);
32             p = n;
33         }
34         b->preds = NULL;
35     }
```

```
36     for (Block *b = blocks_head; b; b = b->next)
37     {
38         if (b->succ1 >= 0)
39             add_pred(b->succ1, b->id);
40         if (b->succ2 >= 0)
41             add_pred(b->succ2, b->id);
42     }
43 }
```

Listing 第三章.7 cfg.c: 第三趟前驱反向构建

对外入口与访问接口, 优化器调用 `cfg_build_only` 以构建 CFG, 然后通过 `cfg_blocks_head` 进行只读遍历。编译器主流程可用 `cfg_build_and_dump_txt` 生成 `cfg.txt` 文件用于调试与分析。生成 `cfg.txt` 的相关函数就不进行详细说明, 只用于调试与优化过程无关。

```
1  /* 构建并导出 CFG 的对外入口 */
2  void cfg_build_and_dump_txt(const char *input_m_filename)
3  {
4      build_blocks();
5      wire_successors();
6      build_predecessors();
7      dump_txt(input_m_filename);
8  }
9
10 /* 仅构建 CFG, 供优化器调用 */
11 void cfg_build_only(void)
12 {
13     build_blocks();
14     wire_successors();
15     build_predecessors();
16 }
17
18 /* 返回块链表头指针 (供只读遍历) */
19 Block *cfg_blocks_head(void)
20 {
21     return blocks_head;
22 }
```

Listing 第三章.8 cfg.c: 对外入口与访问接口

3.2 数据流分析

在拥有程序的控制流图之后，就可以进行数据流分析，在本项目的优化方法中，总的使用了以下的三种数据流分析方法，每一种数据流分析方法都有其转换函数（transfer function）以及控制流（control flow）。

3.2.1 Reaching Definitions

到达定义分析用于确定在程序的某个点上，哪些变量定义可能影响该点的值。通过分析基本块的前驱和后继关系，计算每个基本块的 IN 和 OUT 集合，从而确定变量的活跃范围。

转换函数定义了如何根据基本块内的指令更新 IN 和 OUT 集合。具体来说，IN 集合表示在进入基本块时可能到达的定义，而 OUT 集合表示在离开基本块时可能到达的定义。

控制流定义了数据流分析的方向以及 IN 集合的生成方式。在到达定义分析中，控制流是从基本块的前驱到后继进行传播的，即从 IN 集合到 OUT 集合。

Reaching Definitions 的 transfer function 为下式：

$$OUT[B] = GEN[B] \cup (IN[B] - KILL[B]) \quad (3-1)$$

其中，GEN[B] 表示在基本块 B 中生成的定义集合，KILL[B] 表示在基本块 B 中被覆盖的定义集合。

Reaching Definitions 是一种 may analysis，就是说该方法分析出的问题集会包含真问题集，可能会误报但不会漏报，control flow 为：

$$IN[B] = \bigcup_{P \in pred(B)} OUT[P] \quad (3-2)$$

其中，pred(B) 表示基本块 B 的前驱集合，这意味着 Reaching Definitions 的分析由程序入口向程序出口传播，IN 集合是前驱块的 OUT 集合的并集。

算法流程如下：

算法 3-1 Reaching Definitions

输入： 控制流图 CFG，基本块集合 $\mathcal{B}$ ，每块的 GEN 与 KILL

输出： 每个基本块入口 IN[B] 和出口 OUT[B] 的到达定值集合

- 1: 对所有块 B：初始化 $IN[B] := \emptyset$ ， $OUT[B] := GEN[B]$
- 2: 选择迭代顺序（例如逆后序），设置 $changed := true$
- 3: **while** $changed$ **do**
- 4: $changed := false$

```

5:  for 每个基本块  $B$  按固定顺序 do
6:       $IN[B] := \bigcup_{P \in \text{pred}(B)} OUT[P]$ 
7:       $newOUT := GEN[B] \cup (IN[B] \setminus KILL[B])$ 
8:      if  $newOUT \neq OUT[B]$  then
9:           $OUT[B] := newOUT; changed := true$ 
10:     end if
11:  end for
12: end while
13: 返回所有  $IN[B], OUT[B]$ 
    
```

3.2.2 Live Variable Analysis

活跃变量分析用于确定在程序的某个点上，哪些变量处于存活状态（当一个变量被定义且到该点未被重定义且被使用，则它是存活的）。通过分析基本块的前驱和后继关系，计算每个基本块的 IN 和 OUT 集合，从而确定变量的活跃范围。

转换函数定义了如何根据基本块内的指令更新 IN 和 OUT 集合。具体来说， IN 集合表示在进入基本块时活跃的变量，而 OUT 集合表示在离开基本块时活跃的变量。

控制流定义了数据流分析的方向以及 OUT 集合的生成方式。在活跃变量分析中，控制流是从基本块的后继到前驱进行传播的，即从 OUT 集合到 IN 集合。

Live Variable Analysis 的 transfer function 为下式：

$$IN[B] = USE[B] \cup (OUT[B] - DEF[B]) \quad (3-3)$$

其中， $USE[B]$ 表示在基本块 B 中使用的变量集合， $DEF[B]$ 表示在基本块 B 中定义的变量集合。

Live Variable Analysis 与 Reaching Definitions 相同，也是一种 analysis，control flow 为：

$$OUT[B] = \bigcup_{S \in \text{succ}(B)} IN[S] \quad (3-4)$$

其中， $\text{succ}(B)$ 表示基本块 B 的后继集合，这意味着 Live Variable Analysis 的分析由程序出口向程序入口传播， OUT 集合是后继块的 IN 集合的并集。

需要注意的是，在某一基本块中，被重新定义与被使用的变量如果为同一个，则需要考虑重新定义的影响，即在计算 IN 集合时，需要先从 OUT 集合中去除 DEF 集合中的变量，然后再加入 USE 集合中的变量。

算法流程如下：

算法 3-2 Live Variable Analysis

输入： 控制流图 *CFG*，基本块集合 *B*，每块的 *USE* 与 *DEF*

输出： 每个基本块入口 *IN[B]* 和出口 *OUT[B]* 的活跃变量集合

- 1: 对所有块 *B*：初始化 *OUT[B] := ∅*, *IN[B] := USE[B]*（或空集）
 - 2: 选择迭代顺序（例如逆后序），设置 *changed := true*
 - 3: **while** *changed* **do**
 - 4: *changed := false*
 - 5: **for** 每个基本块 *B* 按固定顺序 **do**
 - 6: $OUT[B] := \bigcup_{S \in \text{succ}(B)} IN[S]$
 - 7: $newIN := USE[B] \cup (OUT[B] \setminus DEF[B])$
 - 8: **if** *newIN* $\neq$ *IN[B]* **then**
 - 9: *IN[B] := newIN*; *changed := true*
 - 10: **end if**
 - 11: **end for**
 - 12: **end while**
 - 13: 返回所有 *IN[B]*, *OUT[B]*
-

3.2.3 Available Expressions Analysis

可用表达式分析用于确定在程序的某个点上，哪些表达式的值已经被计算并且在该点之前没有被重新计算或修改（即组成表达式的变量是否都存活）。通过分析基本块的前驱和后继关系，计算每个基本块的 *IN* 和 *OUT* 集合，从而确定表达式的可用范围。

转换函数定义了如何根据基本块内的指令更新 *IN* 和 *OUT* 集合。具体来说，*IN* 集合表示在进入基本块时可用的表达式，而 *OUT* 集合表示在离开基本块时可用的表达式。

控制流定义了数据流分析的方向以及 *IN* 集合的生成方式。在可用表达式分析中，控制流是从基本块的前驱到后继进行传播的，即从 *IN* 集合到 *OUT* 集合。

Available Expressions Analysis 的 transfer function 为下式：

$$OUT[B] = GEN[B] \cup (IN[B] - KILL[B]) \quad (3-5)$$

其中，*GEN[B]* 表示在基本块 *B* 中生成的表达式集合，*KILL[B]* 表示在基本块 *B* 中被修改的表达式集合。

Available Expressions Analysis 是一种 *must analysis*，与前两种不同，该方法分析出的问题集是真问题集的子集，不会误报但可能会漏报，control flow 为：

$$IN[B] = \bigcap_{P \in pred(B)} OUT[P] \quad (3-6)$$

其中， $pred(B)$ 表示基本块 B 的前驱集合，这意味着 Available Expressions Analysis 的分析由程序入口向程序出口传播， IN 集合是前驱块的 OUT 集合的交集。

算法流程如下：

算法 3-3 Available Expressions Analysis

输入： 控制流图 CFG ，基本块集合 B ，每块的 GEN 与 $KILL$

输出： 每个基本块入口 $IN[B]$ 和出口 $OUT[B]$ 的可用表达式集合

- 1: 对所有块 B ：初始化 $IN[B] := U$ （全域表达式集合或按边界约定）
 - 2: 同时初始化 $OUT[B] := GEN[B] \cup (IN[B] \setminus KILL[B])$
 - 3: 选择迭代顺序（例如逆后序），设置 $changed := true$
 - 4: **while** $changed$ **do**
 - 5: $changed := false$
 - 6: **for** 每个基本块 B 按固定顺序 **do**
 - 7: $IN[B] := \bigcap_{P \in pred(B)} OUT[P]$ （无前驱时按边界设定，通常为 U 或 $\emptyset$ ）
 - 8: $newOUT := GEN[B] \cup (IN[B] \setminus KILL[B])$
 - 9: **if** $newOUT \neq OUT[B]$ **then**
 - 10: $OUT[B] := newOUT$; $changed := true$
 - 11: **end if**
 - 12: **end for**
 - 13: **end while**
 - 14: 返回所有 $IN[B]$, $OUT[B]$
-

在介绍完程序控制流图以及使用到的三种数据流分析方法后，就可以具体的实践各类优化方法，下文将根据表格对本项目使用到的优化方法来逐个展开说明其实现思路与代码细节。以下优化方法除 LRU 之外都在 `opt.c` 文件中实现，使用 LRU 的寄存器优化对于 `obj.c` 文件进行了添加与修改，后文就不再赘述。

1) ConstantFolding	2) StrengthReduction
3) CopyPropagation	4) CSE
5) LICM	6) DCE
7) LRU	8) Clean

表 3-1 性能优化部分优化方法列表

3.3 ConstantFolding

3.3.1 实现思路

常量折叠的核心思路是把仍处在语义等价层面上的静态计算提前到编译阶段完成：把三地址码视为纯函数式的表达式图，在不改变副作用边界与控制流语义的前提下，将可判定为常量的子表达式直接求值并把结果改为字面量。具体做法是按照数据依赖来界定折叠的范围，仅处理算术与比较等纯表达式节点，避开可能携带或依赖副作用的操作（如 I/O、跳转、调用等），并在表达式两侧操作数均为编译期常量时进行替换。

3.3.2 代码细节

判定常量辅助函数 `is_const`，以语义层面定义常量的判定标准：对象 `s` 必须非空且其符号类型为 `SYM_INT`，对应三地址码系统中的整数字面量。

```

1 static int is_const(SYM *s)
2 {
3     return s && s->type == SYM_INT;
4 }

```

Listing 第三章.9 opt.c: 常量判定辅助函数

常量折叠主过程 `opt_constant_folding`，线性遍历整个三地址码链表 `tac_first -> ... -> tac_last`，并对两类表达式进行判定与替换。

第一类是二元算术与比较（`ADD/SUB/MUL/DIV/EQ/NE/LT/LE/GT/GE`），当两个操作数 `b` 与 `c` 都是编译期整型常量（`is_const`）时，即在编译器内计算出结果 `v`，然后将原指令更改为赋值 `COPY`，把 `a = op(b,c)` 变为 `a = const(v)`，从而保留定义-使用边界且不改变控制流。对除法 `DIV` 做了保守检查：当右操作数 `lc` 为零时直接跳过折叠，避免在编译期引入未定义行为。

第二类是取负 `NEG` 的一元常量折叠：当 `b` 是常量时，计算其相反数并同样改写为 `COPY`。

两类改写都通过 `mk_const(v)` 生成新的整型常量符号，并将三地址码精简。

```

1  /* ----- 常量折叠 (Constant Folding) -----
   */
2  void opt_constant_folding(void)
3  {
4      // fprintf(stderr, "OPT: constant folding start\n");
5      for (TAC *i = tac_first; i; i = i->next)
6      {
7          int op = i->op;
8          if (op == TAC_ADD || op == TAC_SUB || op == TAC_MUL || op
9              == TAC_DIV ||
10             op == TAC_EQ || op == TAC_NE || op == TAC_LT || op ==
11             TAC_LE || op == TAC_GT || op == TAC_GE)
12             {
13                 if (is_const(i->b) && is_const(i->c))
14                 {
15                     int lb = i->b->value;
16                     int lc = i->c->value;
17                     int v = 0;
18                     switch (op)
19                     {
20                         {
21                             case TAC_ADD:
22                                 v = lb + lc;
23                                 break;
24                             case TAC_SUB:
25                                 v = lb - lc;
26                                 break;
27                             case TAC_MUL:
28                                 v = lb * lc;
29                                 break;
30                             case TAC_DIV:
31                                 if (lc != 0)
32                                     v = lb / lc;
33                                 else
34                                     continue;
35                                 break;
36                             case TAC_EQ:
37                                 v = (lb == lc);

```

```
35         break;
36     case TAC_NE:
37         v = (lb != lc);
38         break;
39     case TAC_LT:
40         v = (lb < lc);
41         break;
42     case TAC_LE:
43         v = (lb <= lc);
44         break;
45     case TAC_GT:
46         v = (lb > lc);
47         break;
48     case TAC_GE:
49         v = (lb >= lc);
50         break;
51     }
52     i->op = TAC_COPY;
53     i->b = mk_const(v);
54     i->c = NULL;
55 }
56 }
57 else if (op == TAC_NEG)
58 {
59     if (is_const(i->b))
60     {
61         i->op = TAC_COPY;
62         i->b = mk_const(-(i->b->value));
63         i->c = NULL;
64     }
65 }
66 }
67 // fprintf(stderr, "OPT: constant folding done\n");
68 }
```

Listing 第三章.10 opt.c: 常量折叠主过程

以上为常量折叠优化方法的实现思路与代码细节说明。

3.4 StrengthReduction

3.4.1 实现思路

强度削减，当乘法的一侧是编译期可判定的常量、且该常量恰为 2 的幂时，将乘法在三地址码层面等价改写为一串加法的倍增序列，从而用更廉价的指令替代高强度运算；

具体而言，在一侧为常量、另一侧为变量的情形上生效，常量为 2 时把 $a=b*2$ 的乘法退化为 $a=b+b$ ，常量为 2 的 n 次幂 ($n \geq 2$) 时先令 $a=b+b$ 完成第一次倍增，再连续 $n-1$ 次执行 $a=a+a$ 完成逐级倍增。

3.4.2 代码细节

幂判定辅助函数 `is_power_of_two`，以位运算的判定方法确认整数 x 是否为 2 的幂：对于正数 x ，若 x 的二进制只有一位为 1，则满足 $x \& (x-1) == 0$ ，即 x 是 2 的幂。

```
1 static int is_power_of_two(int x)
2 {
3     return x > 0 && (x & (x - 1)) == 0;
4 }
```

Listing 第三章.11 opt.c: 幂判定辅助函数

整数对数计算 `log2_int`，返回 x 的整数对数（即 $\log_2(x)$ 的整数值），在已知 x 为 2 的 n 次幂的前提下，循环右移实现 n 的计数。强度削减中，当常量为 2 的 n 次幂且 $n \geq 2$ 时，需要生成 $n-1$ 次的倍增序列 ($a=a+a$)；即使用此函数得到 n 。

```
1 static int log2_int(int x)
2 {
3     int k = 0;
4     while (x > 1)
5     {
6         x >>= 1;
7         k++;
8     }
9     return k;
10 }
```

Listing 第三章.12 opt.c: 整数对数计算辅助函数

强度削减主过程 `opt_strength_reduction_basic`, 强度削减的主体逻辑。遍历整条三地址码链表, 首先筛选出乘法指令 `TAC_MUL`, 然后只在“常量 $\times$ 变量”或“变量 $\times$ 常量”时将常量的数值提取到 `c`, 将变量的一侧绑定到 `var`。若该常量不是 2 的幂或大于 32 则跳过。对于 `c == 2` 的特殊情况, 直接把乘法替换为一次加法 `a = var + var`。当 `c == 2` 的 n 次幂且 $n \geq 2$ 时, 先将当前乘法指令改写为首个倍增 `a = var + var`, 随后在链表中原地插入 $n-1$ 条 `a = a + a` 的加法指令。插入操作维护 `prev/next` 指针与 `tac_last`, 保证双向链表结构一致。最后将迭代指针 `i` 更新到插入序列的最后一个指令, 保持遍历进度。

```

1  void opt_strength_reduction_basic(void)
2  {
3      for (TAC *i = tac_first; i; i = i->next)
4      {
5          if (i->op != TAC_MUL)
6              continue;
7          int c = 0;
8          SYM *var = NULL;
9          if (is_const(i->b) && is_var(i->c))
10         {
11             c = i->b->value;
12             var = i->c;
13         }
14         else if (is_var(i->b) && is_const(i->c))
15         {
16             c = i->c->value;
17             var = i->b;
18         }
19         else
20             continue;
21
22         if (!is_power_of_two(c) || c > 32)
23             continue;
24
25         if (c == 2)
26         {
27             i->op = TAC_ADD;
28             i->b = var;
29             i->c = var;

```

```

30         continue;
31     }
32
33     int n = log2_int(c); /* c == 2^n, n>=2 */
34     /* 首先更改当前指令为a=b+b */
35     i->op = TAC_ADD;
36     i->b = var;
37     i->c = var;
38     TAC *last = i;
39     /* 之后插入 n-1 条 a=a+a */
40     for (int step = 2; step <= n; ++step)
41     {
42         TAC *add = mk_tac(TAC_ADD, i->a, i->a, i->a);
43         add->prev = last;
44         add->next = last->next;
45         if (last->next)
46             last->next->prev = add;
47         else
48             tac_last = add;
49         last->next = add;
50         last = add;
51     }
52     i = last;
53 }
54 }

```

Listing 第三章.13 opt.c: 强度削减主过程

以上为强度削减优化方法的实现思路与代码细节说明。

3.5 CopyPropagation

3.5.1 实现思路

本项目中复制传播为全局形式，建立在 CFG 和数据流分析上，先为所有有左值的定义编号，按块构造 gen/kill 集合，做一趟前向不动点求得每块入口/出口的 Reaching Definitions 集合，然后在块内顺序扫描，维护当前位置的可达集合；当看到一次变量使用时，只有当所有到达它的定义都是 COPY 且源完全一致才进行替换，否则在合流处就放弃，这是因为 Reaching Definitions 是一种 may analysis，直

接进行替换可能会因为程序流实际上不可达（误报）而使程序出错。

3.5.2 代码细节

全局的使用点替换辅助。在使用点尝试进行复制传播，接收当前点的位集 `curr`（表示可达的定义编号集合），逐个枚举置位的 `id`，仅保留那些左值 `lhs[id]` 恰好等于该使用处变量 `u` 的定义；并要求所有候选定义都是 `TAC_COPY`，且来源 `src` 完全一致。如果条件满足，就将使用处替换为该统一来源。返回值不显式，但通过修改 `*pp` 达成替换。

```

1 /* 根据到达的 COPY 定义（位集 curr），在使用处进行替换 */
2 static void rd_try_replace(SYM **pp, unsigned char *curr, SYM
   **lhs, TAC **defs, int dcount)
3 {
4     SYM *u = *pp;
5     if (!is_var(u))
6         return;
7     SYM *src = NULL;
8     int ok = -1; /* -1:未遇到候选; 0:不一致; 1:一致 */
9     for (int id = 0; id < dcount; ++id)
10    {
11        if (!curr[id])
12            continue;
13        if (lhs[id] != u)
14            continue;
15        TAC *dt = defs[id];
16        if (dt->op != TAC_COPY)
17        {
18            ok = 0;
19            break;
20        }
21        SYM *s = dt->b;
22        if (ok == -1)
23        {
24            src = s;
25            ok = 1;
26        }
27        else if (src != s)
28        {

```

```

29         ok = 0;
30         break;
31     }
32 }
33 if (ok == 1 && src)
34     *pp = src;
35 }

```

Listing 第三章.14 opt.c: 全局使用点替换辅助函数

遍历全局三地址码链表，收集所有有左值的定义指令（算术、比较、取负、复制、输入、调用），为每一个定义分配一个连续编号。其输出是两组并行数组：`defs[id]` 指向该定义的 TAC 节点，`lhs[id]` 指向该定义的左值符号。为后续的基于编号的位集分析提供基础数据结构。

```

1  /* 收集所有定义类 TAC，并分配 id */
2  static int collect_defs(TAC ***out_defs, SYM ***out_lhs)
3  {
4      int cap = 256, cnt = 0;
5      TAC **defs = (TAC **)malloc(cap * sizeof(TAC *));
6      SYM **lhs = (SYM **)malloc(cap * sizeof(SYM *));
7      for (TAC *t = tac_first; t; t = t->next)
8      {
9          switch (t->op)
10         {
11             case TAC_ADD:
12             case TAC_SUB:
13             case TAC_MUL:
14             case TAC_DIV:
15             case TAC_EQ:
16             case TAC_NE:
17             case TAC_LT:
18             case TAC_LE:
19             case TAC_GT:
20             case TAC_GE:
21             case TAC_NEG:
22             case TAC_COPY:
23             case TAC_INPUT:
24             case TAC_CALL:
25                 if (t->a)

```

```

26         {
27             if (cnt >= cap)
28             {
29                 cap *= 2;
30                 defs = (TAC **)realloc(defs, cap * sizeof(TAC
31                                     *));
32                 lhs = (SYM **)realloc(lhs, cap * sizeof(SYM
33                                     *));
34                 defs[cnt] = t;
35                 lhs[cnt] = t->a;
36                 cnt++;
37             }
38             break;
39         default:
40             break;
41     }
42     *out_defs = defs;
43     *out_lhs = lhs;
44     return cnt;
45 }

```

Listing 第三章.15 opt.c: 收集定义编号

全局复制传播主过程，先用 `collect_defs` 收集并编号所有定义，再为每个块分配位集 `gen/kill` 并填充（`gen[b][id]=1` 表示该块生成定义 `id`，`kill[b][id]=1` 表示该块的定义会杀死其他块的同名定义）；随后用迭代不动点求解每块的 `in/out`（`new_in` 为前驱 `out` 的并集；`new_out = gen U (new_in - kill)`），直到所有块的集合稳定。最后，按块线性遍历：维护一个工作位集 `curr`（从 `in[b]` 拷贝），在使用点调用 `rd_try_replace` 按严格语义尝试替换；当遇到定义指令时，先将同 `lhs` 的旧定义位清零，再把当前 `id` 置位，表示当前位置之后，到达该左值的定义为最新的这一个。收尾对所有位集矩阵逐行释放内存。

```

1 void opt_copy_propagation_global(void)
2 {
3     cfg_build_only();
4     TAC **defs = NULL;
5     SYM **lhs = NULL;

```

```

6   int dcount = collect_defs(&defs, &lhs);
7   if (dcount == 0)
8   {
9       free(defs);
10      free(lhs);
11      return;
12  }
13  /* 为每个基本块构建定义 id 的 gen/kill 集合 (位集) */
14  int nblocks = blocks_count();
15  unsigned char **gen = (unsigned char **)malloc(nblocks *
16      sizeof(unsigned char *));
17  unsigned char **kill = (unsigned char **)malloc(nblocks *
18      sizeof(unsigned char *));
19  for (int b = 0; b < nblocks; ++b)
20  {
21      gen[b] = (unsigned char *)calloc(dcount, 1);
22      kill[b] = (unsigned char *)calloc(dcount, 1);
23  }
24  /* 为每个定义 id 找到其所属的块 */
25  int *def_block = (int *)malloc(dcount * sizeof(int));
26  for (int id = 0; id < dcount; ++id)
27  {
28      TAC *t = defs[id];
29      /* 找到包含该指令 t 的块 */
30      int bid = -1;
31      for (Block *b = cfg_blocks_head(); b; b = b->next)
32      {
33          for (TAC *x = b->first;; x = x->next)
34          {
35              if (x == t)
36              {
37                  bid = b->id;
38                  break;
39              }
40              if (x == b->last)
41                  break;
42          }
43      }
44      if (bid >= 0)

```

```

42         break;
43     }
44     def_block[id] = bid;
45     if (bid >= 0)
46         gen[bid][id] = 1;
47 }
48 /* 构建 kill: 对于每个块, 所有具有相同 LHS 且在其他块定义的 id
   都被该块杀死 */
49 for (int b = 0; b < nblocks; ++b)
50 {
51     for (int id = 0; id < dcount; ++id)
52     {
53         if (gen[b][id])
54             continue; /* 本块自身的定义不被 kill[b] 标记 */
55         SYM *v = lhs[id];
56         for (int gid = 0; gid < dcount; ++gid)
57         {
58             if (gen[b][gid] && lhs[gid] == v)
59             {
60                 kill[b][id] = 1;
61                 break;
62             }
63         }
64     }
65 }
66 /* RD 前向不动点: 基于定义 id 的 in[b]、out[b] 集合 (位集) */
67 unsigned char **in = (unsigned char **)malloc(nblocks *
68         sizeof(unsigned char *));
69 unsigned char **out = (unsigned char **)malloc(nblocks *
70         sizeof(unsigned char *));
71 for (int b = 0; b < nblocks; ++b)
72 {
73     in[b] = (unsigned char *)calloc(dcount, 1);
74     out[b] = (unsigned char *)calloc(dcount, 1);
75 }
76 int changed = 1;
77 int iter = 0;
78 while (changed)

```

```

77 {
78     iter++;
79     changed = 0;
80     for (Block *b = cfg_blocks_head(); b; b = b->next)
81     {
82         int bid = b->id;
83         unsigned char *new_in = (unsigned char
84             *)calloc(dcount, 1);
85         /* 前驱 out 的并集 (OR) */
86         if (b->preds)
87         {
88             for (Pred *p = b->preds; p; p = p->next)
89                 for (int id = 0; id < dcount; ++id)
90                     new_in[id] = (unsigned char)(new_in[id] |
91                         out[p->id][id]);
92         }
93         unsigned char *new_out = (unsigned char
94             *)malloc(dcount);
95         memcpy(new_out, gen[bid], dcount);
96         /* out = gen U (in - kill) */
97         for (int id = 0; id < dcount; ++id)
98         {
99             unsigned char avail = (unsigned char)(new_in[id]
100                 && !kill[bid][id]);
101             new_out[id] = (unsigned char)(new_out[id] | avail);
102         }
103         if (memcmp(new_in, in[bid], dcount) != 0 ||
104             memcmp(new_out, out[bid], dcount) != 0)
105         {
106             memcpy(in[bid], new_in, dcount);
107             memcpy(out[bid], new_out, dcount);
108             changed = 1;
109         }
110         free(new_in);
111         free(new_out);
112     }
113 }
114
115 /* 逐块扫描，维护当前可达集合，并在使用处执行复制传播 */

```

```
110     for (Block *b = cfg_blocks_head(); b; b = b->next)
111     {
112         unsigned char *curr = (unsigned char *)malloc(dcount);
113         memcpy(curr, in[b->id], dcount);
114         for (TAC *t = b->first;; t = t->next)
115         {
116             /* 在该 TAC 中进行使用处替换 */
117             switch (t->op)
118             {
119                 case TAC_ADD:
120                 case TAC_SUB:
121                 case TAC_MUL:
122                 case TAC_DIV:
123                 case TAC_EQ:
124                 case TAC_NE:
125                 case TAC_LT:
126                 case TAC_LE:
127                 case TAC_GT:
128                 case TAC_GE:
129                     rd_try_replace(&t->b, curr, lhs, defs, dcount);
130                     rd_try_replace(&t->c, curr, lhs, defs, dcount);
131                     break;
132                 case TAC_NEG:
133                 case TAC_IFZ:
134                     rd_try_replace(&t->b, curr, lhs, defs, dcount);
135                     break;
136                 case TAC_OUTPUT:
137                     rd_try_replace(&t->a, curr, lhs, defs, dcount);
138                     break;
139                 default:
140                     break;
141             }
142
143             /* 更新当前集合：找到对应此指令的定义
144              * id，并应用杀死/加入 */
145             for (int id = 0; id < dcount; ++id)
146             {
147                 if (defs[id] == t)
```

```
147         {
148             /* remove killed defs for this block (defs
149                with same lhs) */
150             for (int k = 0; k < dcount; ++k)
151                 if (lhs[k] == lhs[id])
152                     curr[k] = 0;
153             curr[id] = 1;
154             break;
155         }
156     }
157     if (t == b->last)
158         break;
159 }
160 free(curr);
161 }
162 /* 清理分配的集合结构 */
163 for (int i = 0; i < nblocks; ++i)
164 {
165     free(gen[i]);
166     free(kill[i]);
167     free(in[i]);
168     free(out[i]);
169 }
170 free(gen);
171 free(kill);
172 free(in);
173 free(out);
174 free(def_block);
175 free(defs);
176 free(lhs);
177 }
```

Listing 第三章.16 opt.c: 全局复制传播主过程

以上为复制传播优化方法的实现思路与代码细节说明。。

3.6 CSE

3.6.1 实现思路

本项目中公共子表达式删除为全局形式，跨基本块用 Available Expressions Analysis 前向数据流分析，在所有前驱都保证该表达式可用且代表结果一致时，用拷贝替代后续计算，从而减少重复算术。具体来说，对每个基本块建立 $Gen[b]$ （本块内第一次出现的表达式）与 $Kill[b]$ （本块定义过的任何变量作为操作数都会杀死相关表达式）的位集，之后用标准不动点迭代计算 $In[b]$ （所有前驱 Out 的交集）与 $Out[b]$ 。再按块线性遍历，当某表达式在当前位置被判定为可用（存在于当前位集）且该表达式已有代表结果符号且与当前目标不同，就将该计算替换为一次拷贝，若当前是第一次定义该表达式，则把它标记为之后可用。

3.6.2 代码细节

候选表达式判定，明确可以参与 CSE 的候选表达式范围，仅限二元算术与比较（加、减、乘、除、相等、不等、大小关系），并且两个操作数必须是变量或常量。

```

1 static int expr_is_candidate(TAC *i)
2 {
3     int op = i->op;
4     switch (op)
5     {
6     case TAC_ADD:
7     case TAC_SUB:
8     case TAC_MUL:
9     case TAC_DIV:
10    case TAC_EQ:
11    case TAC_NE:
12    case TAC_LT:
13    case TAC_LE:
14    case TAC_GT:
15    case TAC_GE:
16        return (is_var(i->b) || is_const(i->b)) && (is_var(i->c)
17            || is_const(i->c));
18    default:
19        return 0;
20    }
21 }
```

Listing 第三章.17 opt.c: 候选表达式判定辅助函数

想要对表达式进行删除，首先要描绘其结构，ExprInfo 结构用于保存一个表达式在全局范围内的关键信息，key 是其规范化键（用于唯一标识表达式语义）；result 是首次计算该表达式得到的结果符号（代表符号）；b 和 c 是规范化顺序后的操作数指针；op 是操作码。使用这一结构让后续的集合与位集索引和匹配表达式，替换时取用代表符号。

```
1 typedef struct ExprInfo
2 {
3     char *key;
4     SYM *result;
5     SYM *b;
6     SYM *c;
7     int op;
8 } ExprInfo;
```

Listing 第三章.18 opt.c: 表达式信息结构

规范化表达式键, global_expr_key 为表达式生成规范化键，对可交换的操作（如加法、乘法、相等、不等）按指针地址顺序交换操作数指针，使 a+b 与 b+a 产生同一键，避免重复。键的格式是操作名 + 两个操作数指针地址的串，同时在交换后也更新 b/c 指针，保证与键一致性。

```
1 static char *global_expr_key(int op, SYM *b, SYM *c)
2 {
3
4     uintptr_t x = (uintptr_t)b, y = (uintptr_t)c;
5     if (is_commutative(op) && y < x)
6     {
7         uintptr_t t = x;
8         x = y;
9         y = t;
10        SYM *ts = b;
11        b = c;
12        c = ts;
13    }
14    char buf[128];
```

```

15     snprintf(buf, sizeof(buf), "%s:%p:%p", op_name(op), (void *)x,
16              (void *)y);
17     return strdup(buf);
18 }

```

Listing 第三章.19 opt.c: 规范化表达式键辅助函数

全局 CSE 的完整流程。

步骤一，遍历全局 TAC，使用 `expr_is_candidate` 判断并为每个表达式生成键，将首次出现的表达式记录为代表项，形成表达式全集 `exprs`。

步骤二，为每个基本块建立位集 `gen/kill`，遇到候选表达式将其键对应位记入 `gen`；同时收集本块定义过的变量，并标记所有以该被定义变量为操作数的表达式为 `kill`。

步骤三，执行前向数据流不动点迭代：每轮计算 `in[b]` 为所有前驱 `out` 的交集；再根据 `out = gen U (in - kill)` 更新 `out[b]`，直到所有块的 `in/out` 稳定。

步骤四，替换阶段：按块线性扫描，对于候选表达式，若其在当前位置可用 (`cur[idMatch] == 1`) 且已有代表符的结果不同于当前目标，就把该计算改写为 `TAC_COPY` 从代表结果拷贝；若当前是首次计算，则将该表达式标记为之后可用。同时在遇到对某变量的定义时，杀死所有以该变量为操作数的表达式可用位。最后统一释放构建的位集与表达式数组。

```

1 void opt_cse_global(void)
2 {
3     cfg_build_only();
4     int cap = 256, ecnt = 0;
5     ExprInfo *exprs = (ExprInfo *)malloc(cap * sizeof(ExprInfo));
6     for (TAC *t = tac_first; t; t = t->next)
7     {
8         if (expr_is_candidate(t))
9         {
10             char *k = global_expr_key(t->op, t->b, t->c);
11             int found = -1;
12             for (int i = 0; i < ecnt; ++i)
13                 if (strcmp(exprs[i].key, k) == 0)
14                 {
15                     found = i;
16                     break;
17                 }

```

```
18     if (found < 0)
19     {
20         if (ecnt >= cap)
21         {
22             cap *= 2;
23             exprs = (ExprInfo *)realloc(exprs, cap *
24                                     sizeof(ExprInfo));
25         }
26         exprs[ecnt].key = k;
27         exprs[ecnt].result = t->a; /*
28             首次定义该表达式的结果符号 */
29         exprs[ecnt].b = t->b;
30         exprs[ecnt].c = t->c;
31         exprs[ecnt].op = t->op;
32         ecnt++;
33     }
34     else
35     {
36         /* 保留已有代表项，释放冗余的 key */
37         free(k);
38     }
39 }
40 if (ecnt == 0)
41 {
42     free(exprs);
43     return;
44 }
45 int nblocks = blocks_count();
46 /* 每个基本块的位集合 (bitset) */
47 unsigned char **gen = (unsigned char **)malloc(nblocks *
48                                     sizeof(unsigned char *));
49 unsigned char **kill = (unsigned char **)malloc(nblocks *
50                                     sizeof(unsigned char *));
51 for (int b = 0; b < nblocks; ++b)
52 {
53     gen[b] = (unsigned char *)calloc(ecnt, 1);
54     kill[b] = (unsigned char *)calloc(ecnt, 1);
55 }
```

```
52     }
53     /* 2. Build gen/kill */
54     for (Block *b = cfg_blocks_head(); b; b = b->next)
55     {
56         /* 收集块内被定义的变量 */
57         SymSetNode *defined = NULL;
58         for (TAC *i = b->first;; i = i->next)
59         {
60             switch (i->op)
61             {
62                 case TAC_ADD:
63                 case TAC_SUB:
64                 case TAC_MUL:
65                 case TAC_DIV:
66                 case TAC_EQ:
67                 case TAC_NE:
68                 case TAC_LT:
69                 case TAC_LE:
70                 case TAC_GT:
71                 case TAC_GE:
72                 case TAC_NEG:
73                 case TAC_COPY:
74                 case TAC_INPUT:
75                 case TAC_CALL:
76                     set_add(&defined, i->a);
77                     break;
78                 default:
79                     break;
80             }
81             if (expr_is_candidate(i))
82             {
83                 /* 标记进入 Gen 集合 */
84                 char *k = global_expr_key(i->op, i->b, i->c);
85                 for (int id = 0; id < ecnt; ++id)
86                     if (strcmp(exprs[id].key, k) == 0)
87                     {
88                         gen[b->id][id] = 1;
89                         break;

```

```

90         }
91         free(k);
92     }
93     if (i == b->last)
94         break;
95 }
96 /* Kill: 任一操作数包含被定义变量的表达式 */
97 for (int id = 0; id < ecnt; ++id)
98 {
99     SYM *vb = exprs[id].b;
100    SYM *vc = exprs[id].c;
101    if ((is_var(vb) && set_has(defined, vb)) ||
102        (is_var(vc) && set_has(defined, vc)))
103        kill[b->id][id] = 1;
104 }
105 set_free(defined);
106 }
107 /* 第 3 步: 数据流不动点计算 (可用表达式) */
108 unsigned char **in = (unsigned char **)malloc(nblocks *
109     sizeof(unsigned char *));
110 unsigned char **out = (unsigned char **)malloc(nblocks *
111     sizeof(unsigned char *));
112 for (int b = 0; b < nblocks; ++b)
113 {
114     in[b] = (unsigned char *)calloc(ecnt, 1);
115     out[b] = (unsigned char *)calloc(ecnt, 1);
116 }
117 int changed = 1;
118 int entry = cfg_blocks_head() ? cfg_blocks_head()->id : -1;
119 while (changed)
120 {
121     changed = 0;
122     for (Block *b = cfg_blocks_head(); b; b = b->next)
123     {
124         int bid = b->id;
125         /* 计算 in[bid] = 所有前驱 out 的交集 */
126         unsigned char *new_in = (unsigned char *)malloc(ecnt);
127         if (!b->preds)

```

```

125     {
126         memset(new_in, 0, ecnt); /* entry or unreachable */
127     }
128     else
129     {
130         memset(new_in, 1, ecnt); /* start with all */
131         for (Pred *p = b->preds; p; p = p->next)
132             for (int id = 0; id < ecnt; ++id)
133                 new_in[id] = (unsigned char)(new_in[id] &
134                                         out[p->id][id]);
135     }
136     /* out = gen U (in - kill) */
137     unsigned char *new_out = (unsigned char *)malloc(ecnt);
138     for (int id = 0; id < ecnt; ++id)
139     {
140         unsigned char avail = (new_in[id] &&
141                             !kill[bid][id]);
142         new_out[id] = (unsigned char)(gen[bid][id] ||
143                                     avail);
144     }
145     if (memcmp(new_in, in[bid], ecnt) != 0 ||
146         memcmp(new_out, out[bid], ecnt) != 0)
147     {
148         memcpy(in[bid], new_in, ecnt);
149         memcpy(out[bid], new_out, ecnt);
150         changed = 1;
151     }
152     free(new_in);
153     free(new_out);
154 }
155 }
156
157 /* 第 4 步：替换遍历 */
158 for (Block *b = cfg_blocks_head(); b; b = b->next)
159 {
160     /* 当前块内的可用性位集 */
161     unsigned char *cur = (unsigned char *)malloc(ecnt);
162     memcpy(cur, in[b->id], ecnt);
163     for (TAC *i = b->first;; i = i->next)

```

```
159     {
160         if (expr_is_candidate(i))
161         {
162             char *k = global_expr_key(i->op, i->b, i->c);
163             int idMatch = -1;
164             for (int id = 0; id < ecnt; ++id)
165                 if (strcmp(exprs[id].key, k) == 0)
166                 {
167                     idMatch = id;
168                     break;
169                 }
170             if (idMatch >= 0 && cur[idMatch] &&
171                 exprs[idMatch].result != i->a)
172             {
173                 /* 用 COPY 替换原来的计算 */
174                 i->op = TAC_COPY;
175                 i->b = exprs[idMatch].result;
176                 i->c = NULL;
177             }
178             else if (idMatch >= 0)
179             {
180                 /* 此计算使该表达式在之后可用 */
181                 cur[idMatch] = 1;
182                 /* ensure representative symbol recorded (keep
183                    first) */
184             }
185             free(k);
186         }
187         /* 在变量定义处执行杀死 (Kill) */
188         switch (i->op)
189         {
190             case TAC_ADD:
191             case TAC_SUB:
192             case TAC_MUL:
193             case TAC_DIV:
194             case TAC_EQ:
195             case TAC_NE:
196             case TAC_LT:
```

```
195         case TAC_LE:
196         case TAC_GT:
197         case TAC_GE:
198         case TAC_NEG:
199         case TAC_COPY:
200         case TAC_INPUT:
201         case TAC_CALL:
202             if (is_var(i->a))
203             {
204                 for (int id = 0; id < ecnt; ++id)
205                 {
206                     SYM *vb = exprs[id].b;
207                     SYM *vc = exprs[id].c;
208                     if ((vb == i->a) || (vc == i->a))
209                         cur[id] = 0;
210                 }
211             }
212             break;
213         default:
214             break;
215     }
216     if (i == b->last)
217         break;
218 }
219 free(cur);
220 }
221 /* 清理分配的临时结构 */
222 for (int b = 0; b < nblocks; ++b)
223 {
224     free(gen[b]);
225     free(kill[b]);
226     free(in[b]);
227     free(out[b]);
228 }
229 free(gen);
230 free(kill);
231 free(in);
232 free(out);
```

```

233     for (int i = 0; i < ecnt; ++i)
234         free(exprs[i].key);
235     free(exprs);
236 }

```

Listing 第三章.20 opt.c: 全局 CSE 主过程

以上为公共子表达式删除优化方法的实现思路与代码细节说明。

3.7 LICM

3.7.1 实现思路

LICM（循环不变代码外提）的整体思路是：先用支配关系来识别回边，从而确定自然环，再把环里不需要每次循环都算的计算提到环前。

具体做法是先计算每个基本块的支配集，然后扫描所有边，凡是某块 b 的后继 h 同时又被 h 支配（也就是 $b \rightarrow h$ 是回边），就以 h 作为环头、 b 作为尾，沿着前驱把所有能回到 h 的块收集起来形成自然环。接着在这个环里统计被定义过的变量集合，用它来判断某条赋值是否在环内不变：

对于二元表达式（加减乘除与比较等）要求两个操作数在环内都没被重新定义，对于一元表达式和 COPY 要求右值在环内不被重新定义或是常量。

一旦确定可外提，则创建一个前序块（**preheader**）：通过在环头前插入一个新的标签 TAC，把所有来自环外、直达环头的跳转（GOTO/IFZ）重定向到新标签，同时保留回边不动，这样控制流就被自然地导向前序块，使外提的指令在进入环头之前执行。

外提的具体策略分两类：对临时变量的纯赋值计算，直接把相同计算搬到 **preheader** 并在环内把对该临时的使用替换为新外提结果；对非临时的完整赋值下沉/外提，在满足环内只定义一次、块内无定义前使用、右值环不变的条件也进行搬移，并统一替换环内使用点。

环和支配关系使用位集来做集合运算，但环内是否定义过则用简单的指针集合（SymSetNode）；

3.7.2 代码细节

计算当前控制流图（CFG）中基本块数量。从 `cfg_blocks_head()` 开始遍历链表中的每个 Block，持续更新遇到的最大块编号 `mx`，最终返回 `mx + 1` 作为块总数，该函数在支配关系与循环构造中用于分配位集的维度。

```

1 /* 支配关系与环构建的辅助函数 */

```

```

2 static int blocks_count(void)
3 {
4     int mx = -1;
5     for (Block *b = cfg_blocks_head(); b; b = b->next)
6         if (b->id > mx)
7             mx = b->id;
8     return mx + 1;
9 }

```

Listing 第三章.21 opt.c: 计算基本块数量辅助函数

`alloc_dom` 分配并初始化一个 $n \times n$ 的位矩阵，用于记录支配关系 `dom[b][k]`：当第 `b` 个块被第 `k` 个块支配时置 1。这里每一行用 `unsigned char *` 表示位向量，初始化为 0。`free_dom` 与之配套，释放逐行内存和行指针数组。位矩阵是后续 `compute_dominators` 迭代求解的核心数据结构。

```

1 static unsigned char **alloc_dom(int n)
2 {
3     unsigned char **dom = (unsigned char **)malloc(n *
4         sizeof(unsigned char *));
5     for (int i = 0; i < n; i++)
6     {
7         dom[i] = (unsigned char *)malloc(n);
8         memset(dom[i], 0, n);
9     }
10    return dom;
11 }
12 static void free_dom(unsigned char **dom, int n)
13 {
14     for (int i = 0; i < n; i++)
15         free(dom[i]);
16     free(dom);
17 }

```

Listing 第三章.22 opt.c: 支配关系位矩阵分配与释放辅助函数

按 `id` 在线性链表中查找并返回对应的基本块指针，若找不到则返回 `NULL`。用于根据块 `id` 进行索引的情况。

```

1 static Block *get_block_by_id_public(int id)
2 {

```

```

3   for (Block *b = cfg_blocks_head(); b; b = b->next)
4       if (b->id == id)
5           return b;
6   return NULL;
7 }

```

Listing 第三章.23 opt.c: 根据 id 查找基本块辅助函数

计算支配关系位矩阵。流程：先确保 CFG 已构建，再根据块数量 n 分配 `dom`。初始化时将入口块的自支配位置设为 1，其余块的行全部设为 1（假设起点为所有块支配目标块）。

随后进行迭代：对于每个非入口块，`newset` 初始为全 1，若没有前驱则置全 0；否则对每个前驱 p 的 `dom[p->id]` 做逐位与（交集），得到所有前驱可达的共同支配者。最后设置 `newset[id] = 1`（一个块总是支配自己），如果与旧值不同则更新并标记 `changed` 继续迭代。结束后通过输出参数返回位矩阵与大小。这是识别回边与自然环的前置分析。

```

1 static void compute_dominators(unsigned char ***pdom, int *pn)
2 {
3     cfg_build_only();
4     int n = blocks_count();
5     *pn = n;
6     unsigned char **dom = alloc_dom(n);
7     if (n == 0)
8     {
9         *pdom = dom;
10        return;
11    }
12    int entry = cfg_blocks_head()->id;
13    for (Block *b = cfg_blocks_head(); b; b = b->next)
14    {
15        int id = b->id;
16        if (id == entry)
17            dom[id][id] = 1;
18        else
19            memset(dom[id], 1, n);
20    }
21    int changed = 1;
22    while (changed)

```

```

23 {
24     changed = 0;
25     for (Block *b = cfg_blocks_head(); b; b = b->next)
26     {
27         int id = b->id;
28         if (id == entry)
29             continue;
30         unsigned char *newset = (unsigned char *)malloc(n);
31         memset(newset, 1, n);
32         if (!b->preds)
33             memset(newset, 0, n);
34         for (Pred *p = b->preds; p; p = p->next)
35             for (int k = 0; k < n; k++)
36                 newset[k] = (unsigned char)(newset[k] &
37                     dom[p->id][k]);
38         newset[id] = 1;
39         int diff = memcmp(newset, dom[id], n);
40         if (diff != 0)
41         {
42             memcpy(dom[id], newset, n);
43             changed = 1;
44         }
45         free(newset);
46     }
47     *pdom = dom;
48 }

```

Listing 第三章.24 opt.c: 计算支配关系主过程

LoopSet 用来表示一个自然环的成员集合与关键头尾信息：in 是大小为 n 的位集，标记属于该环的块；header 是环头，tail 是产生回边的尾块。build_natural_loop 接收头、尾、总块数，通过从 tail 倒着沿前驱回溯，收集所有能在保持 header 不入栈的情况下回到环头的块，将它们在位集里标 1。

其算法是经典的自然环构造：header 与 tail 先置为环内，然后在前驱图上做栈式 DFS/BFS，遇到未标记的前驱就置位并入栈，直到栈空。最终返回该环集合及其参数。

```
1 typedef struct LoopSet
```

```
2 {
3     unsigned char *in;
4     int n;
5     int header;
6     int tail;
7 } LoopSet;
8
9 static LoopSet build_natural_loop(int header, int tail, int n)
10 {
11     unsigned char *in = (unsigned char *)calloc(n, 1);
12     in[header] = 1;
13     in[tail] = 1;
14     int cap = 64, top = 0;
15     int *stack = (int *)malloc(cap * sizeof(int));
16     stack[top++] = tail;
17     while (top)
18     {
19         int x = stack[--top];
20         Block *bx = get_block_by_id_public(x);
21         for (Pred *p = bx->preds; p; p = p->next)
22         {
23             int pid = p->id;
24             if (!in[pid])
25             {
26                 in[pid] = 1;
27                 if (pid != header)
28                 {
29                     if (top >= cap)
30                     {
31                         cap *= 2;
32                         stack = (int *)realloc(stack, cap *
33                                                 sizeof(int));
34                     }
35                     stack[top++] = pid;
36                 }
37             }
38         }
39     }
```

```

39     free(stack);
40     LoopSet ls = {in, n, header, tail};
41     return ls;
42 }

```

Listing 第三章.25 opt.c: 自然环构造辅助函数

判断符号是否为编译器生成的临时变量，检测 `SYM_VAR` 且名字首字符为't'。在 LICM 中，优先外提临时变量的赋值，是因为这些属于中间计算值，外提风险更低、效果更直接。

```

1 static int sym_is_temp(SYM *s)
2 {
3     return s && s->type == SYM_VAR && s->name && s->name[0] == 't';
4 }

```

Listing 第三章.26 opt.c: 临时变量判定辅助函数

在 TAC 指令层面替换使用点：若某个操作数等于 `from`，则替换为 `to`。`replace_uses_in_tac` 根据不同操作，定位可能使用符号的位置（如二元算术与比较在 `b/c`，一元与条件在 `b`，输出在 `a`），逐一调用 `replace_use_with`。在 LICM 外提后，需要在环内把对原临时或变量的使用重定向到外提产生的新结果，确保语义一致。

```

1 static void replace_use_with(SYM **p, SYM *from, SYM *to)
2 {
3     if (*p == from)
4         *p = to;
5 }
6 static void replace_uses_in_tac(TAC *i, SYM *from, SYM *to)
7 {
8     switch (i->op)
9     {
10     case TAC_ADD:
11     case TAC_SUB:
12     case TAC_MUL:
13     case TAC_DIV:
14     case TAC_EQ:
15     case TAC_NE:
16     case TAC_LT:
17     case TAC_LE:
18     case TAC_GT:

```

```

19     case TAC_GE:
20         replace_use_with(&i->b, from, to);
21         replace_use_with(&i->c, from, to);
22         break;
23     case TAC_NEG:
24         replace_use_with(&i->b, from, to);
25         break;
26     case TAC_COPY:
27         replace_use_with(&i->b, from, to);
28         break;
29     case TAC_IFZ:
30         replace_use_with(&i->b, from, to);
31         break;
32     case TAC_OUTPUT:
33         replace_use_with(&i->a, from, to);
34         break;
35     default:
36         break;
37 }
38 }

```

Listing 第三章.27 opt.c: TAC 使用点替换辅助函数

判断一条指令是否为纯赋值（没有显式副作用的计算或复制）。限定 LICM 外提的候选类型：调用、输入输出、跳转、标签等含控制流或副作用的指令不会被当作可外提目标。纯赋值的集合包括所有算术、比较、一元负号以及复制。

```

1 static int tac_is_pure_assign(TAC *i)
2 {
3     switch (i->op)
4     {
5         case TAC_ADD:
6         case TAC_SUB:
7         case TAC_MUL:
8         case TAC_DIV:
9         case TAC_EQ:
10        case TAC_NE:
11        case TAC_LT:
12        case TAC_LE:
13        case TAC_GT:

```

```

14     case TAC_GE:
15     case TAC_NEG:
16     case TAC_COPY:
17         return 1;
18     default:
19         return 0;
20 }
21 }

```

Listing 第三章.28 opt.c: 纯赋值判定辅助函数

变量符号集合的单链表节点类型，每个节点保存一个 **SYM**（指向变量的符号结构），以及指向下一个节点的指针。LICM 使用该结构记录环内被定义过的变量集合，并做成员检测与并集等操作。

```

1  /* ----- 针对 SYM* 的简单集合结构 ----- */
2  typedef struct SymSetNode
3  {
4      SYM *s;
5      struct SymSetNode *next;
6  } SymSetNode;

```

Listing 第三章.29 opt.c: 符号集合节点结构

从头到尾线性扫描链表，判断集合中是否包含某个符号指针 **s**。比较使用指针相等（即同一变量对象），与 LICM 的不变性检查（操作数是否在环内定义过）对应，直接以 **SYM** 判断，无需映射成整数 **id**。

```

1  static int set_has(SymSetNode *h, SYM *s)
2  {
3      for (; h; h = h->next)
4          if (h->s == s)
5              return 1;
6      return 0;
7  }

```

Listing 第三章.30 opt.c: 符号集合成员检测辅助函数

把一个变量符号加入集合的头部，LICM 在收集环内被定义的变量时，遍历所有纯赋值/调用/输入等能定义 LHS 的指令，通过 `set_add(&defsInLoop, i->a)` 把左值变量加入集合。

```

1 static void set_add(SymSetNode **h, SYM *s)
2 {
3     if (!s)
4         return;
5     if (s->type != SYM_VAR)
6         return;
7     if (set_has(*h, s))
8         return;
9     SymSetNode *n = (SymSetNode *)malloc(sizeof(SymSetNode));
10    n->s = s;
11    n->next = *h;
12    *h = n;
13 }

```

Listing 第三章.31 opt.c: 符号集合添加元素辅助函数

LICM 的完整核心逻辑，按如下步骤工作：

首先调用 `cfg_build_only()` 建 CFG，然后用 `compute_dominators` 得到支配矩阵与块数 `n`。若 `n <= 0`，释放并退出。遍历每个块 `b` 的两个后继 (`succ1/succ2`)，若某后继 `h` 满足 `dom[b->id][h] == 1`，则 `b -> h` 是回边。对这样的回边，用 `build_natural_loop(h, b->id, n)` 构造自然环。若环头 `header` 或其首指令不存在，则跳过；否则准备环头与可能的前序块 (`preheader`) 上下文：如果环头的第一条是 `TAC_LABEL`，取其符号作为 `headerLabelSym`；其他与 `preheader` 相关的变量 (`preLabelTac/preLabelSym/ins/preheader_built`) 初始化为空。

收集环内所有被定义的变量到集合 `defsInLoop`，定义范围包含算术、比较、NEG、COPY、INPUT、CALL 等，把它们加入集合。用于后续不变性判断（检查操作数是否在环内被定义过）。遍历环内每个块 `bb` 的每条指令 `i2`，分两类处理：

第一类：外提临时变量的纯赋值。条件依次为：指令为纯赋值且 LHS 是临时变量；该 LHS 在整个环内只有一次真实定义；在当前块中该 LHS 不存在定义前使用；该计算的所有操作数在环内未被定义（或对一元/COPY 判断右值不被定义）。满足时触发外提：若尚未构建 `preheader`，则构建一个：创建新标签符号与 `TAC_LABEL` 节点并插到环头前，将所有环外指向原环头标签的 `GOTO/IFZ` 重定向到新标签，同时保留回边不变；将插入点 `ins` 设为新标签，并标记已构建。在 `ins` 之后插入一条新临时变量的 `VAR` 声明与一条等价计算（把临时作为结果），并更新 `ins` 指向新插入的尾。在环内所有块的所有指令（除当前要外提的 `i2`）上，调用 `replace_uses_in_tac` 把对原 LHS 的使用替换为新临时。将原指令 `i2` 从全局

TAC 双向链表摘除，同时维护 `tac_first/tac_last` 以及块的 `first/last` 边界，增加计数 `hoisted_count`，并继续遍历。

第二类：对非临时变量的完整赋值外提/下沉（形如 `d = invariantTemp/constant`）。条件为：指令是 `TAC_COPY`，且 LHS 是非临时变量；该 LHS 在环内真实定义次数为 1；块内不存在定义前使用该 LHS；右值不变（是常量或变量且不在环内被定义）。若满足则创建前序块（`preheader`）：在环头前插入一个新的 `TAC_LABEL` 并把来自环外指向环头标签的 `GOTO/IFZ` 重定向到该新标签（保留回边不动）；随后把 `TAC_COPY(i2->a, i2->b)` 直接插入到 `preheader`（不使用新临时变量）；最后将环内原始 `COPY` 指令从全局双向链表和块边界中摘除，并计入 `hoisted_count`。

每个回边处理完后，释放 `defsInLoop` 和环位集 `loop.in`。外层循环结束后，释放支配矩阵，返回成功外提的条数 `hoisted_count`。

具体实现代码过长，文档中就不再赘述，见随文压缩包中的 `opt.c` 文件。

以上为循环不变代码外提优化方法的实现思路与代码细节说明。

3.8 DCE

3.8.1 实现思路

死代码消除（DCE）的实现思路是先搞清哪些变量在程序执行里会被用到，再从后往前把不活跃的赋值指令安全地去除。

具体做法是先按基本块做一轮 **Live Variable Analysis**：每块里按照顺序收集 `use/def`，把块间的活跃信息通过后继传播成不动点（`out` 是后继 `in` 的并集，`in` 是 `use` 与 `out` 减去 `def` 的合并），得到每个块入口和出口的变量活跃集合。

然后进入消除阶段，对每个基本块从末尾向前扫描指令，遇到没有副作用的赋值或计算，就看它定义的左值在当前的 `live_after` 集合里是否还活跃——如果不活跃，说明这个结果之后不再会被使用，整条指令可以删掉；如果活跃，就把它的定义从活跃集合里去掉，并把该指令的所有使用操作数加回到活跃集合（相当于计算 `live_before = (live_after - defs) U uses`），继续向前推进。直到所有三地址码遍历完毕。

3.8.2 代码细节

判断一条三地址码指令是否具有副作用或控制流影响。返回值为 1 表示存在副作用或改变控制流（例如跳转、条件、返回、标签、函数边界、参数传递、调用、输入、输出），此类指令在 DCE 中绝不删除；只有无副作用的纯赋值类指令才会被考虑删除。

```
1 static int is_side_effect_op(int op)
2 {
3     switch (op)
4     {
5         case TAC_GOTO:
6         case TAC_IFZ:
7         case TAC_RETURN:
8         case TAC_LABEL:
9         case TAC_BEGINFUNC:
10        case TAC_ENDFUNC:
11        case TAC_ACTUAL:
12        case TAC_FORMAL:
13        case TAC_CALL:
14        case TAC_INPUT:
15        case TAC_OUTPUT:
16            return 1;
17        default:
18            return 0;
19    }
20 }
```

Listing 第三章.32 opt.c: 副作用指令判定辅助函数

针对变量符号指针的集合实现，提供成员检测、加入、删除、并集、复制、相等性比较、释放等操作。DCE 与活跃变量分析都使用这个集合来表示变量是否活跃。结构体设计、成员检测、加入在上一节 LICM 中已经提到过，就不再赘述。

```
1 static void set_remove(SymSetNode **h, SYM *s)
2 {
3     SymSetNode **pp = h;
4     while (*pp)
5     {
6         if ((*pp)->s == s)
7         {
8             SymSetNode *t = *pp;
9             *pp = t->next;
10            free(t);
11            return;
12        }
13        pp = &(*pp)->next;
```

```
14     }
15 }
16 static void set_union_into(SymSetNode **dst, SymSetNode *src)
17 {
18     for (; src; src = src->next)
19         set_add(dst, src->s);
20 }
21 static SymSetNode *set_copy(SymSetNode *h)
22 {
23     SymSetNode *r = NULL;
24     for (; h; h = h->next)
25         set_add(&r, h->s);
26     return r;
27 }
28 static int set_equal(SymSetNode *a, SymSetNode *b)
29 {
30     /* 先比较大小，再逐项比较成员是否相等 */
31     int ca = 0, cb = 0;
32     for (SymSetNode *t = a; t; t = t->next)
33         ca++;
34     for (SymSetNode *t = b; t; t = t->next)
35         cb++;
36     if (ca != cb)
37         return 0;
38     for (SymSetNode *t = a; t; t = t->next)
39         if (!set_has(b, t->s))
40             return 0;
41     return 1;
42 }
43 static void set_free(SymSetNode *h)
44 {
45     while (h)
46     {
47         SymSetNode *n = h->next;
48         free(h);
49         h = n;
50     }
51 }
```

Listing 第三章.33 opt.c: 符号集合操作辅助函数

计算单条指令的定义集（defs）与使用集（uses）。对于二元算术/比较，定义出现在 a，使用在 b/c；一元负号与复制定义在 a，使用在 b；输入与调用定义目标，条件使用 b，输出使用 a。这两个集合是活跃变量分析的基础，也是 DCE 在逆序过程中更新活跃集所需的当前指令的信息。

```
1  /* 计算一条 TAC 的 定义/使用 集合 */
2  static void tac_def_use(TAC *i, SymSetNode **defs, SymSetNode
   **uses)
3  {
4      switch (i->op)
5      {
6          case TAC_ADD:
7          case TAC_SUB:
8          case TAC_MUL:
9          case TAC_DIV:
10         case TAC_EQ:
11         case TAC_NE:
12         case TAC_LT:
13         case TAC_LE:
14         case TAC_GT:
15         case TAC_GE:
16             set_add(defs, i->a);
17             if (is_var(i->b))
18                 set_add(uses, i->b);
19             if (is_var(i->c))
20                 set_add(uses, i->c);
21             break;
22         case TAC_NEG:
23             set_add(defs, i->a);
24             if (is_var(i->b))
25                 set_add(uses, i->b);
26             break;
27         case TAC_COPY:
28             set_add(defs, i->a);
29             if (is_var(i->b))
30                 set_add(uses, i->b);
```

```

31     break;
32 case TAC_INPUT:
33     set_add(defs, i->a);
34     break;
35 case TAC_CALL:
36     if (i->a)
37         set_add(defs, i->a);
38     /* 参数已通过 ACTUAL 指令发出 */
39     break;
40 case TAC_IFZ:
41     if (is_var(i->b))
42         set_add(uses, i->b);
43     break;
44 case TAC_OUTPUT:
45     if (is_var(i->a))
46         set_add(uses, i->a);
47     break;
48 default:
49     break; /* 此处忽略
          VAR/FORMAL/ACTUAL/LABEL/GOTO/BEGIN/END/RETURN */
50 }
51 }

```

Listing 第三章.34 opt.c: 指令 defs/uses 计算辅助函数

基本块级别的 use/def 集合。顺序遍历块内所有指令，借助 tac_def_use 得到单条的集合，再将在定义之前出现的使用累计到块的 use 集中（若某变量已在 def 中出现，则该使用不计入 use），并把所有定义累加到块的 def。

```

1 /* 基本块级别的 use/def 计算 */
2 static void block_use_def(Block *b, SymSetNode **use, SymSetNode
   **def)
3 {
4     *use = NULL;
5     *def = NULL;
6     for (TAC *i = b->first;; i = i->next)
7     {
8         SymSetNode *d = NULL, *u = NULL;
9         tac_def_use(i, &d, &u);
10        /* 仅当使用在定义之前时才记入 use */

```

```

11     for (SymSetNode *t = u; t; t = t->next)
12         if (!set_has(*def, t->s))
13             set_add(use, t->s);
14     for (SymSetNode *t = d; t; t = t->next)
15         set_add(def, t->s);
16     if (i == b->last)
17         break;
18 }
19 }

```

Listing 第三章.35 opt.c: 基本块 use/def 计算辅助函数

块级活跃变量分析的核心。不动点迭代依据公式： $out[b] = \bigcap in[succ]$ （后继入口的并集）， $in[b] = use[b] \cup (out[b] - def[b])$ （出集去掉本块定义再并上本块使用）。代码中 new_out 通过并入两个后继的 in，new_in 由 use 复制后按 out - def 增补；使用 set_equal 监测收敛。最终得到每个块的 IN/OUT 集合，是 DCE 初始化块末尾活跃集的依据。

```

1  /* ----- 活跃变量分析（反向，按基本块）
   * ----- */
2  typedef struct
3  {
4      SymSetNode *in, *out, *use, *def;
5  } LV;
6
7  static LV *live_vars_compute(int *nblocks)
8  {
9      int maxId = -1;
10     for (Block *b = cfg_blocks_head(); b; b = b->next)
11         if (b->id > maxId)
12             maxId = b->id;
13     *nblocks = maxId + 1;
14     LV *lv = (LV *)calloc(*nblocks, sizeof(LV));
15     for (Block *b = cfg_blocks_head(); b; b = b->next)
16     {
17         block_use_def(b, &lv[b->id].use, &lv[b->id].def);
18     }
19     int changed = 1;
20     while (changed)

```

```

21 {
22     changed = 0;
23     for (Block *b = cfg_blocks_head(); b; b = b->next)
24     {
25         SymSetNode *new_out = NULL;
26         if (b->succ1 >= 0)
27             set_union_into(&new_out, lv[b->succ1].in);
28         if (b->succ2 >= 0)
29             set_union_into(&new_out, lv[b->succ2].in);
30         SymSetNode *new_in = set_copy(lv[b->id].use);
31         /* 计算 out - def */
32         for (SymSetNode *t = new_out; t; t = t->next)
33             if (!set_has(lv[b->id].def, t->s))
34                 set_add(&new_in, t->s);
35         if (!set_equal(new_in, lv[b->id].in) ||
36             !set_equal(new_out, lv[b->id].out))
37         {
38             set_free(lv[b->id].in);
39             set_free(lv[b->id].out);
40             lv[b->id].in = new_in;
41             lv[b->id].out = new_out;
42             changed = 1;
43         }
44         else
45         {
46             set_free(new_in);
47             set_free(new_out);
48         }
49     }
50     return lv;
51 }

```

Listing 第三章.36 opt.c: 块级活跃变量分析主过程

DCE 主过程。对每个块，先将 live 初始化为块的 OUT（代表当前指令之后活跃的变量），然后逆序扫描指令。若指令无副作用且其定义的所有变量都不在 live 中（没有后续使用），则删除指令，并维护全局双向链表与块的首尾边界以保持结构一致；

否则根据后向方程更新活跃集：先从 live 中移除本指令的定义变量（旧值被覆盖不再需要），再并入本指令使用变量（为了执行当前指令，之前位置必须呈现这些变量为活跃），继续向前。块扫描结束释放临时活跃集；

最后释放块级活跃分析结构，保证以逐指令的活跃信息进行删除判定，避免误删或误保留。

```

1  /* ----- 死代码消除 (DCE) ----- */
2  void opt_dead_code_elim(void)
3  {
4      cfg_build_only();
5      int n = 0;
6      LV *lv = live_vars_compute(&n);
7      for (Block *b = cfg_blocks_head(); b; b = b->next)
8      {
9          /* 初始化: live_after = OUT[b] */
10         SymSetNode *live = set_copy(lv[b->id].out);
11         /* 逆序遍历块中的指令 */
12         TAC *i = b->last;
13         while (i)
14         {
15             TAC *prev = i->prev; /* 在可能删除当前指令之前保存前驱 */
16             /*
17              * SymSetNode *defs = NULL, *uses = NULL;
18              * tac_def_use(i, &defs, &uses);
19              * int can_remove = 0;
20              * if (!is_side_effect_op(i->op) && defs)
21              * {
22              *     /* 若 defs 中的变量在 live_after 中都不活跃 */
23              *     int any_live = 0;
24              *     for (SymSetNode *t = defs; t; t = t->next)
25              *         if (set_has(live, t->s))
26              *         {
27              *             any_live = 1;
28              *             break;
29              *         }
30              *     if (!any_live)
31              *         can_remove = 1;
32              * }
33             if (can_remove)

```

```

33     {
34         /* 从全局 TAC 双向链表中摘除当前指令 */
35         if (i->prev)
36             i->prev->next = i->next;
37         else
38             tac_first = i->next;
39         if (i->next)
40             i->next->prev = i->prev;
41         else
42             tac_last = i->prev;
43         /* 若影响块边界, 进行修正 */
44         if (b->first == i)
45             b->first = i->next;
46         if (b->last == i)
47             b->last = i->prev;
48         /* 是否释放节点? 为避免其他结构悬挂, 保持分配状态 */
49         /*
50         */
51     }
52     else
53     {
54         /* live_before = (live_after - defs) U uses */
55         for (SymSetNode *t = defs; t; t = t->next)
56             set_remove(&live, t->s);
57         set_union_into(&live, uses);
58     }
59     if (i == b->first)
60         break;
61     i = prev;
62 }
63 set_free(live);
64 /* 清理临时集合 */
65 for (int i = 0; i < n; i++)
66 {
67     set_free(lv[i].in);
68     set_free(lv[i].out);
69     set_free(lv[i].use);
70     set_free(lv[i].def);

```

```

70     }
71     free(lv);
72 }

```

Listing 第三章.37 opt.c: 死代码消除主过程

以上为死代码消除优化方法的实现思路与代码细节说明。

3.9 LRU

3.9.1 实现思路

寄存器优化方面，本项目使用的是 LRU（最久未使用）的寄存器分配算法。

实现思路是：用一个寄存器描述表维护寄存器里当前是谁、是否被修改、最近一次使用的时间戳这三种信息，在三地址码逐条翻译时，每次寄存器获取都会使用该 LRU 策略，每次需要把某个符号装入寄存器时，先查寄存器里是否已缓存该符号（命中就直接复用并刷新最近使用时间），否则先找空闲寄存器（快速装载），还没有空闲就优先挑选未被修改的最久未使用寄存器（因为未修改意味着它与内存一致，替换不会引入额外的写回），再不行才选择全局最久未使用寄存器并执行溢出写回，最后把目标符号从内存或常量池加载进来。

3.9.2 代码细节

`rdesc` 是每个寄存器的三元状态：绑定的符号 `var`（为空则表示该寄存器空闲）、修改标记 `mod`（为 1 表示需要写回）、最近使用时间戳 `last_use`（用于做 LRU 的比较）。LRU 的核心即通过维护并比较 `last_use` 来选择最久未使用的寄存器作为候选。

```

1 struct rdesc /* 寄存器描述符 */
2 {
3     struct sym *var; /* 当前寄存器中的变量 */
4     int mod;         /* 是否被修改（脏） */
5     int last_use;    /* 最近使用计数（用于 LRU） */
6 };

```

Listing 第三章.38 codegen.c: 寄存器描述结构

全局使用计数器 `reg_use_counter`。每次寄存器被使用（读取/写回/装载/填充）都会更新寄存器 `last_use`，为其 `++reg_use_counter`，形成单调递增的时间戳，从而支持 LRU 的选择。

```

1 static int reg_use_counter = 0; /* 全局寄存器使用计数器 */

```

Listing 第三章.39 codegen.c: 全局寄存器使用计数器

使用内联函数将寄存器 `r` 的最近使用时间戳更新为全局递增计数器值，确保每一次真正的使用都会刷新 LRU 时间戳，维持最近使用的语义。

```
1 static inline void touch_reg(int r)
2 {
3     if (r >= 0 && r < R_NUM)
4         rdesc[r].last_use = ++reg_use_counter;
5 }
```

Listing 第三章.40 codegen.c: 刷新寄存器最近使用时间戳辅助函数

主要修改点是在寄存器分配函数 `reg_alloc` 中，其余的寄存器加载、绑定填充函数中都会调用 `touch_reg` 刷新对应寄存器的时间戳，修改较少就不再赘述，这里主要来说说 `reg_alloc` 的实现。

LRU 策略的核心实现分四步，这四步保证在必须溢出时尽量选择代价最低的寄存器：

命中复用：若变量 `s` 已在某寄存器中，直接返回该寄存器并更新对应寄存器的时间戳。

空闲优先：若有空寄存器，调用 `asm_load` 装载并填充，再更新对应寄存器的时间戳。

未修改的 LRU：优先选择未被修改（UNMODIFIED）且最久未使用的寄存器作为候选，这样替换不会引起写回开销。

溢出写回 + LRU：如果都被修改，则选择全局最久未使用的寄存器，先 `asm_write_back` 写回，再 `asm_load` 装载新符号，最后填充并更新对应寄存器的时间戳。

```
1 int reg_alloc(SYM *s)
2 {
3     int r;
4
5     /* 若变量已在寄存器中，直接返回该寄存器 */
6     for (r = R_GEN; r < R_NUM; r++)
7     {
8         if (rdesc[r].var == s)
9         {
10             touch_reg(r);
```

```
11         return r;
12     }
13 }
14
15 /* 选择空寄存器 */
16 for (r = R_GEN; r < R_NUM; r++)
17 {
18     if (rdesc[r].var == NULL)
19     {
20         asm_load(r, s);
21         rdesc_fill(r, s, UNMODIFIED);
22         touch_reg(r);
23         return r;
24     }
25 }
26
27 /* 选择未修改的最久未使用寄存器 (LRU) */
28 int cand = -1;
29 int oldest = INT_MAX;
30 for (r = R_GEN; r < R_NUM; r++)
31 {
32     if (!rdesc[r].mod)
33     {
34         if (rdesc[r].last_use < oldest)
35         {
36             oldest = rdesc[r].last_use;
37             cand = r;
38         }
39     }
40 }
41 if (cand >= 0)
42 {
43     asm_load(cand, s);
44     rdesc_fill(cand, s, UNMODIFIED);
45     touch_reg(cand);
46     return cand;
47 }
48
```

```

49  /* 否则选择最久未使用寄存器进行溢出写回 (LRU) */
50  cand = -1;
51  oldest = INT_MAX;
52  for (r = R_GEN; r < R_NUM; r++)
53  {
54      if (rdesc[r].last_use < oldest)
55      {
56          oldest = rdesc[r].last_use;
57          cand = r;
58      }
59  }
60  if (cand < 0)
61      cand = R_GEN; /* fallback */
62  asm_write_back(cand);
63  asm_load(cand, s);
64  rdesc_fill(cand, s, UNMODIFIED);
65  touch_reg(cand);
66  return cand;
67 }

```

Listing 第三章.41 codegen.c: 寄存器分配函数 (LRU 实现)

以上为 LRU 寄存器分配算法的实现思路与代码细节说明。

3.10 Clean

在进行以上各种优化后，三地址码中可能会遗留一些无用的变量声明 (VAR 指令)。不影响程序语义，但会增加代码冗余与后续处理负担。因此在所有优化结束后，进行一次清理，删除所有未被使用的变量声明。

3.10.1 实现思路

先收集在非 VAR 指令中被使用到的用户变量符号，然后遍历整个 TAC 链表，删除那些声明了但未在任何指令使用的变量声明节点。

具体流程：第一段遍历所有 TAC 指令，跳过 TAC_VAR。对每条指令的操作数 a/b/c，凡是类型为 SYM_VAR 的都加入 used 集合，表示这些变量在实际代码中被引用过。第二段再次遍历 TAC 链表，定位所有 TAC_VAR 声明。对于每个声明，如果其符号不在 used 集合，就把对应的声明节点从全局双向链表中摘除，并正确维护前驱/后继、tac_first 和 tac_last，以保持链表一致性。最后释放临时的 used 集

合。

3.10.2 代码细节

```

1  /* ----- 删除未使用的变量声明 ----- */
2  /* 在所有转换后，可能有 TAC_VAR 声明的符号从未被剩余指令引用。*/
3  void opt_remove_unused_var_decls(void)
4  {
5      /* 收集所有在非 VAR 指令中被引用的 SYM_VAR */
6      SymSetNode *used = NULL;
7      for (TAC *t = tac_first; t; t = t->next)
8      {
9          if (t->op == TAC_VAR)
10             continue;
11          if (t->a && t->a->type == SYM_VAR)
12             set_add(&used, t->a);
13          if (t->b && t->b->type == SYM_VAR)
14             set_add(&used, t->b);
15          if (t->c && t->c->type == SYM_VAR)
16             set_add(&used, t->c);
17      }
18      /* 删除 used 集中不存在的变量声明 */
19      for (TAC *t = tac_first; t;)
20      {
21          TAC *next = t->next;
22          if (t->op == TAC_VAR && !set_has(used, t->a))
23          {
24              if (t->prev)
25                  t->prev->next = t->next;
26              else
27                  tac_first = t->next;
28              if (t->next)
29                  t->next->prev = t->prev;
30              else
31                  tac_last = t->prev;
32              /* no need to free symbol; declaration node discarded
33               */
34          }
35          t = next;

```

```
35     }  
36     set_free(used);  
37 }
```

Listing 第三章.42 opt.c: 无用变量声明清理主过程

第四章 测试

以上，功能拓展部分以及性能优化部分的实现过程、细节、思路均已说明，接下来将展示所有的拓展功能测试用例通过情况以及性能优化测试用例的跑分。

4.1 拓展功能测试用例通过情况

对于所有 `testcase` 目录下的测试用例，都需要进行下面的三道步骤：

- 编译生成三地址码以及汇编代码： `./mini xxx.m`
- 汇编代码编译至可执行文件： `./asm xxx.s`
- 运行可执行文件： `./machine xxx.o`

4.1.1 自动测试

由于手动逐个编译、输入、测试，检验输出结果较为繁琐，因此编写了一个自动化测试脚本，在其中提前设定输入，并比对预期输出与实际输出结果，能够快速得到所有测试用例的通过情况。

另外，由于功能拓展部分中编写 `function` 功能时将主函数也视为函数了，所以在所有的测试用例中为主函数添加了 `int` 返回值。

以下是测试脚本中规定的输入与预期输出：

```
1 # Default expectations (fallback when expected.json absent or jq
  not available)
2 default_stdin(){
3     case "$1" in
4         arr) echo "5";;
5         arr-while) echo "1";;
6         char|char-int) echo -e "Z\nA";;
7         for) echo "5";;
8         for-bc) echo "12";;
9         func-char) echo -e "A\nB";;
10        func-int) echo -e "7\n3";;
11        if) echo -e "2\n5";;
12        int) echo "5";;
13        ptr-char) echo -e "Z\nA";;
14        ptr-char-int) echo -e "K\n123";;
15        ptr-int) echo "5";;
```

```

16     struct) echo -e "1\n2\n3";;
17     switch) echo "3";;
18     while) echo "5";;
19     while-bc) echo "15";;
20     *) return 1;;
21     esac
22 }
23
24 default_expected_all(){
25     case "$1" in
26         arr) printf "%s\n" "17" ;;
27         arr-struct) printf "%s\n" "12ab" ;;
28         arr-while) printf "%s\n" "10987654321" ;;
29         char) printf "%s\n" "AbcZ" "B" ;;
30         char-int) printf "%s\n" "AbcZ" "65989990" ;;
31         for) printf "%s\n" "01234" ;;
32         for-bc) printf "%s\n" "012345678910break"
33             "0123456789continue11" ;;
34         func-char) printf "%s\n" "B C" ;;
35         func-int) printf "%s\n" "7 8" ;;
36         if) printf "%s\n" "not equal" "7" ;;
37         int) printf "%s\n" "515-5-150" ;;
38         ptr-char) printf "%s\n" "AbcZ" "AB" ;;
39         ptr-char-int) printf "%s\n" "K123" ;;
40         ptr-int) printf "%s\n" "515-5-150" "111222" ;;
41         struct) printf "%s\n" "103202301" ;;
42         struct-arr) printf "%s\n" "12ab" ;;
43         struct-ptr) printf "%s\n" "999" ;;
44         switch) printf "%s\n" "6" ;;
45         while) printf "%s\n" "01234" ;;
46         while-bc) printf "%s\n" "012345678910break" "11121314" ;;
47         *) return 1;;
48     esac
49 }

```

Listing 第四章.1 run_testcases.sh: 自动化测试脚本

脚本全文见随文压缩包中的 run_testcases.sh 文件。

运行脚本后，可以得到所有测试用例的通过情况以及实际输出，输出会保存

```
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ make clean
rm -fr *.l.* *.y.* *.s *.x *.o core mini asm machine
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ make
lex -o mini.l.c mini.l
yacc -d -v -Wcounterexamples -o mini.y.c mini.y
gcc -g3 main.c mini.l.c mini.y.c tac.c obj.c -o mini
lex -o asm.l.c asm.l
yacc -d -Wcounterexamples -o asm.y.c asm.y
gcc -g3 asm.l.c asm.y.c -o asm
gcc -g3 machine.c -o machine
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./run_testcases.sh
Build: make
make: Entering directory '/mnt/d/vs code project/challenge/mini-object-extend'
make: Nothing to be done for 'all'.
make: Leaving directory '/mnt/d/vs code project/challenge/mini-object-extend'
PASS arr-struct
PASS arr-while
PASS arr
PASS char-int
PASS char
PASS for-bc
PASS for
PASS func-char
PASS func-int
PASS if
PASS int
PASS ptr-char-int
PASS ptr-char
PASS ptr-int
PASS struct-arr
PASS struct-ptr
PASS struct
PASS switch
PASS while-bc
PASS while

Summary: 20/20 passed
```

图 4-1 拓展功能测试用例通过情况

到规定的 csv 文件中。

图4-1展示了所有拓展功能测试用例的通过情况，可以看到所有测试用例均已通过，

```
name,status,output
arr-struct,PASS,12ab
arr-while,PASS,10987654321
arr,PASS,17
char-int,PASS,AbcZ\n65989990
char,PASS,AbcZ\nB
for-bc,PASS,012345678910break\n0123456789continue11
for,PASS,01234
func-char,PASS,B C
func-int,PASS,7 8
if,PASS,not equal\n7
int,PASS,515-5-150
ptr-char-int,PASS,K123
ptr-char,PASS,AbcZ\nAB
ptr-int,PASS,515-5-150\n111222
struct-arr,PASS,12ab
struct-ptr,PASS,999
struct,PASS,103202301
switch,PASS,6
while-bc,PASS,012345678910break\n11121314
while,PASS,01234
```

图 4-2 拓展功能测试用例实际输出

图4-2展示了所有拓展功能测试用例的实际输出，可以看到实际输出均与预期输出一致，由于功能拓展不太关注跑分，所以筛去了跑分输出。

4.1.2 手动测试

完备性起见，这里手动测试所有的功能拓展测试用例，输入与测试结果与自动测试相同。

```
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./mini testcase/arr.m
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./asm testcase/arr.s
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./machine testcase/arr.o
5
17

-----
CLOCK CYCLES : 594
MUL DIV : 12
MEM READ : 6
MEM WRITE : 40
-----
```

图 4-3 arr.m

```
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./mini testcase/arr-while.m
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./asm testcase/arr-while.s
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./machine testcase/arr-while.o
1
10987654321

-----
CLOCK CYCLES : 3094
MUL DIV : 20
MEM READ : 85
MEM WRITE : 176
-----
```

图 4-4 arr-while.m

```
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./mini testcase/arr-struct.m
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./asm testcase/arr-struct.s
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./machine testcase/arr-struct.o
12ab

-----
CLOCK CYCLES : 1211
MUL DIV : 22
MEM READ : 11
MEM WRITE : 84
-----
```

图 4-5 arr-struct.m

```
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./mini testcase/char.m
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./asm testcase/char.s
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./machine testcase/char.o
Z
A
AbcZ
B

-----
CLOCK CYCLES : 191
MUL DIV : 0
MEM READ : 8
MEM WRITE : 8
-----
```

图 4-6 char.m

```
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./mini testcase/char-int.m
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./asm testcase/char-int.s
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./machine testcase/char-int.o
Z
A
AbcZ
65989990

-----
CLOCK CYCLES : 259
MUL DIV : 0
MEM READ : 12
MEM WRITE : 10
-----
```

图 4-7 char-int.m

```
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./mini testcase/for.m
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./asm testcase/for.s
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./machine testcase/for.o
5
01234

-----
CLOCK CYCLES : 530
MUL DIV : 0
MEM READ : 25
MEM WRITE : 20
-----
```

图 4-8 for.m

```
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./mini testcase/for-bc.m
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./asm testcase/for-bc.s
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./machine testcase/for-bc.o
12
012345678910break
0123456789continue11

-----
CLOCK CYCLES : 2769
MUL DIV : 0
MEM READ : 122
MEM WRITE : 100
-----
```

图 4-9 for-bc.m

```
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./mini testcase/func-char.m
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./asm testcase/func-char.s
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./machine testcase/func-char.o
A
B
B C

-----
CLOCK CYCLES : 299
MUL DIV : 0
MEM READ : 12
MEM WRITE : 14
-----
```

图 4-10 func-char.m

```
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./mini testcase/func-int.m
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./asm testcase/func-int.s
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./machine testcase/func-int.o
7
3
7 8

-----
CLOCK CYCLES : 298
MUL DIV : 0
MEM READ : 12
MEM WRITE : 14
-----
```

图 4-11 func-int.m

```
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./mini testcase/if.m
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./asm testcase/if.s
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./machine testcase/if.o
2
5
not equal
7

-----
CLOCK CYCLES : 157
MUL DIV : 0
MEM READ : 6
MEM WRITE : 7
-----
```

图 4-12 if.m

```
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./mini testcase/int.m
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./asm testcase/int.s
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./machine testcase/int.o
5
515-5-150

-----
CLOCK CYCLES : 178
MUL DIV : 1
MEM READ : 6
MEM WRITE : 9
-----
```

图 4-13 int.m

```
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./mini testcase/ptr-char.m
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./asm testcase/ptr-char.s
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./machine testcase/ptr-char.o
Z
A
AbcZ
AB

-----
CLOCK CYCLES : 257
MUL DIV : 0
MEM READ : 11
MEM WRITE : 11
-----
```

图 4-14 ptr-char.m

```
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./mini testcase/ptr-int.m
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./asm testcase/ptr-int.s
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./machine testcase/ptr-int.o
5
515-5-150
111222

-----
CLOCK CYCLES : 270
MUL DIV : 1
MEM READ : 9
MEM WRITE : 14
-----
```

图 4-15 ptr-int.m

```
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./mini testcase/ptr-char-int.m
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./asm testcase/ptr-char-int.s
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./machine testcase/ptr-char-int.o
K
123
K123

-----
CLOCK CYCLES : 228
MUL DIV : 0
MEM READ : 8
MEM WRITE : 12
-----
```

图 4-16 ptr-char-int.m

```
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./mini testcase/struct.m
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./asm testcase/struct.s
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./machine testcase/struct.o
1
2
3
103202301

-----
CLOCK CYCLES : 482
MUL DIV : 0
MEM READ : 13
MEM WRITE : 29
-----
```

图 4-17 struct.m

```
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./mini testcase/struct-arr.m
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./asm testcase/struct-arr.s
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./machine testcase/struct-arr.o
12ab

-----
CLOCK CYCLES : 974
MUL DIV : 14
MEM READ : 10
MEM WRITE : 68
-----
```

图 4-18 struct-arr.m

```
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./mini testcase/struct-ptr.m
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./asm testcase/struct-ptr.s
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./machine testcase/struct-ptr.o
999

-----
CLOCK CYCLES : 379
MUL DIV : 4
MEM READ : 4
MEM WRITE : 27
-----
```

图 4-19 struct-ptr.m

```
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./mini testcase/switch.m
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./asm testcase/switch.s
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./machine testcase/switch.o
3
6

-----
CLOCK CYCLES : 134
MUL DIV : 0
MEM READ : 4
MEM WRITE : 7
-----
```

图 4-20 switch.m

```
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./mini testcase/while.m
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./asm testcase/while.s
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./machine testcase/while.o
5
01234

-----
CLOCK CYCLES : 480
MUL DIV : 0
MEM READ : 20
MEM WRITE : 20
-----
```

图 4-21 while.m

```

miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./mini testcase/while-bc.m
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./asm testcase/while-bc.s
miki@MY:/mnt/d/vs code project/challenge/mini-object-extend$ ./machine testcase/while-bc.o
15
012345678910break
11121314

-----
CLOCK CYCLES : 1504
MUL DIV : 0
MEM READ : 50
MEM WRITE : 65
-----

```

图 4-22 while-bc.m

4.2 性能优化测试用例跑分

4.2.1 自动测试

性能优化部分与功能拓展部分相同，都需要经过三个步骤，故同样编写了自动化测试脚本，由于性能优化部分主要关注跑分且测试用例不多，故将所有测试用例的输出单独生成为一个文件，以便查看。对于 `nested_loop.m`，由于回车输出导致的输出窗口太大不方便记录，所以将输出中的回车替换为空格，并在外层循环回车分割每次内层循环的输出内容。

性能优化部分的测试用例只有 `loop.m` 需要输入且输入固定为 1, 2, 3，所以就不再特地展示脚本内容，对于每一个测试用例查看输出文件来比对预期输出与实际输出。脚本全文见随文压缩包中的 `run_all_tests.sh` 文件。

图4-23展示了所有性能优化测试用例进行测试。

```

miki@MY:/mnt/d/vs code project/challenge/mini-object-optimized$ make clean
rm -fr *.l.* *.y.* *.s *.x *.o core mini asm machine
miki@MY:/mnt/d/vs code project/challenge/mini-object-optimized$ make
lex -o mini.l.c mini.l
yacc -d -o mini.y.c mini.y
gcc -g3 main.c mini.l.c mini.y.c tac.c obj.c cfg.c opt.c -o mini
lex -o asm.l.c asm.l
yacc -d -o asm.y.c asm.y
gcc -g3 asm.l.c asm.y.c -o asm
gcc -g3 machine.c -o machine
miki@MY:/mnt/d/vs code project/challenge/mini-object-optimized$ ./run_all_tests.sh
[RUN] loop
[DONE] loop -> /mnt/d/vs code project/challenge/mini-object-optimized/results/loop.out.txt
[RUN] nested_loop
[DONE] nested_loop -> /mnt/d/vs code project/challenge/mini-object-optimized/results/nested_
loop.out.txt
[RUN] one_while
[DONE] one_while -> /mnt/d/vs code project/challenge/mini-object-optimized/results/one_while
.out.txt
[RUN] three_while
[DONE] three_while -> /mnt/d/vs code project/challenge/mini-object-optimized/results/three_w
hile.out.txt
\nAll tests finished. Outputs are in: /mnt/d/vs code project/challenge/mini-object-optimized
/results

```

图 4-23 性能优化测试用例测试

下面展示四个性能优化测试用例的整体输出结果：

```
5987654321 4987654321 3987654321 2987654321 1987654321
14 15

-----
CLOCK CYCLES : 3247
MUL DIV : 3
MEM READ : 125
MEM WRITE : 122
-----
```

图 4-24 loop.m.output

```
45360 106920

-----
CLOCK CYCLES : 387243
MUL DIV : 3
MEM READ : 20021
MEM WRITE : 13739
-----
```

图 4-25 three_while.m.output

```
maze is fun!

-----
CLOCK CYCLES : 12336
MUL DIV : 12
MEM READ : 416
MEM WRITE : 431
-----
```

图 4-26 one_while.m.output

```

9  11  13  15  17  19  21  23  25  27
9  11  13  15  17  19  21  23  25  27
9  11  13  15  17  19  21  23  25  27
9  11  13  15  17  19  21  23  25  27
9  11  13  15  17  19  21  23  25  27
9  11  13  15  17  19  21  23  25  27
9  11  13  15  17  19  21  23  25  27
9  11  13  15  17  19  21  23  25  27
9  11  13  15  17  19  21  23  25  27
9  11  13  15  17  19  21  23  25  27
-----
CLOCK CYCLES : 8935
MUL DIV : 0
MEM READ : 333
MEM WRITE : 344
-----

```

图 4-27 nested loop.m.output

可以看到，首先所有测试用例的输出与预期输出一致，其次都展示了性能指标，包括时钟周期数、乘除次数、读内存与写内存数。

4.2.2 手动测试

完备性起见，这里手动测试所有的性能优化测试用例，输入与测试结果与自动测试相同。

```
miki@MY:/mnt/d/vs code project/challenge/mini-object-optimized$ ./mini testcase/loop.m
miki@MY:/mnt/d/vs code project/challenge/mini-object-optimized$ ./asm testcase/loop.s
miki@MY:/mnt/d/vs code project/challenge/mini-object-optimized$ ./machine testcase/loop.o
1
2
3
5987654321 4987654321 3987654321 2987654321 1987654321
14 15

-----
CLOCK CYCLES : 3247
MUL DIV : 3
MEM READ : 125
MEM WRITE : 122
-----
```

图 4-28 loop.m

```
miki@MY:/mnt/d/vs code project/challenge/mini-object-optimized$ ./mini testcase/three_while.m
miki@MY:/mnt/d/vs code project/challenge/mini-object-optimized$ ./asm testcase/three_while.s
miki@MY:/mnt/d/vs code project/challenge/mini-object-optimized$ ./machine testcase/three_while.o
45360 106920

-----
CLOCK CYCLES : 387243
MUL DIV : 3
MEM READ : 20021
MEM WRITE : 13739
-----
```

图 4-29 three_while.m

```
miki@MY:/mnt/d/vs code project/challenge/mini-object-optimized$ ./mini testcase/one_while.m
miki@MY:/mnt/d/vs code project/challenge/mini-object-optimized$ ./asm testcase/one_while.s
miki@MY:/mnt/d/vs code project/challenge/mini-object-optimized$ ./machine testcase/one_while.o
maze is fun!

-----
CLOCK CYCLES : 12336
MUL DIV : 12
MEM READ : 416
MEM WRITE : 431
-----
```

图 4-30 one_while.m

```
miki@MY:/mnt/d/vs code project/challenge/mini-object-optimized$ ./mini testcase/nested_loop.m
miki@MY:/mnt/d/vs code project/challenge/mini-object-optimized$ ./asm testcase/nested_loop.s
miki@MY:/mnt/d/vs code project/challenge/mini-object-optimized$ ./machine testcase/nested_loop.o
9 11 13 15 17 19 21 23 25 27
9 11 13 15 17 19 21 23 25 27
9 11 13 15 17 19 21 23 25 27
9 11 13 15 17 19 21 23 25 27
9 11 13 15 17 19 21 23 25 27
9 11 13 15 17 19 21 23 25 27
9 11 13 15 17 19 21 23 25 27
9 11 13 15 17 19 21 23 25 27
9 11 13 15 17 19 21 23 25 27
9 11 13 15 17 19 21 23 25 27
9 11 13 15 17 19 21 23 25 27
-----
CLOCK CYCLES : 8935
MUL DIV : 0
MEM READ : 333
MEM WRITE : 344
-----
```

图 4-31 nested_loop.m

第五章 总结

本项目在原有 miniC 编译器的基础上，完成了功能拓展与性能优化两大部分内容。功能拓展部分实现了数组、字符类型、for 循环、函数调用、if 语句、指针类型、结构体类型、switch 语句等功能，并通过了所有相应的测试用例，达到了预期目标。性能优化部分实现了循环不变代码外提、死代码消除、LRU 寄存器分配算法等优化方法，并通过了所有相应的测试用例，达到了预期目标。

在实现过程中，遇到了一些挑战，例如对复杂语法结构的支持、优化算法的正确性保证等，但通过反复调试，查找工具克服了这些困难。

由于时间原因以及能力有限，实际上还有可以扩展与完善的部分，例如语法上来讲，扩展的编译器实际上不支持赋初值，包括变量赋初值以及数组赋初值。优化方面，公共子表达式删除并没有实际替换为用户变量，而是改为对表达式左值的 COPY，主要是因为实现简单，另一个原因是直接替换用户变量可能会引入新的变量依赖，影响到后续的优化，例如寄存器分配，还可能扰乱控制流导致一些意想不到的问题。还有 LICM 的不变代码外提使用了非常严格的条件外提，一方面是因为不论如何都不想改变语义，另一方面是因为实现难度较大，时间有限未能实现更复杂的版本。除此之外，还有死代码消除没有消除无用的 label，可能会出现无意义的跳转。寄存器优化也实现的较为简单，能够发挥有限的效用。最后就是回过头发现功能拓展部分由于完全没有考虑效率，写的重复意义上的代码以及结构设计较多，实际上可以进行精简。

总的来说，对于要求的功能板块以及性能优化效果都成功实现与达成，仍然有优化与扩展空间，希望在未来的学习与工作中持续提升编译器相关的知识与技能。